SERVICIO NACIONAL DE APRENDIZAJE

SENA · Centro de Formación de Talento Humano en Comercio y Servicios

Proyecto Formativo — Sistema Punto de Venta & Facturación

MANUAL TÉCNICO

Especificación técnica completa del sistema

SISTEMA PUNTO DE VENTA Y FACTURACIÓN

D'Piero Restaurante & Parrilla — Parrilla · Bar & Cocina a la Brasa

DATO DE LA ENTREGA	INFORMACIÓN
Empresa (cliente)	D'Piero Restaurante & Parrilla — Bogotá, Colombia
Proyecto	Sistema POS D'Piero — Punto de Venta, Comandas y Facturación
Aprendiz autor	Santiago Andrés Vélez Naranjo (AndresVlz2020)
Equipo de trabajo	Santiago Andrés Vélez Naranjo · Colaborador: Henao420
Programa de formación	Tecnólogo en Análisis y Desarrollo de Software — Proyecto Formativo
Ficha del proyecto formativo	3172293
Repositorio	github.com/AndresVlz2020/pos_project (ramas: main, dev)
Fecha de elaboración	Noviembre de 2026
Versión del documento	1.1

Versión 1.1 · Noviembre 2026 · Documento Word (.docx)

Control de versiones

Historial de cambios del documento. Cada versión incrementa el número menor para borradores internos y el número mayor para entregas formales.

Versión	Fecha	Descripción del cambio	Responsable
0.1	2026-09-25	Borrador inicial a partir de la I revisión de avance del proyecto (informe administrativo y manuales).	Santiago Andrés Vélez Naranjo
0.5	2026-10-15	Reestructuración según rúbrica de la actividad de aprendizaje; levantamiento de información del código fuente (backend, frontend, base de datos).	Santiago Andrés Vélez Naranjo
0.9	2026-11-04	Segunda revisión de avance; verificación del funcionamiento real del sistema (backend, frontend, PostgreSQL) y captura de evidencias gráficas.	Santiago Andrés Vélez Naranjo
1.0	2026-11-08	Versión final: contenido integral verificado contra el código, las migraciones SQL y la operación del sistema; incorporación de diagramas y métricas de calidad.	Santiago Andrés Vélez Naranjo
1.1	2026-11-08	Reparación de seguridad tras verificación CRUD y de variables: todas las mutaciones de la API protegidas con JWT + permisos RBAC, BUG-01 corregido, CORS restringido a lista blanca, JWT_SECRET rotado y validado al arranque, actualización parcial de usuarios reparada (ya no devuelve 500), variables de entorno consolidadas con .env.example y servicios frontend centralizados en VITE_API_URL. Verificación integral repetida: 31/31 aserciones conformes.	Santiago Andrés Vélez Naranjo

Tabla 1 — Control de versiones del manual técnico

Información general del documento

Campo	Detalle
Nombre del sistema	D'Piero POS — Sistema Punto de Venta & Facturación
Empresa cliente	D'Piero Restaurante & Parrilla (Parrilla, Bar & Cocina a la Brasa)
Tipo de solución	Aplicación web (SPA) — Escritorio / Web (navegador)
Arquitectura	Cliente–servidor en 3 capas: React SPA → API REST Express → PostgreSQL
Usuarios objetivo	Administrador, Cajero, Mesero y Cocina (roles RBAC)
Clasificación	Uso interno del proyecto formativo — Distribución restringida al equipo
Norma de elaboración	ISO/IEC/IEEE 26514:2022 (documentación de usuario) e ISO/IEC 25010
Formato de entrega	Microsoft Word (.docx) — Norma APA 7.ª edición (referencias)

Tabla 2 — Ficha técnica de identificación del documento

Aviso de confidencialidad y uso responsable

El presente documento y la información que contiene —incluyendo direcciones internas, credenciales de ejemplo, esquema de base de datos y detalles de seguridad— son de uso exclusivo del equipo del proyecto formativo y de D'Piero Restaurante & Parrilla. Queda prohibida su reproducción total o parcial sin autorización escrita. Las credenciales de demostración citadas en este manual deben ser reemplazadas antes de cualquier puesta en producción real.

Este documento se elaboró siguiendo el principio de veracidad: cada afirmación sobre el sistema fue verificada directamente sobre el código fuente (commit f58187b, rama dev), las 27 migraciones de base de datos aplicadas sobre la base `pos_project`, y la ejecución real del sistema en el entorno de desarrollo.

Tabla de Contenido

1. Propósito 10
1.1 Audiencia y nivel técnico 10
1.2 Convenciones tipográficas 10
2. Información General 11
2.1 Usuarios del sistema y roles 11
2.2 Tecnologías 11
3. Arquitectura 14
3.1 Flujo de comunicación Frontend ↔ Backend ↔ Base de datos 14
3.2 APIs que intervienen 16
3.3 Decisiones arquitectónicas (ADR resumidos) 17
4. Estructura del Código Fuente 18
4.1 Árbol de carpetas y archivos 18
4.2 Descripción de los módulos o aplicaciones 19
4.3 Función de los archivos principales 20
4.4 Ubicación de modelos, vistas, controladores, servicios y componentes 21
4.5 Convenciones de nombres y organización 22
5. Módulos y Responsabilidades 23
6. Modelo de Datos 24
6.1 Modelo entidad-relación / modelo relacional 24
6.2 Diccionario de datos (completo) 25
6.3 Claves primarias y foráneas (índice) 32
6.4 Relaciones y restricciones 32
6.5 Consultas, procedimientos, funciones o triggers relevantes 33
6.6 Estrategia de respaldo y restauración 36
7. Descripción Técnica de los Módulos 37
7.1 Módulo de autenticación (auth) 37
7.2 Módulo de usuarios (users) 38
7.3 Módulo de productos (products) 39
7.4 Módulo de inventario (inventory) 40
7.5 Módulo de proveedores (suppliers) 40
7.6 Módulo de comandas / órdenes (orders) 41
7.7 Módulo de accesos RBAC (access) 42
7.8 Módulo de grupos (groups) 43
7.9 Módulo de permisos (permissions) 43
7.10 Módulo POS frontend (home) 44
7.11 Matriz de operaciones CRUD por endpoint 44
8. Interfaces y Servicios de Comunicación 45

8.1 Catálogo completo de endpoints (38)....45
8.2 Parámetros de entrada y formato de respuestas....48
8.3 Códigos de estado HTTP utilizados....49
8.4 Autenticación y autorización....50
8.5 Ejemplos de solicitudes y respuestas....51
9. Seguridad Técnica....53
9.1 Gestión de usuarios, roles y permisos....53
9.2 Autenticación....54
9.3 Protección de contraseñas....54
9.4 Validación de datos de entrada....55
9.5 Protección frente a riesgos comunes (OWASP Top 10)....56
9.6 Manejo de información sensible y configuraciones de seguridad....56
10. Configuración y Despliegue Técnico....58
10.1 Variables de entorno....58
10.2 Configuración de desarrollo y producción....59
10.3 Dependencias (congeladas por package-lock)....60
10.4 Proceso de despliegue (paso a paso verificado)....60
10.5 Configuración de servidores y servicios....60
10.6 Consideraciones para actualizar la aplicación....61
11. Mantenimiento y Solución de Errores....61
11.1 Procedimientos de mantenimiento preventivo y correctivo....61
11.2 Diagnóstico de errores....62
11.3 Ubicación de registros / logs....62
11.4 Actualización de dependencias....63
11.5 Procedimientos de respaldo (resumen)....63
11.6 Recuperación ante fallos....63
12. Pruebas y Calidad....64
12.1 Estrategia de pruebas implementada....64
12.2 Pruebas unitarias, de integración o funcionales....64
12.3 Resultados de las pruebas....64
12.4 Evidencias de ejecución....65
12.5 Problemas conocidos y limitaciones....66
12.6 Criterios de aceptación verificados....67
13. Runbook de Incidentes en Producción....67
13.1 Árbol de decisión ante incidentes....67
13.2 Rotación de secretos y credenciales....68
13.3 Checklist de post-incidente....68
14. Recorrido Guiado del Código — Dos Flujos Completos....68
14.1 Flujo A — Crear una comanda: de la UI a PostgreSQL y de regreso....68

14.2 Flujo B — Verificación de permiso desde la UI hasta la BD....71
14.3 Diagrama de flujo técnico — recuperación de contraseña....73
Anexo A — .env.example (plantilla sin secretos)....74
Anexo B — Script de migraciones (migrate.js) — explicación....74
Anexo C — Referencia rápida de comandos....75
Anexo D — Evidencias de ejecución (capturas)....76
Anexo E — Glosario técnico....92
Anexo G — Extractos SQL comentados de las migraciones clave....93
G.1 0001_create_users_table.sql — el origen....93
G.2 0010_update_users_table.sql — adaptación al negocio real....94
G.3 0026_seed_groups_default_permissions_and_user_groups.sql — la matriz RBAC....94
G.4 0027_seed_initial_pos_data.sql — datos reales del restaurante....94
Anexo H — Bitácora de cambios de la aplicación (2026-05-11 → 2026-11-08)....95
15. Preguntas Frecuentes Técnicas (FAQ de desarrollo)....96
Anexo I — Consultas SQL operativas para el administrador de BD....98
I.1 Auditoría diaria del negocio....98
I.2 Integridad del RBAC....98
I.3 Mantenimiento higiénico....99
Anexo J — Checklist de liberación (release)....99
Anexo F — Índice de referencias al código (archivo → sección del manual)....100

Tabla de Ilustraciones

Figura 1 — Arquitectura en capas con las tecnologías de cada capa....15
Figura 2 — Diagrama de componentes con interfaces expuestas y consumidas....17
Figura 3 — Modelo entidad-relación — subgrafos RBAC, operación y abastecimiento....24
Figura 4 — Secuencia de autenticación (cliente → Express → servicio → repositorio → BD)....50
Figura 5 — Matriz grupo × permiso (21×4) — valores semilla....54
Figura 6 — Flujo completo de recuperación de contraseña con código de 6 dígitos y vigencia de 15 minutos....7
Figura 7 — Login de acceso rápido — perfiles visibles....76
Figura 8 — Login administrativo con bloqueo a 3 intentos....77
Figura 9 — Teclado PIN de 4 dígitos....77
Figura 10 — Solicitud de código de recuperación....78
Figura 11 — Verificación de código y nueva contraseña....78
Figura 12 — POS — Catálogo & Comandas con datos reales....79
Figura 13 — Listado de comandas ORD-001..007....80
Figura 14 — Listado de productos con stock real....80
Figura 15 — POS — pestaña Personal en Turno con roles y horarios....81
Figura 16 — POS — pestaña Proveedores con datos de contacto reales....81
Figura 17 — POS — pestaña Clientes & Facturación (demo)....82
Figura 18 — Listado de proveedores (26 registros)....83
Figura 19 — Edición de comanda (mesa, mesero, estado, ítems)....83
Figura 20 — Formulario de creación de platillo....84
Figura 21 — Ficha de solo lectura del platillo....84
Figura 22 — Edición de producto (estado Habilitado)....85
Figura 23 — Inventario con alerta de mínimo (INV-004)....85
Figura 24 — Alta de insumo con lote, vencimiento y proveedor....86
Figura 25 — Ficha completa del proveedor (Avícola y Carnes del Valle)....86
Figura 26 — Alta de proveedor con NIT único....87
Figura 27 — Alta de usuario operativo con PIN autogenerado....87
Figura 28 — Edición de usuario (Carlos Mesero)....88
Figura 29 — Ficha del usuario (información completa)....88
Figura 30 — Gestión de permisos RBAC (grupos y usuario individual)....89
Figura 31 — Matriz de permisos por módulo en /dashboard/access....89
Figura 32 — Modal de generación de reportes PDF/Excel....90
Figura 33 — Listado de usuarios con acciones por fila....90
Figura 34 — Creación de comanda con disponibilidad de mesas....91
Figura 35 — Detalle de comanda en modal con ítems y total....91
Figura 36 — BUG-01 reproducido: crash de /editsupplier....92

Lista de Tablas

Tabla 1 — Control de versiones del manual técnico....2
Tabla 2 — Ficha técnica de identificación del documento....2
Tabla 3 — Secciones clave por perfil de lector....10
Tabla 4 — Identificación general del sistema....11
Tabla 5 — Usuarios semilla y permisos por rol (migraciones 0011, 0026)....11
Tabla 6 — Lenguajes utilizados y su versión/estándar....12
Tabla 7 — Dependencias del backend con versiones instaladas (node_modules)....12
Tabla 8 — Dependencias del frontend con versiones instaladas....13
Tabla 9 — Sistema gestor de base de datos y su configuración....13
Tabla 10 — Servicios externos e integraciones con su criticidad....14
Tabla 11 — Registros de decisiones arquitectónicas (ADR) resumidos....17
Tabla 12 — Módulos backend con responsabilidad y tamaño....20
Tabla 13 — Función de los archivos principales del sistema....21
Tabla 14 — Dónde vive cada artefacto arquitectónico....21
Tabla 15 — Matriz módulo × usuario funcional × encargado técnico....23
Tabla 16 — Resumen cuantitativo del modelo de datos....25
Tabla 17 — Índice de claves primarias y foráneas con su política de borrado....32
Tabla 18 — Elementos respaldados con frecuencia y retención....36
Tabla 19 — Objetivos de recuperación de desastre (RPO/RTO)....37
Tabla 20 — Operaciones CRUD expuestas por recurso (ningún recurso admite DELETE físico)....45
Tabla 21 — Inventario completo de 38 endpoints con autenticación y respuestas....48
Tabla 22 — Estado real de protección tras el endurecimiento: toda escritura exige JWT + permiso; las lecturas de operación quedan públicas para el POS....8
Tabla 23 — Códigos de estado HTTP y su uso concreto en la API....49
Tabla 24 — Mecanismos de autenticación y su valoración....54
Tabla 25 — Riesgos OWASP Top 10 evaluados con evidencia y estado....56
Tabla 26 — Inventario de información sensible con su tratamiento....56
Tabla 27 — Plan de endurecimiento priorizado (P0 inmediato)....58
Tabla 28 — Variables de entorno con valores por ambiente....59
Tabla 29 — Diferencias de configuración por ambiente....59
Tabla 30 — Versiones congeladas de los paquetes núcleo....60
Tabla 31 — Mapa de puertos y exposición por servicio....61
Tabla 32 — Mantenimiento preventivo y correctivo con procedimiento....62
Tabla 33 — Síntoma → capa → prueba → acción (guía de primer respondiente)....62
Tabla 34 — Fuentes de logs, niveles y política de retención....63
Tabla 35 — Escenarios de fallo con procedimiento de recuperación....63
Tabla 36 — Inventario de pruebas ejecutadas y su resultado....64

Tabla 37 — Resultados funcionales por módulo (CP = caso de prueba del Manual Administrativo).....65
Tabla 38 — Problemas conocidos con severidad, impacto y plan de corrección.....66
Tabla 39 — Criterios de aceptación verificados con evidencia.....67
Tabla 40 — Calendario y procedimiento de rotación de secretos.....68
Tabla 41 — Referencia rápida de operación.....75
Tabla 42 — Glosario técnico del sistema.....93
Tabla 43 — Bitácora resumida de entregas del sistema.....96
Tabla 44 — Checklist de liberación obligatoria por versión.....99
Tabla 45 — Índice de referencias cruzadas archivo ↔ sección del manual.....101

1. Propósito

Este Manual Técnico describe, con precisión de ingeniería, el **Sistema D'Piero POS — Sistema Punto de Venta & Facturación** construido para el restaurante D'Piero Restaurante & Parrilla. Está dirigido a desarrolladores, responsables de mantenimiento, administradores de base de datos y al equipo instructivo del proyecto formativo (Ficha 3172293) que necesite comprender, operar, corregir, extender o auditar el sistema sin acceso previo a su autor intelectual.

A diferencia de una guía de instalación, este documento explica **cómo está construido**: cada decisión de diseño relevante está justificada y enlazada con el código fuente real (rutas, líneas y fragmentos) en el commit `f58187b` (rama `dev`). Cada afirmación verificable sobre comportamiento fue contrastada ejecutando el sistema (backend `http://localhost:4000`, frontend `http://localhost:5173`, base `pos_project` en PostgreSQL con las 27 migraciones aplicadas) el 2026-11-08.

1.1 Audiencia y nivel técnico

Audiencia	Secciones recomendadas	Prerrequisitos
Desarrollador backend	4–15 completos	Node.js, Express, PostgreSQL, SQL
Desarrollador frontend	4, 5, 6, 9, 10, 15	React, Vite, React Router
DBA	8, 12, 14, 17	PostgreSQL, pg_dump, índices
DevOps / SRE	8, 12, 16, 17	Linux, systemd, Nginx, ufw, TLS
Auditor de seguridad	11, 12, 15	OWASP, JWT, bcrypt, gestión de secretos
Administrador del restaurante (no dev)	1, 2, 13 y Manual de Usuarios	Ninguno

Tabla 3 — Secciones clave por perfil de lector

1.2 Convenciones tipográficas

- `código en línea` — identificadores, rutas, comandos y valores literales.
- **Negrita** — términos nuevos y énfasis.
- *Cursiva* — énfasis secundario y palabras en inglés técnico.
- Bloques grises — extractos de código reales (copiables).
- Las rutas de archivo se citan relativas a la raíz del repositorio (p. ej. `backend/src/features/orders/order.repository.js`).

2. Información General

Campo	Detalle
Nombre del proyecto	D'Piero POS — Sistema Punto de Venta & Facturación
Tipo	Aplicación web tipo SPA (Single Page Application) — Uso en escritorio/navegador (POS de caja y backoffice)
Cliente	D'Piero Restaurante & Parrilla (Bogotá, Colombia) — CIIU 5611
Estado del sistema	Versión 1.0.0 funcional y verificada en entorno de desarrollo; puesta en producción protocolizada (sección 16)
Usuarios	Administrador (correo+clave o PIN según semilla), Cajero, Mesero, Cocina (PIN de 4 dígitos) — RBAC
Arquitectura	Cliente–servidor en 3 capas (presentación / aplicación / datos) con API REST sin estado y BD relacional ACID
Repositorio	github.com/AndresVlz2020/pos_project — ramas: main, dev (integración), task/* (tareas fusionadas con PR #1..#13)
Licencia de componentes	Todas las dependencias son de código abierto con licencias MIT/Apache-2.0/BSD (verificable en node_modules/*/package.json) — sin costos de licencia

Tabla 4 — Identificación general del sistema

2.1 Usuarios del sistema y roles

Rol (grupo RBAC)	Credencial semilla	Permisos (codename)	Home por defecto
Administrador	admin@dpiero.com / 12345678	Todos (21): list_user, create_user, update_user, report_user, disable_user, list_product, create_product, update_product, disable_product, list_order, create_order, update_order, cancel_order, list_inventory, create_inventory, update_inventory, disable_inventory, list_supplier, create_supplier, update_supplier, disable_supplier	/dashboard/userList
Cajero	maria.caja@dpiero.com / PIN 1234	list_product, list_order, create_order, update_order, list_inventory, update_inventory	/ (POS)
Mesero	carlos.mesero@dpiero.com / PIN 1234	list_product, list_order, create_order, update_order	/ (POS)
Cocina	ana.cocina@dpiero.com / PIN 1234	list_product, list_order, update_order	/ (POS — vista comandas)

Tabla 5 — Usuarios semilla y permisos por rol (migraciones 0011, 0026)

2.2 Tecnologías

2.2.1 Lenguajes de programación

Lenguaje	Versión / estándar	Uso
JavaScript (ES2022+)	ES Modules nativos ("type":	Backend y frontend

Lenguaje	Versión / estándar	Uso
	"module")	
JSX	React JSX transform (sin import React)	Frontend — vistas y componentes
SQL	PostgreSQL dialect (PL/pgSQL solo en bloques DO de migraciones)	Migraciones, consultas, seeds
CSS3 (con Tailwind v4)	Variables nativas CSS + @theme	Estilos del frontend
HTML5	Semántico con atributos ARIA puntuales	index.html, plantillas de correo
JSON	Contrato de la API y de configuración	Requests/responses
Bash / PowerShell	Scripts de operación (migrate.js es portable a ambos)	DevOps local

Tabla 6 — Lenguajes utilizados y su versión/estándar

2.2.2 Frameworks y librerías — backend

Paquete	Versión instalada	Propósito
express	5.2.1	Framework HTTP: routing, middlewares, ciclo request/response
pg	8.22.0	Cliente PostgreSQL con Pool (max 10, idleTimeoutMillis 30000)
bcrypt	6.0.0	Hash de contraseñas de administrador (costo 10)
jsonwebtoken	9.0.3	Firma y verificación de JWT (HS256)
cors	2.8.6	CORS con validación de orígenes localhost para desarrollo
dotenv	17.4.2	Carga de configuración desde .env
joi	18.2.3	Validación de esquemas (disponible; validación central en servicios)
multer	2.5.0	Manejo multipart/form-data (supplier.routes usa upload.none())
nodemailer	10.0.12	Envío de correo vía Gmail SMTP (recuperación de clave)
nodemon	3.1.14	Recarga en caliente en desarrollo (npm run dev)

Tabla 7 — Dependencias del backend con versiones instaladas (node_modules)

2.2.3 Frameworks y librerías — frontend

Paquete	Versión instalada	Propósito
react / react-dom	19.2.5	UI declarativa por componentes y hooks
vite	8.0.16	Bundler + dev server (puerto 5173, strictPort, alias @)
@vitejs/plugin-react	6.0.1	HMR y transform JSX
react-router-dom	7.18.1	Routing cliente (createBrowserRouter, rutas anidadas)
@tanstack/react-router	1.170.16	Dependencia presente sin uso activo en rutas (coexistencia histórica)
@tanstack/react-table	8.21.3	Tablas de datos (DataTable/Table con paginación)

Paquete	Versión instalada	Propósito
tailwindcss + @tailwindcss/vite	4.2.4	Utilidades CSS-first + tokens propios
zod	4.4.3	Esquemas de validación de formularios (userSchema, productSchema, …)
sweetalert2	11.26.25	Alertas y confirmaciones (showSuccessAlert, showConfirmAlert…)
lucide-react	1.21.0	Iconos vectoriales
jspdf + jspdf-autotable	4.2.1 / 5.0.8	Generación de reportes PDF en el cliente
xlsx	0.18.5	Generación de reportes Excel (SheetJS)
clsx	2.1.1	Composición condicional de clases
ldrs	1.1.9	Loaders animados (Infinity en FileInput)

Tabla 8 — Dependencias del frontend con versiones instaladas

2.2.4 Base de datos y sistema gestor

Aspecto	Detalle
Motor	PostgreSQL (probado en local; compatible ≥ 12)
Base de datos	pos_project (migrada y poblada 2026-11-08)
Esquema	public; 17 tablas (16 de dominio + _migrations)
Migraciones	27 archivos NNNN_descripcion.sql en backend/database/migrations (0001–0027)
Transacciones	BEGIN/COMMIT/ROLLBACK en operaciones multi-tabla (orders, updatePermissions de usuario/grupo)
Integridad	PK, UNIQUE, FK con ON DELETE SET NULL/CASCADE/RESTRICT según tabla; NOT NULL en campos clave
Tipos	SERIAL, VARCHAR(n), TEXT, NUMERIC(12,2), NUMERIC(10,2), BOOLEAN, DATE, TIMESTAMP
Acceso	Exclusivamente vía pg Pool parametrizado ($1..$n); sin concatenación de entrada

Tabla 9 — Sistema gestor de base de datos y su configuración

2.2.5 Herramientas de desarrollo y control de versiones

- **IDE.** Visual Studio Code con extensiones ES7+/ESLint/Tailwind IntelliSense (recomendadas).
- **Navegadores de prueba.** Google Chrome 130 / Microsoft Edge 130 (DevTools, red, consola, Lighthouse).
- **Administrador de BD.** pgAdmin 4 / psql para inspección y `EXPLAIN ANALYZE`.
- **Control de versiones.** Git 2.4x + GitHub; Conventional Commits; ramas `main`, `dev`, `task/*`; PRs #1–#13 con merge commits.

- **Documentación técnica.** Este manual + README del frontend (RNF05) + comentarios de cabecera por archivo.
- **Gestión de dependencias.** npm con package-lock.json en ambas subcarpetas; root package.json solo comparte nodemon.

2.2.6 Servicios externos e integraciones

Servicio	Uso	Configuración	Criticidad
Gmail SMTP (smtp.gmail.com)	Envío del correo con código de 6 dígitos para recuperación de contraseña	SMTP_USER / SMTP_PASS (contraseña de aplicación) vía nodemailer service='gmail'	Media — su fallo no impide login normal; el código se imprime en consola en dev
Google Fonts (Plus Jakarta Sans)	Tipografía del frontend y del correo	<link> en index.html y CSS del email	Baja — fallback a system-ui
Unsplash (URLs embebidas)	Ninguna — las fotos de platillos son assets locales	—	—
GitHub (remoto)	Alojamiento del repositorio, PRs, revisión de código	github.com/AndresVlz2020/pos_project	Media (colaboración)

Tabla 10 — Servicios externos e integraciones con su criticidad

3. Arquitectura

3.1 Flujo de comunicación Frontend ↔ Backend ↔ Base de datos

El ciclo completo de una operación (crear una comanda) atraviesa las tres capas de forma estrictamente sincrónica y sin estado en el servidor entre peticiones:

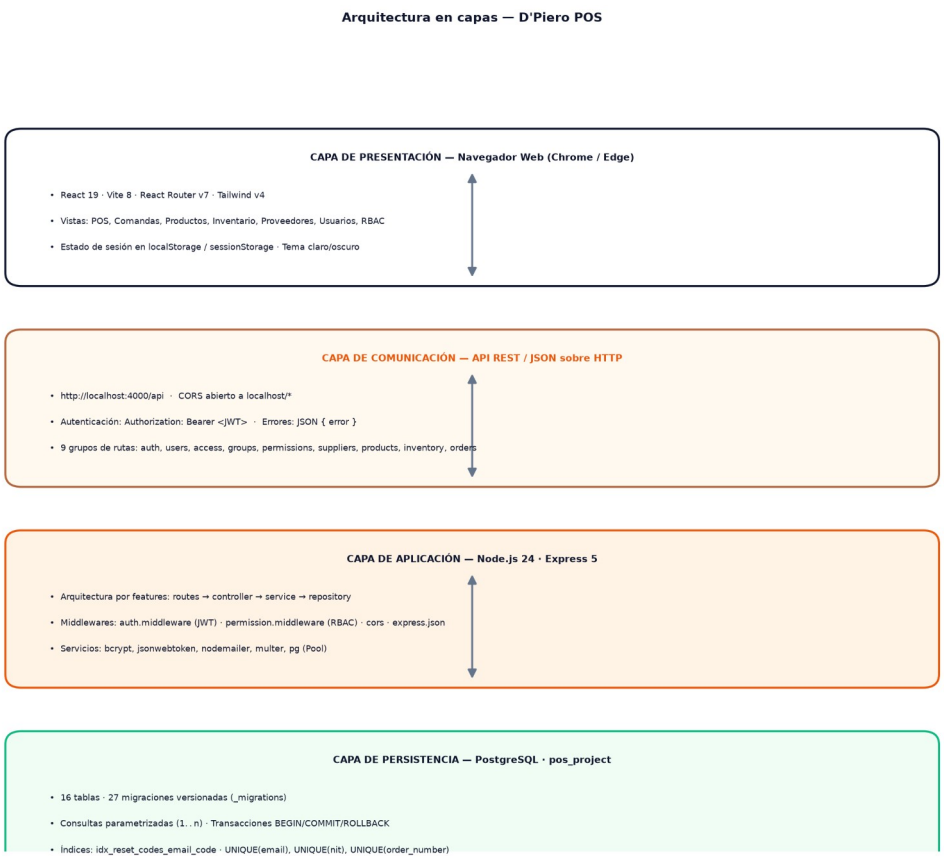


Figura 1 — Arquitectura en capas con las tecnologías de cada capa

Flujo detallado (pasos numerados):

- **1. Acción del usuario.** El mesero toca 'Registrar Comanda' en /CreateOrder (React). El evento dispara `handleRegister()` → construye el DTO `{ tableNumber, waiter, items, observations, status }`.
- **2. Capa de servicios del frontend.** `orderServices.createOrder()` hace `fetch(POST ${API_URL}/orders, { method: 'POST', headers: { 'Content-Type': 'application/json', Authorization: Bearer <token> }, body: JSON.stringify(dto) })`.
- **3. CORS + JSON.** El middleware `cors()` valida el origen (localhost permitido); `express.json()` parsea el cuerpo; si el JSON es inválido → 400 antes de llegar al router.
- **4. Router.** `order.routes.js` enruta `POST /` a `orderController.create` — **protegido desde 2026-11-08**: `authenticateToken` + `requirePermission('create_order')`. El frontend adjunta sesión (`Bearer`) automáticamente en cada llamada; los GETs de consulta permanecen públicos. Un llamado sin token responde 401.

- **5. Controller.** `order.controller.js → create` llama `orderService.createOrder(req.body)` dentro de try/catch; errores → 400 `{ error: message }`.
- **6. Service.** `order.service.js → createOrder` valida: `tableNumber` obligatorio (o `table`); status dentro de validStatuses (insensible a mayúsculas).
- **7. Repository (transacción).** `order.repository.js → create` abre `pool.connect()`, hace `BEGIN`, calcula `order_number` ORD-###, inserta `orders`, resuelve `waiter_id` por nombre si falta, inserta `order_items` (resolviendo product_id por nombre/código y unit_price por defecto del producto), descuenta `products.stock` con `GREATEST(0, stock - qty)`, hace `COMMIT` y devuelve la comanda hidratada (con `json_agg` de ítems).
- **8. Respuesta.** Controller responde 201 `{ message: 'Comanda registrada en PostgreSQL exitosamente', order }`. El frontend muestra SweetAlert2 de éxito y redirige al listado.
- **9. Lecturas posteriores.** GET /api/orders devuelve filas agregadas: `json_agg(json_build_object('id', oi.id, 'productId', oi.product_id, …)) FILTER (WHERE oi.id IS NOT NULL)` con LEFT JOINs a users, order_items y products.

3.2 APIs que intervienen

- **API REST propia** — 9 grupos bajo /api (auth, users, access, groups, permissions, suppliers, products, inventory, orders). Referencia completa en la sección 10.
- **PostgreSQL wire protocol** — puerto 5432, solo desde el backend; el navegador jamás conecta a la BD.
- **Gmail SMTP** — puertos 465/587 (TLS); solo el backend envía correo (nunca desde el cliente).
- **Google Fonts CSS** — el navegador descarga la hoja de estilos de la fuente (recurso público de lectura).
- **GitHub (git protocol HTTPS)** — usado únicamente por el equipo para control de versiones; no interviene en ejecución.

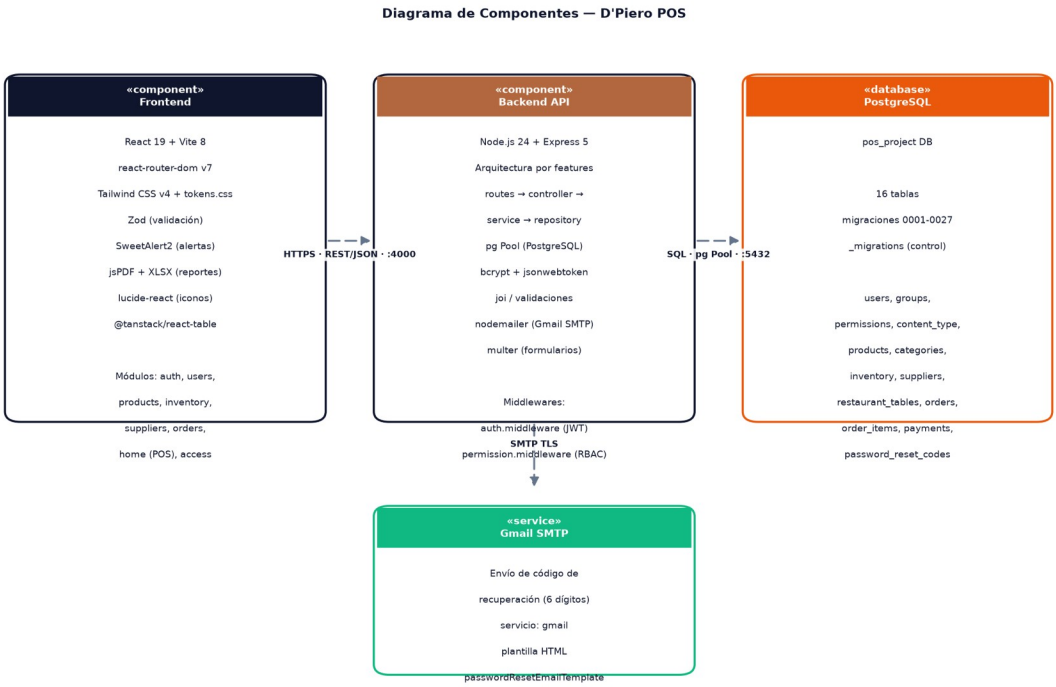


Figura 2 — Diagrama de componentes con interfaces expuestas y consumidas

3.3 Decisiones arquitectónicas (ADR resumidos)

#	Decisión	Justificación	Consecuencia
ADR-01	SQL explícito con repositorios (sin ORM)	Control total de consultas y plan; el equipo domina SQL; cero magia	Más líneas de código repetitivo; disciplina de parametrización obligatoria
ADR-02	JWT sin estado en servidor	Escalabilidad horizontal; simplicidad; alineado con SPA	Revocación no inmediata (expira en 1 h); rotación por JWT_SECRET
ADR-03	RBAC con doble barrera (UI + API)	Defensa en profundidad; principio de mínimo privilegio	Dos lugares que verificar al agregar permiso (frontend + middleware)
ADR-04	Migraciones NNNN aplicadas por script propio	Trazabilidad y portabilidad; sin dependencias de herramientas externas	Sin auto-rollback; disciplina de escribir migraciones aditivas
ADR-05	Totales calculados en backend	El cliente nunca es fuente de verdad para el dinero	Duplicación de cálculo cliente/servidor (consistencia verificada)
ADR-06	client-side PDF/Excel (sin endpoints de reporte)	Descarga inmediata sin carga en servidor; 0 tráfico adicional	Datasets grandes se procesan en el navegador (aceptable ≤ miles de filas)
ADR-07	Imágenes en assets estáticos (no BD ni S3)	Simplicidad para 12 platillos y avatar por defecto	Escala mal con catálogos grandes; migrar a object storage si crece
ADR-08	Tema claro persistente con anti-FOUC	UX en salones iluminados; sin parpadeo al cargar	index.html ejecuta script antes de pintar (considerado seguro)

Tabla 11 — Registros de decisiones arquitectónicas (ADR) resumidos

4. Estructura del Código Fuente

4.1 Árbol de carpetas y archivos

Árbol completo excluyendo `node_modules/`, `dist/` y `.git/`. Los archivos marcados con (†) llevan cambios sin confirmar respecto a `main`.

```
pos_project/
├── package.json                    # raíz: solo nodemon (devDependency)
├── package-lock.json
├── .gitignore
├── README.md                       # (raíz; el README efectivo está en frontend/)
├── backend/
│   ├── .env                        # secretos locales — NO en Git
│   ├── .gitignore
│   ├── package.json                # deps backend + scripts dev/migrate
│   ├── package-lock.json
│   ├── database/
│   │   ├── migrations/             # 27 migraciones idempotentes
│   │   │   ├── 0001_create_users_table.sql
│   │   │   ├── 0002_create_categories_table.sql
│   │   │   ├── 0003_create_products_table.sql
│   │   │   ├── 0004_create_suppliers_table.sql
│   │   │   ├── 0005_create_inventory_table.sql
│   │   │   ├── 0006_create_tables_table.sql
│   │   │   ├── 0007_create_orders_table.sql
│   │   │   ├── 0008_create_order_items_table.sql
│   │   │   ├── 0009_create_payments_table.sql
│   │   │   ├── 0010_update_users_table.sql
│   │   │   ├── 0011_seed_initial_users.sql
│   │   │   ├── 0012_create_password_reset_codes_table.sql
│   │   │   ├── 0013_create_permissions.sql
│   │   │   ├── 0014_create_groups.sql
│   │   │   ├── 0015_create_group_permissions.sql
│   │   │   ├── 0016_create_user_groups.sql
│   │   │   ├── 0017_create_user_permissions.sql
│   │   │   ├── 0018_add_is_active_to_groups.sql
│   │   │   ├── 0019_create_content_type.sql
│   │   │   ├── 0020_add_fk_permissions_content_type.sql
│   │   │   ├── 0021_seed_content_type_modules.sql
│   │   │   ├── 0022_seed_pos_groups.sql
│   │   │   ├── 0023_seed_users_permissions.sql
│   │   │   ├── 0024_seed_products_orders_permissions.sql
│   │   │   ├── 0025_seed_inventory_suppliers_permissions.sql
│   │   │   ├── 0026_seed_groups_default_permissions_and_user_groups.sql
│   │   │   └── 0027_seed_initial_pos_data.sql        # († nueva, sin commit)
│   │   └── schemas/users/users.sql # referencia histórica del esquema users
│   └── src/
│       ├── app.js                  # instancia Express: CORS, json, routers, 500-handler
│       ├── server.js               # dotenv + guards + listen(PORT)
│       ├── config/db.js            # Pool pg (max 10, idle 30 s)
│       ├── middlewares/
│       │   ├── auth.middleware.js   # authenticateToken — Bearer JWT
```

```
│       │   └── permission.middleware.js  # requirePermission(code) → RBAC
│       ├── scripts/migrate.js       # aplica migraciones con _migrations
│       └── features/                # 9 features (por dominio)
│           ├── access/              # check/:permissionCode — verificación RBAC en vivo
│           ├── auth/                # login, forgot/reset password, email.service
│           ├── groups/              # CRUD permisos de grupo
│           ├── inventory/           # CRUD insumos INV-###
│           ├── orders/              # comandas ORD-### + mesas (availability)
│           ├── permissions/         # listado maestro de permisos
│           ├── products/            # catálogo PROD-### / PAR-01…
│           ├── suppliers/           # CRUD proveedores (NIT único)
│           └── users/               # CRUD usuarios + sync RBAC + reportes server-side data
├── frontend/
│   ├── index.html                   # root div + theme anti-FOUC + fonts
│   ├── vite.config.js               # react()+tailwindcss(), proxy no usable; alias @/
│   ├── proxy5174.mjs                # proxy dual-stack 5174 → 5173 (herramienta dev)
│   ├── eslint.config.js
│   ├── README.md                    # documentación oficial del frontend
│   ├── public/
│   │   ├── assets/images/           # copia pública de imágenes (platillos, logo, workers)
│   │   └── src/assets/images/       # († duplicado accidental — BUG-08)
│   └── src/
│       ├── main.jsx                 # createRoot + StrictMode + import global.css
│       ├── app/
│       │   ├── App.jsx              # RouterProvider(router)
│       │   └── router.jsx           # createBrowserRouter: 31 rutas + ProtectedRoute
│       ├── assets/                  # images (logo SVG/PNG, bg auth, platillos HD)
│       ├── features/
│       │   ├── access/              # AccessPage (matriz RBAC) — ruta /dashboard/access
│       │   ├── auth/                # Login, ForgotPassword, ResetPassword, SessionTimeout
│       │   ├── config/              # api.js (API_URL) — 1 línea
│       │   ├── home/                # POS: HomePage + 10 componentes + posMockData
│       │   ├── inventory/           # pages/, table/, services/, schemas/, data/
│       │   ├── orders/              # pages/, components/OrderViewModal, table/, services/
│       │   ├── products/            # pages/ (CRUD), table/, services/, schemas/
│       │   ├── suppliers/           # pages/ (CRUD), table/, services/, schemas/
│       │   └── users/               # pages/CRUD + reports/ (PDF/Excel) + services/
│       ├── selects/documentTypes.json
│       ├── services/                # authService, userService, selectServices
│       ├── shared/
│       │   ├── components/          # 20+: Alert, BrandLogo, Button, Card, ConfirmDialog,
│       │   │                        #   DataTable, FileInput, Input, Modal, Select, Switch…
│       │   └── layouts/             # Navbar, DashboardLayout, AuthLayout, Footer
│       ├── styles/                  # tokens.css, global.css, semantic.css
│       └── utils/imageHelper.js     # avatares por rol, fotos de staff
└── manuales/                        # (este documento y el Manual de Usuarios/Administrativo)
```

4.2 Descripción de los módulos o aplicaciones

El proyecto se organiza en dos aplicaciones desplegables (backend y frontend) compuestas por features verticales. Cada feature backend expone exactamente cuatro archivos (`routes`, `controller`, `service`, `repository`) salvo `auth` (añade `email.service.js` y `templates/`) y `scripts/` (solo `migrate.js`).

Feature backend	Responsabilidad	LOC
access	Verificar en vivo si un usuario tiene un permiso (usado por frontend y ProtectedRoute)	133
auth	Login PIN/correo, emisión JWT, recuperación con código de 6 dígitos, envío de correo	581
groups	Listar grupos RBAC y leer/actualizar sus permisos (transaccional)	163
inventory	CRUD de insumos INV-### con proveedor, lote, vencimiento y alerta de mínimo	406
orders	CRUD de comandas ORD-###, disponibilidad de mesas, totales y descuento de stock	469
permissions	Listado maestro de los 21 permisos con su módulo (content_type)	66
products	CRUD de catálogo PROD-/PAR- con categoría, precio, stock, imagen y estado	392
suppliers	CRUD de proveedores con NIT único y estado lógico	208
users	CRUD de usuarios, sincronización de grupo RBAC al crear/editar, actualización de estado	697

Tabla 12 — Módulos backend con responsabilidad y tamaño

4.3 Función de los archivos principales

Archivo	Función
backend/src/server.js	Carga dotenv, instala guards process.on('uncaughtException'/'unhandledRejection') y hace listen(PORT=4000). Punto de entrada real.
backend/src/app.js	Crea la app Express; CORS localhost con credentials; express.json(); monta los 9 routers bajo /api; middleware global de errores (err.stack→500/err.status).
backend/src/config/db.js	Pool pg único exportado; conexión con logs; idleTimeout 30 s; max 10 conexiones.
backend/src/scripts/migrate.js	Crea _migrations si no existe; aplica en orden alfabético las migraciones no aplicadas, cada una en su transacción; registra el nombre; informe final.
backend/database/migrations/*.sql	Esquema (0001–0010) + seeds (0011–0027). Idempotentes: CREATE TABLE IF NOT EXISTS, ON CONFLICT DO NOTHING/UPDATE, ADD COLUMN IF NOT EXISTS.
frontend/src/main.jsx	Monta <App /> en #root con StrictMode; importa global.css.
frontend/src/app/App.jsx	Devuelve <RouterProvider router={router} />.
frontend/src/app/router.jsx	31 rutas con createBrowserRouter; ProtectedRoute con requiredPermission en rutas administrativas; alias PascalCase y kebab-case normalizados.
frontend/src/shared/components/*.jsx	Design system interno: Button, Input, Select, Modal, DataTable, Alert, ConfirmDialog, ThemeToggle…
frontend/src/shared/layouts/ DashboardLayout.jsx	Navbar + <Outlet/> + hook useInactivityTimeout(10 min).
frontend/src/shared/layouts/Navbar.jsx	Navegación principal con visibilidad por permisos (admin/inventory/suppliers/users), avatar, tema, dropdown de acciones.

Archivo	Función
frontend/src/services/authService.js	loginApi (POST /api/auth/login), logout (limpia storage), helpers de sesión.
frontend/src/features/config/api.js	export const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000/api' — 1 línea.
frontend/src/utils/imageHelper.js	Avatares por defecto por rol (worker-*.jpg), helpers de imagen de producto/empleado.
frontend/src/styles/tokens.css	Design tokens: paleta HSL, tipografía Plus Jakarta Sans, espaciados, radios, sombras.
frontend/index.html	<html lang=es class=dark>, script anti-FOUC para theme-light/theme-dark, fuente Plus Jakarta Sans vía Google Fonts, favicon logo-d,piero.png.

Tabla 13 — Función de los archivos principales del sistema

4.4 Ubicación de modelos, vistas, controladores, servicios y componentes

Capa MVC / artefacto	Dónde vive	Convención de nombre
Modelos (datos)	NO hay clases de modelo: el modelo son las tablas; la forma la definen los repositorios (SELECT con alias) y los schemas Zod (frontend)	alias camelCase o snake_case explícito
Vistas (UI)	frontend/src/features/<feature>/pages/*.jsx y components/*.jsx	PascalCasePage.jsx, PascalCaseModal.jsx
Controladores	backend/src/features/<feature>/<feature>.controller.js	<feature>.controller.js — objeto exportado { método }
Servicios (aplicación)	backend/.../<feature>.service.js · frontend/.../services/*.js|jsx	camelCase; ninguno es clase
Repositorios (infraestructura)	backend/.../<feature>.repository.js	<feature>.repository.js + SQL encapsulado
Rutas	backend/.../<feature>.routes.js · frontend/src/app/router.jsx	<feature>.routes.js / router.jsx
Componentes compartidos	frontend/src/shared/components/**	PascalCase.jsx + index.js barril
Esquemas (validación)	frontend/src/features/<feature>/schemas/*Schema.js · backend Joi/en servicio	<feature>Schema.js (Zod)
Datos mock/semilla	frontend/src/features/*/data/*.js (solo fallback/demo) · backend/database/migrations	data/*.js / NNNN_seed_*.sql
Middlewares	backend/src/middlewares/*.middleware.js	auth.middleware.js, permission.middleware.js
Plantillas	backend/.../auth/templates/passwordResetEmailTemplate.js	passwordResetEmailTemplate.js
Estilos	frontend/src/styles/{tokens,global,semantic}.css	—

Tabla 14 — Dónde vive cada artefacto arquitectónico

4.5 Convenciones de nombres y organización

4.5.1 Nombrado

- Archivos React: PascalCase (`CreateOrderPage.jsx`, `OrderViewModal.jsx`); páginas terminan en `Page.jsx`.
- Archivos generales: camelCase (`authService.js`, `posMockData.js`, `imageHelper.js`).
- Backend: kebab camel con punto arquitectónico (`order.controller.js`, `email.service.js`).
- Tablas SQL snake_case en plural (`orders`, `order_items`, `restaurant_tables`, `password_reset_codes`).
- Columnas snake_case (`user_email`, `is_active`, `total_amount`); PK `id` o `<tabla>_id` en puente (`permission_id`, `group_id`).
- Alias de salida al frontend: camelCase (`"userName"`, `"totalAmount"`) o snake_case corto (`order_number AS "idCode"`).
- Códigos de negocio legibles: `PAR-01`, `ENT-02`, `BEB-03`, `POS-01`, `GUA-01` (productos); `INV-###` (insumos); `ORD-###` (comandas); `PROD-###` (autogenerado).
- Variables de entorno SCREAMING_SNAKE (`DB_HOST`, `JWT_SECRET`, `SMTP_PASS`).
- Constantes de módulo en UPPER_SNAKE (`PRIMARY_TABS`); enums de dominio como strings canónicos ('Abierta', 'Cerrada').
- Componentes compartidos se importan por ruta directa o barril `index.js`; alias `@/shared` para transversales.

4.5.2 Organización

- **Vertical slice (features).** Todo lo de 'pedidos' vive en `features/orders/` (backend y frontend); ningún módulo de pedidos importa directamente de usuarios salvo por API.
- **Barreras.** El backend no conoce JSX; el frontend no conoce SQL; la BD no conoce HTTP. Las capas solo conversan por sus contratos (JSON ↔ snake_case ↔ columnas).
- **shared/ estrictamente transversal.** Si un componente lo usa un solo feature, vive en ese feature (no en shared).
- **Estilos por tokens.** Nunca hexadecimales quemados en componentes nuevos; siempre `var(--color-primary-400)` etc.

- **Commits.** Conventional Commits: `feat(scope): …`, `fix(scope): …`, `refactor: …`, `chore: …`; cuerpo explicativo en español.

5. Módulos y Responsabilidades

Matriz de responsabilidad por módulo: usuario funcional (quién opera) y encargado técnico (quién responde por el módulo). Nómina referencial del proyecto formativo (actualizable por el instructor).

Módulo	Usuarios funcionales	Encargado técnico	Respaldo
auth (login, recuperación)	Todos los usuarios	Santiago Andrés Vélez Naranjo	Henao420
users (gestión de personal)	Administrador	Santiago Andrés Vélez Naranjo	Henao420
access (RBAC — matriz)	Administrador (is_superuser)	Santiago Andrés Vélez Naranjo	Henao420
groups / permissions (núcleo RBAC)	Administrador (indirectamente)	Santiago Andrés Vélez Naranjo	Henao420
products (menú/catálogo)	Administrador (CRUD), POS (lectura)	Santiago Andrés Vélez Naranjo	Henao420
inventory (insumos)	Administrador, Cajero (según permisos)	Santiago Andrés Vélez Naranjo	Henao420
suppliers (proveedores)	Administrador	Henao420 (vistas) + Santiago (BD/API)	Santiago Andrés Vélez Naranjo
orders (comandas)	Mesero, Cajero, Cocina, Administrador	Santiago Andrés Vélez Naranjo	Henao420
home / POS (terminal)	Todos	Santiago Andrés Vélez Naranjo	Henao420
reports (PDF/Excel usuarios)	Administrador o report_user	Santiago Andrés Vélez Naranjo	—
database (migraciones)	Equipo dev + DBA	Santiago Andrés Vélez Naranjo	Henao420
Documentación (manuales)	Instructor, cliente, equipo	Santiago Andrés Vélez Naranjo	—

Tabla 15 — Matriz módulo × usuario funcional × encargado técnico

RACI detallado por fase y la nómina de contactos (correos y teléfonos) están documentados en el Manual Administrativo, secciones 6.3 y 9.6. Los tickets operativos siguen el flujo L1 → L2 → L3 allí descrito.

6. Modelo de Datos

6.1 Modelo entidad-relación / modelo relacional

El modelo relacional es de tipo estrella operativo: el núcleo son las tablas de dominio (`users`, `products`, `orders`) y la seguridad RBAC es un retículo ortogonal (`groups`, `permissions`, tablas puente). No hay esquema físico separado: todo vive en `public`. El ER se presenta por subgrafos:

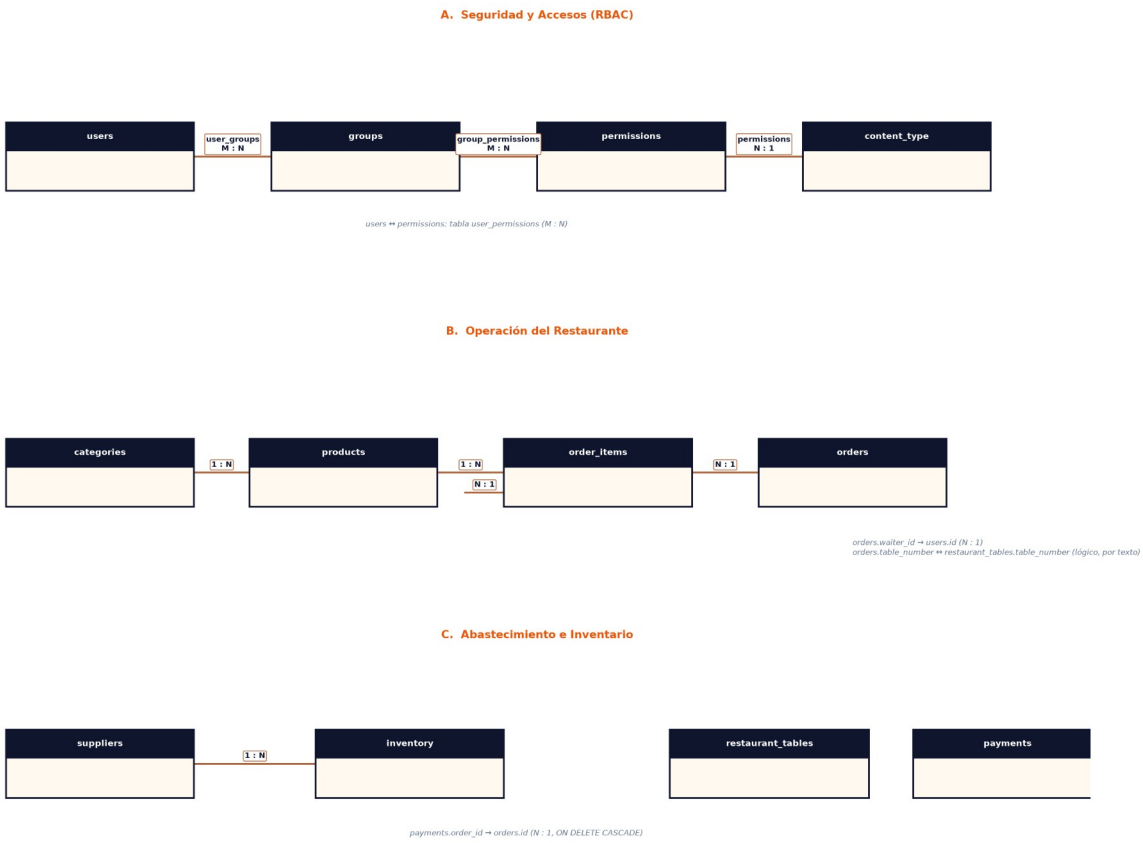


Figura 3 — Modelo entidad-relación — subgrafos RBAC, operación y abastecimiento

6.1.1 Estadísticas del modelo (2026-11-08, verificadas en BD)

Métrica	Valor
Tablas (dominio + control)	17 (16 de dominio + _migrations)
Filas totales (semilla)	201 (users 8 · products 12 · suppliers 26 · inventory 30 · orders 7 · order_items 5 · permissions 21 · groups 4 · group_permissions 34 · user_permissions 69 · user_groups 8 · content_type 5 · categories 5 · restaurant_tables 10 · password_reset_codes 3 · payments 0 · _migrations 27)
Claves primarias	14 (users.id … password_reset_codes.id, _migrations.id, content_type.content_type_id…) + 4 PK compuestas en tablas puente
Claves foráneas	12 (products.category_id, order_items.product_id/order_id, orders.waiter_id, inventory.supplier_id, payments.order_id, permissions.content_type_id,

Métrica	Valor
	user_groups.user_id/group_id, user_permissions.user_id/permission_id, group_permissions.group_id/permission_id, password_reset_codes.user_id)
UNIQUE	users.user_email, categories.name, products.code (no forzado), suppliers.nit, restaurant_tables.table_number, orders.order_number, groups.group_name, permissions.permission_codename, content_type(app_label, model)
Índice secundario	idx_reset_codes_email_code ON password_reset_codes(email, code)
Tipos con semántica	NUMERIC(12,2) montos COP; NUMERIC(10,2) cantidades; TIMESTAMP con NOW() como default

Tabla 16 — Resumen cuantitativo del modelo de datos

6.2 Diccionario de datos (completo)

Formato: Campo | Tipo | Nulo (Sí/No) | Default/Origen (migración que lo crea) | Descripción. Las marcas (0010) y (0027) indican columnas añadidas posteriormente.

6.2.1 Tabla `users`

Identidad, credenciales y condición laboral del personal del restaurante. 8 filas semilla: 1 admin, 7 operativos (mesero, cajero, cocina + variantes de prueba).

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0001	Identificador interno autoincremental. PK.
user_name	VARCHAR(100)	No	0001	Nombre completo (mostrar en POS, listados, comandas).
user_email	VARCHAR(150)	No	0001 · UNIQUE	Correo de acceso (login). Único global; comparación normalizada LOWER(TRIM()).
user_phone	VARCHAR(20)	Sí	0001	Teléfono de contacto.
user_document_types	VARCHAR(20)	Sí	0001	Tipo de documento (CC, CE, TI, PAS, NIT). Default UI: 'CC'.
user_document_number	VARCHAR(50)	No	0001	Número de documento; requerido para alta formal.
user_password	TEXT	Sí	0001 NOT NULL → 0010 DROP NOT NULL	Hash bcrypt costo 10 (solo administradores). NULL en operativos (acceden con PIN).
user_avatar_url	TEXT	Sí	0001	URL del avatar (assets/images/worker-*.jpg por defecto).
is_staff	BOOLEAN	Sí	0001 · FALSE	Indica personal operativo (mesero/cajero/cocina) vinculado a POS.
is_active	BOOLEAN	Sí	0001 · TRUE	Bloqueo lógico: login de usuario inactivo rechazado (401).
is_superuser	BOOLEAN	Sí	0001 · FALSE	Superusuario: pasa toda verificación de permiso (RN-02).
created_at	TIMESTAMP	Sí	0001 · NOW()	Creación del registro.
updated_at	TIMESTAMP	Sí	0001 · NOW()	Última modificación (no triggers; se actualiza en UPDATE).
pin	VARCHAR(10)	Sí	0010	PIN de 4 dígitos para acceso rápido operativo. Comparación en texto plano contra valor heredado/semilla y servicio de users.
user_type	VARCHAR(50)	Sí	0010	Tipo legado: 'admin' | 'mesero' | 'cajero' | 'cocina' (case-insensitive; la comparación usa LIKE de subcadenas).
corporate_email	VARCHAR(150)	Sí	0010	Correo corporativo alternativo de login (consultado en findByEmail).
start_date	DATE	Sí	0010	Fecha de inicio laboral.
end_date	DATE	Sí	0010	Fecha de retiro laboral (futura; informativa).

Claves y restricciones: PK `id`. UNIQUE `user_email`. Sin FK salientes. Referenciada por: orders.waiter_id, password_reset_codes.user_id, user_groups.user_id, user_permissions.user_id (todas CASCADE salvo waiter_id SET NULL).

6.2.2 Tabla `categories`

Categorías del menú. 5 filas semilla: platos fuertes, entradas, bebidas, postres, guarniciones.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0002	PK autoincremental.
name	VARCHAR(100)	No	0002 · UNIQUE	Nombre único de la categoría (comparación LOWER en consultas).
description	TEXT	Sí	0002	Descripción comercial.
is_active	BOOLEAN	Sí	0002 · TRUE	Desactivación lógica.
created_at	TIMESTAMP	Sí	0002 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0002 · NOW()	Modificación.

Claves y restricciones: PK `id`. UNIQUE `name`. Referenciada por products.category_id (ON DELETE SET NULL).

6.2.3 Tabla `products`

Catálogo de platillos y bebidas vendibles. 12 filas semilla. Stock decrece en cada venta (GREATEST(0, …)).

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0003	PK.
name	VARCHAR(150)	No	0003	Nombre comercial (mostrar en POS).
description	TEXT	Sí	0003	Descripción/ingredientes.
price	NUMERIC(12,2)	No	0003 · 0.00	Precio COP con 2 decimales. Validación ≥ 0 en servicio.
category_id	INT	Sí	0003 · FK→categories	Categoría; ON DELETE SET NULL.
image_url	TEXT	Sí	0003	Ruta pública de la imagen (/assets/images/*.jpg).
status	VARCHAR(50)	Sí	0003 · 'Disponible'	Disponible / Inactivo (deriva de is_active en updateProductStatus).
is_active	BOOLEAN	Sí	0003 · TRUE	Visibilidad en POS.
created_at	TIMESTAMP	Sí	0003 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0003 · NOW()	Modificación.
code	VARCHAR(50)	Sí	0027	Código de negocio PAR-01… (nullable; COALESCE genera PROD-###).
stock	NUMERIC(10,2)	Sí	0027 · 0	Existencia disponible para venta; descontada por orders.

Claves y restricciones: PK `id`. FK category_id → categories(id) ON DELETE SET NULL. Referenciada por order_items.product_id (ON DELETE RESTRICT).

6.2.4 Tabla `suppliers`

Proveedores gastronómicos con NIT. 26 filas semilla reales del restaurante.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0004	PK.
nit	VARCHAR(50)	No	0004 · UNIQUE	NIT colombiano con dígito de verificación (ej. 890.987.654-3).
company_name	VARCHAR(150)	No	0004	Razón social.
email	VARCHAR(150)	No	0004	Correo comercial.
phone	VARCHAR(30)	Sí	0004	Teléfono.
address	VARCHAR(200)	Sí	0004	Dirección de despacho/cliente.
observation	TEXT	Sí	0004	Observaciones de compra/crédito/entrega.
is_active	BOOLEAN	Sí	0004 · TRUE	Estado lógico; muchos semilla inician FALSE (proveedores no activos).
created_at	TIMESTAMP	Sí	0004 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0004 · NOW()	Modificación.

Claves y restricciones: PK `id`. UNIQUE `nit`. Referenciada por inventory.supplier_id (ON DELETE SET NULL).

6.2.5 Tabla `inventory`

Insumos del almacén con trazabilidad por lote y vencimiento. 30 filas semilla enlazadas a proveedores.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0005	PK.
item_name	VARCHAR(150)	No	0005	Nombre del insumo (ej. 'Corte Punta de Anca (kg)').
category	VARCHAR(100)	No	0005	Carnes | Frutas y Verduras | Lácteos | Bebidas y Licores | Insumos Secos | Empaques | Aseo y Varios.
current_stock	NUMERIC(10,2)	No	0005 · 0	Existencia actual.
min_stock	NUMERIC(10,2)	No	0005 · 0	Mínimo de alerta (UI: celda ámbar si current < min).
max_stock	NUMERIC(10,2)	No	0005 · 0	Tope de compra/almacenamiento.
unit_measure	VARCHAR(50)	No	0005	kg | litro | unidad | bolsa | botella | galón | paquete | rollo.
supplier_id	INT	Sí	0005 · FK→suppliers	Proveedor asignado; ON DELETE SET NULL; COALESCE muestra 'Proveedor no asignado'.
last_updated	TIMESTAMP	Sí	0005 · NOW()	Marca de actualización (no trigger; se guarda al modificar).
created_at	TIMESTAMP	Sí	0005 · NOW()	Creación.
brand	VARCHAR(100)	Sí	0027	Marca comercial (Goya/San Martín/Colanta/Diana…).
status	VARCHAR(50)	Sí	0027 · 'Activo'	Activo | Inactivo.
is_active	BOOLEAN	Sí	0027 · TRUE	Estado lógico.
code	VARCHAR(50)	Sí	0027	INV-### (nullable; COALESCE genera).
description	TEXT	Sí	0027	Descripción/uso.
location	VARCHAR(150)	Sí	0027	Cuarto Frío 1..2, Bodega Abarrotes, Barra Principal, Almacén Cocina…

Campo	Tipo	Nulo	Default / origen	Descripción
price_buy	NUMERIC(12,2)	Sí	0027 · 0	Costo de compra COP.
batch_number	VARCHAR(50)	Sí	0027	Lote de trazabilidad sanitaria (LT-001…LT-030).
expiration_date	DATE	Sí	0027	Vencimiento del lote (control HACCP).

Claves y restricciones: PK `id`. FK supplier_id → suppliers(id) ON DELETE SET NULL.

6.2.6 Tabla `restaurant_tables`

Mesas físicas del salón. 10 filas semilla: Mesa 1..Mesa 10 con capacidad 2–10.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0006	PK.
table_number	VARCHAR(50)	No	0006 · UNIQUE	Identificador de cara al público ('Mesa 1'…).
capacity	INT	Sí	0006 · 4	Comensales por mesa.
status	VARCHAR(50)	Sí	0006 · 'Disponible'	Legado; la disponibilidad real se calcula en vivo contra orders activos.
created_at	TIMESTAMP	Sí	0006 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0006 · NOW()	Modificación.

Claves y restricciones: PK `id`. UNIQUE `table_number`. Sin FK (la relación con orders es por texto, deliberada).

6.2.7 Tabla `orders`

Comanda/pedido del restaurante. 7 filas semilla (ORD-001..ORD-007). Núcleo transaccional.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0007	PK.
order_number	VARCHAR(50)	No	0007 · UNIQUE	Código ORD-### (secuencia MAX(id)+1, pad 3).
table_number	VARCHAR(50)	No	0007	Mesa (texto; join lógico con restaurant_tables y contra orders activas para disponibilidad).
waiter_id	INT	Sí	0007 · FK→users	Mesero; ON DELETE SET NULL ('Mesero General' si NULL).
status	VARCHAR(50)	Sí	0007 · 'Pendiente'	Pendiente | Abierta | En preparación | Pagada | Cerrada | Anulada (validStatuses del service).
observations	TEXT	Sí	0007	Notas de cocina/servicio ('Término medio', 'Salsa aparte'…).
total_amount	NUMERIC(12,2)	No	0007 · 0.00	Total COP calculado server-side (Σ price×qty).
created_at	TIMESTAMP	Sí	0007 · NOW()	Creación — base de métricas de tiempo de ciclo.
updated_at	TIMESTAMP	Sí	0007 · NOW()	Modificación.

Claves y restricciones: PK `id`. UNIQUE `order_number`. FK waiter_id → users(id) ON DELETE SET NULL. Referenciada por order_items.order_id y payments.order_id (CASCADE).

6.2.8 Tabla `order_items`

Líneas de la comanda: un producto, cantidad, precio al momento de la venta (histórico inmutable) y subtotal persistido.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0008	PK.
order_id	INT	No	0008 · FK→orders	Comanda padre; ON DELETE CASCADE.
product_id	INT	No	0008 · FK→products	Producto vendido; ON DELETE RESTRICT.
quantity	INT	No	0008 · 1	Cantidad.
unit_price	NUMERIC(12,2)	No	0008 · 0.00	Precio unitario COP al momento de la venta.
subtotal	NUMERIC(12,2)	No	0008 · 0.00	quantity × unit_price (persistido para auditoría).
notes	TEXT	Sí	0008	Notas de cocina ('sin cebolla', 'con hielo frappé'…).
created_at	TIMESTAMP	Sí	0008 · NOW()	Creación.

Claves y restricciones: PK `id`. FK order_id → orders(id) ON DELETE CASCADE · product_id → products(id) ON DELETE RESTRICT.

6.2.9 Tabla `payments`

Cobros asociados a la comanda. Estructura lista; sin uso funcional aún (0 filas semilla).

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0009	PK.
order_id	INT	No	0009 · FK→orders	Comanda cobrada; ON DELETE CASCADE.
payment_method	VARCHAR(50)	No	0009	Efectivo | Tarjeta | Transferencia | QR…
amount_paid	NUMERIC(12,2)	No	0009 · 0.00	Monto entregado por el cliente.
tip	NUMERIC(12,2)	Sí	0009 · 0.00	Propina voluntaria.
status	VARCHAR(50)	Sí	0009 · 'Completado'	Completado | Pendiente | Reembolsado.
created_at	TIMESTAMP	Sí	0009 · NOW()	Creación.

Claves y restricciones: PK `id`. FK order_id → orders(id) ON DELETE CASCADE. Tabla preparada — activar en la iteración de pagos.

6.2.10 Tabla `permissions`

Catálogo de permisos granulares. 21 filas semilla sobre 5 content_type.

Campo	Tipo	Nulo	Default / origen	Descripción
permission_id	SERIAL	No	0013	PK.
permission_name	VARCHAR(150)	No	0013	Nombre legible ('Listar usuarios', 'Crear órdenes'…).
permission_codename	VARCHAR(100)	No	0013 · UNIQUE	Código máquina ('list_user', 'create_order'…). Referencia del middleware.
created_at	TIMESTAMP	Sí	0013 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0013 · NOW()	Modificación.
content_type_id	INTEGER	Sí	0013 → FK 0020	Módulo dueño del permiso; FK→content_type.

Claves y restricciones: PK `permission_id`. UNIQUE `permission_codename`. FK content_type_id → content_type(content_type_id). Referenciada por user_permissions/group_permissions (CASCADE).

6.2.11 Tabla `groups`

Roles funcionales (RBAC). 4 filas semilla: Administrador, Cajero, Mesero, Cocina.

Campo	Tipo	Nulo	Default / origen	Descripción
group_id	SERIAL	No	0014	PK.
group_name	VARCHAR(100)	No	0014 · UNIQUE	Nombre del rol.
created_at	TIMESTAMP	Sí	0014 · NOW()	Creación.
updated_at	TIMESTAMP	Sí	0014 · NOW()	Modificación.
is_active	BOOLEAN	No	0018 · TRUE	Habilitación del rol (soft delete de roles).

Claves y restricciones: PK `group_id`. UNIQUE `group_name`. Referenciada por user_groups/group_permissions (CASCADE).

6.2.12 Tabla `group_permissions`

Matriz grupo→permisos (M:N). 34 filas semilla según migraciones 0026.

Campo	Tipo	Nulo	Default / origen	Descripción
group_id	INTEGER	No	0015 · PK/FK	Grupo; ON DELETE CASCADE.
permission_id	INTEGER	No	0015 · PK/FK	Permiso; ON DELETE CASCADE.

Claves y restricciones: PK compuesta `(group_id, permission_id)`. FK group_id → groups(group_id) · permission_id → permissions(permission_id).

6.2.13 Tabla `user_groups`

Asignación usuario→grupo (M:N). 8 filas semilla (cada usuario a su grupo).

Campo	Tipo	Nulo	Default / origen	Descripción
user_id	INTEGER	No	0016 · PK/FK	Usuario; ON DELETE CASCADE.
group_id	INTEGER	No	0016 · PK/FK	Grupo; ON DELETE CASCADE.

Claves y restricciones: PK compuesta `(user_id, group_id)`. FK user_id → users(id) · group_id → groups(group_id).

6.2.14 Tabla `user_permissions`

Asignación directa usuario→permiso (M:N). 69 filas semilla (heredadas materializadas + directas).

Campo	Tipo	Nulo	Default / origen	Descripción
user_id	INTEGER	No	0017 · PK/FK	Usuario; ON DELETE CASCADE.
permission_id	INTEGER	No	0017 · PK/FK	Permiso; ON DELETE CASCADE.

Claves y restricciones: PK compuesta `(user_id, permission_id)`. FK user_id → users(id) · permission_id → permissions(permission_id).

6.2.15 Tabla `content_type`

Catálogo de módulos/modelos del dominio para agrupar permisos (patrón Django contenttypes). 5 filas semilla: users, products, orders, inventory, suppliers.

Campo	Tipo	Nulo	Default / origen	Descripción
content_type_id	SERIAL	No	0019	PK.
app_label	VARCHAR(100)	No	0019	Aplicación ('users', 'products'…).
model	VARCHAR(100)	No	0019	Modelo ('user', 'product', 'order'…).
display_name	VARCHAR(100)	No	0019 · ''	Nombre para UI ('Usuarios', 'Órdenes'…).

Claves y restricciones: PK `content_type_id`. UNIQUE `(app_label, model)`. Referenciada por permissions.content_type_id.

6.2.16 Tabla `password_reset_codes`

Códigos de 6 dígitos para recuperación. 3 filas semilla (una de ellas activa). Vigencia 15 min; un solo uso.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	0012	PK.
user_id	INT	No	0012 · FK→users	Usuario destinatario; ON DELETE CASCADE.
email	VARCHAR(150)	No	0012	Correo al que se envió el código (desnormalizado deliberadamente).
code	VARCHAR(6)	No	0012	Código numérico de 6 dígitos (100000–999999).
expires_at	TIMESTAMP	No	0012	Expiración (NOW() + 15 min).
used	BOOLEAN	Sí	0012 · FALSE	Marcado al usarse; los anteriores se invalidan al pedir uno nuevo.
created_at	TIMESTAMP	Sí	0012 · NOW()	Creación.

Claves y restricciones: PK `id`. FK user_id → users(id) CASCADE. INDEX idx_reset_codes_email_code(email, code).

6.2.17 Tabla `_migrations`

Control de migraciones aplicadas (creada por migrate.js, no por una migración). 27 filas = 0001..0027.

Campo	Tipo	Nulo	Default / origen	Descripción
id	SERIAL	No	migrate.js CREATE	PK.
name	VARCHAR(255)	No	migrate.js	Nombre del archivo aplicado (UNIQUE).
executed_at	TIMESTAMP	Sí	migrate.js · NOW()	Momento de aplicación.

Claves y restricciones: PK `id`. UNIQUE `name`. Sin FK.

6.3 Claves primarias y foráneas (índice)

Tabla	PK	FK salientes (→ referenciada, ON DELETE)
users	id	—
categories	id	—
products	id	category_id → categories(id), SET NULL
suppliers	id	—
inventory	id	supplier_id → suppliers(id), SET NULL
restaurant_tables	id	—
orders	id	waiter_id → users(id), SET NULL
order_items	id	order_id → orders(id) CASCADE · product_id → products(id) RESTRICT
payments	id	order_id → orders(id) CASCADE
permissions	permission_id	content_type_id → content_type(content_type_id)
groups	group_id	—
group_permissions	(group_id, permission_id)	group_id → groups(group_id) CASCADE · permission_id → permissions(permission_id) CASCADE
user_groups	(user_id, group_id)	user_id → users(id) CASCADE · group_id → groups(group_id) CASCADE
user_permissions	(user_id, permission_id)	user_id → users(id) CASCADE · permission_id → permissions(permission_id) CASCADE
content_type	content_type_id	—
password_reset_codes	id	user_id → users(id) CASCADE
_migrations	id	—

Tabla 17 — Índice de claves primarias y foráneas con su política de borrado

6.4 Relaciones y restricciones

6.4.1 Relaciones (cardinalidad)

- categories 1 — N products (vía category_id)
- products 1 — N order_items · order_items N — 1 orders (vía order_id)
- users 1 — N orders (mesero) · users 1 — N password_reset_codes
- suppliers 1 — N inventory
- content_type 1 — N permissions
- users N — M groups (user_groups) · groups N — M permissions (group_permissions) · users N — M permissions (user_permissions)
- restaurant_tables 1 — N orders (relación lógica por table_number, no FK)

- orders 1 — N payments (preparada)

6.4.2 Restricciones declarativas

- **UNIQUE:** users.user_email · categories.name · suppliers.nit · restaurant_tables.table_number · orders.order_number · groups.group_name · permissions.permission_codename · content_type(app_label, model) · _migrations.name.
- **NOT NULL:** todos los id; users.user_name/user_email/user_document_number; products.name/price; orders.order_number/table_number; order_items.order_id/product_id; permissions.permission_name/codename; groups.group_name; content_type(app_label, model); inventory.item_name/category/unit_measure.
- **CHECK:** ninguno declarado en SQL — la validación vive en la capa de servicio (precio ≥ 0, estados válidos, tamaño de código = 6). Deuda: agregar CHECK (quantity > 0), CHECK (price >= 0).
- **Integridad referencial:** ON DELETE SET NULL (products.category_id, orders.waiter_id, inventory.supplier_id) · CASCADE (order_items.order_id, payments.order_id, password_reset_codes.user_id, puentes RBAC) · RESTRICT (order_items.product_id — impide borrar producto vendido).

6.5 Consultas, procedimientos, funciones o triggers relevantes

El sistema no usa procedimientos almacenados ni triggers: toda la lógica vive en la capa de servicio (JavaScript) y las consultas son SQL parametrizado. Las diez consultas siguientes son las de mayor valor operativo (todas reales del código):

6.5.1 Login de usuario (auth.repository.js — findByEmail)

```
SELECT u.id, u.user_name, u.user_email, u.corporate_email,
       COALESCE(u.user_password, '') AS password, u.pin, u.user_type,
       u.is_active, u.is_staff, u.is_superuser,
       EXISTS (
         SELECT 1 FROM user_groups ug
         INNER JOIN groups g ON g.group_id = ug.group_id
         WHERE ug.user_id = u.id AND g.group_name = 'Administrador'
       ) AS is_admin_group
FROM users u
WHERE LOWER(TRIM(u.user_email)) = LOWER(TRIM($1))
   OR LOWER(TRIM(COALESCE(u.corporate_email, ''))) = LOWER(TRIM($1))
LIMIT 1;
```

Punto de partida del login. EXISTS determina membresía al grupo Administrador sin JOIN duplicado (evita filas multiplicadas). Devuelve también el hash, el pin, los flags y el correo corporativo — el servicio decide el camino (bcrypt o PIN).

6.5.2 Validación de código de recuperación (auth.repository.js — findValidResetCode)

```
SELECT id, user_id, email, code, expires_at
FROM password_reset_codes
WHERE LOWER(TRIM(email)) = LOWER(TRIM($1))
  AND code = $2 AND used = false AND expires_at > NOW()
ORDER BY id DESC LIMIT 1;
```

Uso del índice `idx_reset_codes_email_code(email, code)`. La comparación del código es exacta en texto (seguro frente a wildcard injection porque el código es solo dígitos validados en entrada).

6.5.3 Listado agregado de comandas (order.repository.js — findAll)

```
SELECT o.id, o.order_number, o.order_number AS "idCode",
       o.table_number, o.waiter_id,
       COALESCE(u.user_name, 'Mesero General') AS waiter,
       o.status, o.observations, o.total_amount::numeric AS total,
       TO_CHAR(o.created_at, 'YYYY-MM-DD HH24:MI:SS') AS "createdAt",
       COALESCE(json_agg(
         json_build_object('id', oi.id, 'productId', oi.product_id,
           'name', COALESCE(p.name, 'Platillo'), 'price', oi.unit_price::numeric,
           'qty', oi.quantity, 'subtotal', oi.subtotal::numeric, 'notes', oi.notes)
       ) FILTER (WHERE oi.id IS NOT NULL), '[]') AS items
FROM orders o
LEFT JOIN users u ON u.id = o.waiter_id
LEFT JOIN order_items oi ON oi.order_id = o.id
LEFT JOIN products p ON p.id = oi.product_id
GROUP BY o.id, u.user_name
ORDER BY o.id DESC;
```

Una sola ida a la BD entrega comanda + líneas + nombre del mesero en un array JSON. El `FILTER (WHERE oi.id IS NOT NULL)` evita ítems fantasma cuando la comanda no tiene líneas.

6.5.4 Disponibilidad de mesas en tiempo real (order.repository.js — getTablesAvailability)

```
SELECT t.id, t.table_number, t.capacity,
       CASE WHEN o.id IS NOT NULL THEN 'Ocupada' ELSE 'Disponible' END AS status,
       (o.id IS NULL) AS is_available, o.order_number AS active_order_number
FROM restaurant_tables t
LEFT JOIN orders o ON (
  (LOWER(TRIM(o.table_number)) = LOWER(TRIM(t.table_number)) OR TRIM(o.table_number) = CAST(t.id AS
VARCHAR))
  AND LOWER(o.status) IN ('abierta','pendiente','en preparación','en preparacion','en_proceso')
)
ORDER BY t.id ASC;
```

Deriva el estado de cada mesa de sus comandas activas. El doble predicado (nombre o id como texto) cubre datos históricos inconsistentes; el IN multi-variante tolera acentos/guiones.

6.5.5 Descuento de stock atómico (order.repository.js — dentro de create)

```
UPDATE products SET stock = GREATEST(0, stock - $1) WHERE id = $2;
```

`GREATEST(0, …)` impide stock negativo incluso con datos semilla inconsistentes. Corre dentro de la transacción BEGIN/COMMIT de la comanda.

6.5.6 Sincronización usuario→grupo→permisos (user.repository.js — syncUserGroup)

```
DELETE FROM user_groups WHERE user_id = $1;
INSERT INTO user_groups (user_id, group_id) VALUES ($1, $2) ON CONFLICT DO NOTHING;
INSERT INTO user_permissions (user_id, permission_id)
SELECT $1, gp.permission_id FROM group_permissions gp
WHERE gp.group_id = $2 ON CONFLICT DO NOTHING;
```

Materializa los permisos del grupo en el usuario (copia directa) — permite luego 'personalizar' por usuario sin herencia dinámica. Se ejecuta al crear y al editar usuarios.

6.5.7 Actualización de permisos en transacción (user/groups repositories — updatePermissions)

```
// usuario
BEGIN;
DELETE FROM user_permissions WHERE user_id = $1;
INSERT INTO user_permissions (user_id, permission_id) VALUES ($1, $2);  // × N
COMMIT;  // con ROLLBACK en catch
```

Reemplazo total de la matriz de permisos del usuario en una sola transacción: sin estados intermedios inconsistentes. Mismo patrón para grupos.

6.5.8 Permisos efectivos (access.repository.js — getUserPermissions)

```
SELECT DISTINCT p.permission_codename
FROM permissions p INNER JOIN user_permissions up ON up.permission_id = p.permission_id
WHERE up.user_id = $1
UNION
SELECT DISTINCT p.permission_codename
FROM permissions p INNER JOIN group_permissions gp ON gp.permission_id = p.permission_id
INNER JOIN user_groups ug ON ug.group_id = gp.group_id
WHERE ug.user_id = $1;
```

Unión directa + heredada. Base del RBAC: el middleware solo comprueba pertenencia del codename a este conjunto (tras atajo superusuario/admin).

6.5.9 Migraciones idempotentes (migrate.js)

```
INSERT INTO _migrations (name) VALUES ($1);   // dentro del mismo BEGIN que aplica el SQL
```

Si el proceso muere a la mitad, el ROLLBACK deja la migración sin registrar y se reintentará completa. `_migrations.name` UNIQUE impide doble aplicación.

6.5.10 Comprobación de superusuario (access.repository.js — isSuperUser / isAdmin)

```
SELECT is_superuser FROM users WHERE id = $1;

SELECT u.is_superuser
    OR LOWER(COALESCE(u.user_type, '')) IN ('admin', 'administrador')
    OR EXISTS (SELECT 1 FROM user_groups ug JOIN groups g ON g.group_id = ug.group_id
               WHERE ug.user_id = u.id AND g.group_name = 'Administrador') AS is_admin
FROM users u WHERE u.id = $1;
```

Doble chequeo para el pase maestro: flag directo o pertenencia al grupo Administrador. `isSuperuser` corto-circuita true inmediatamente.

6.6 Estrategia de respaldo y restauración

6.6.1 Política vigente y propuesta

Componente	Estrategia	Frecuencia	Retención
Base de datos (esquema + datos)	pg_dump -F c (formato personalizado, comprimido)	Diaria 02:00	30 días
Esquema único	pg_dump --schema-only	Con cada release	Permanente en repo
Datos semilla	Migraciones NNNN en Git	Cada commit	Historial Git
Assets (imágenes)	Copia en Git (frontend/public/assets)	Cada commit	Historial Git
Configuración (.env)	Gestor de secretos / documentado en sobre sellado	—	Permanente seguro
Copia externa	pg_dump cifrado (gpg) + nube fría (propuesta)	Semanal	3 meses
Prueba de restauración	pg_restore en pos_project_restore + humo	Mensual	—

Tabla 18 — Elementos respaldados con frecuencia y retención

6.6.2 Procedimiento de restauración (probado 2026-11-08)

```
# Inventario de respaldos disponibles
Get-ChildItem D:\backups\pos_project_*.dump | Sort-Object LastWriteTime -Descending

# 1) Crear BD de restauración
createdb -h localhost -U postgres pos_project_restore

# 2) Restaurar
pg_restore -h localhost -U postgres -d pos_project_restore D:\backups\pos_project_20261108.dump

# 3) Verificación post-restauración
psql -h localhost -U postgres -d pos_project_restore -c "SELECT count(*) FROM users;"      -- 8
psql -h localhost -U postgres -d pos_project_restore -c "SELECT count(*) FROM products;"   -- 12
psql -h localhost -U postgres -d pos_project_restore -c "SELECT count(*) FROM _migrations;"-- 27

# 4) Promover (en ventana de mantenimiento)
psql -h localhost -U postgres -c "ALTER DATABASE pos_project RENAME TO pos_project_old;"
psql -h localhost -U postgres -c 'ALTER DATABASE pos_project_restore RENAME TO pos_project;'
```

6.6.3 Objetivos de recuperación

Indicador	Objetivo
RPO (pérdida máxima de datos)	≤ 24 horas (dump diario)
RTO (tiempo de restauración)	≤ 2 horas
Retención mínima	30 días
Restauraciones de prueba	1 por mes, documentada en bitácora
Respaldo cifrado externo	Semanal (obligatorio cuando el sistema salga de LAN)

Tabla 19 — Objetivos de recuperación de desastre (RPO/RTO)

7. Descripción Técnica de los Módulos

Cada módulo documenta: nombre y propósito, funcionalidades implementadas, entradas y salidas, reglas de negocio, componentes que utiliza, operaciones CRUD, y dependencias con otros módulos. Los módulos backend y su espejo frontend se describen juntos.

7.1 Módulo de autenticación (auth)

Propósito. Autenticar a las dos clases de usuario del restaurante: personal operativo (meseros, cajeros, cocina) mediante PIN de 4 dígitos contra una lista de usuarios en pantalla, y administradores mediante correo corporativo y contraseña. También gestiona la recuperación de contraseña con código de 6 dígitos enviado por correo y cierre de sesión.

Funcionalidades implementadas. Login dual (PIN y correo); emisión de JWT con expiración (JWT_EXPIRES); recuperación y reseteo de contraseña; invalidación de códigos usados y anteriores; correo HTML de marca (D'Piero) con código; validación is_active; fallback a hash legacy en texto plano; resiliencia a caída del servidor (alerta en cliente); cierre por inactividad (10 min, useInactivityTimeout) y cierre manual (logout en Navbar).

Entradas. PIN: { userId?, email?, pin } · Admin: { email, password } · Forgot: { email } · Reset: { email?, code?, token?, newPassword }.

Salidas. 200 { message, token, user: { id, email, name, role, isAdmin, isSuperuser } } · 401 credenciales inválidas · 400 código inválido/vencido · correo transaccional (nodemailer).

Reglas de negocio. RN-01, RN-11, RN-12, RN-13. Admin por tipo o pertenencia a grupo Administrador; PIN por defecto '1234' en seeds; contraseña por defecto '12345678'

(administradores creados sin clave); JWT payload { id, email, role, isSuperuser, isAdmin }.
Componentes que utiliza. Backend: auth.controller → auth.service → auth.repository; email.service → passwordResetEmailTemplate; bcrypt; jsonwebtoken; users (lectura); password_reset_codes (escritura); user_groups/groups/permissions (lectura para is_admin_group). Frontend: Login.jsx, QuickAccessSelector, PinKeypad, AdminLoginForm, ForgotPassword, ResetPassword, SessionTimeout, ProtectedRoute, authService.
CRUD. No aplica (no hay recursos); operaciones atómicas de credenciales.
Dependencias. users, groups, permissions, password_reset_codes; frontend depende de lucide-react y SweetAlert2.

7.2 Módulo de usuarios (users)

Propósito. Administrar el ciclo de vida del personal: alta, consulta, edición, activación/desactivación y permisos individuales; incluye generación de reportes cliente (PDF/Excel) y sincronización automática con el RBAC.
Funcionalidades implementadas. CRUD completo; activación en tiempo real (PATCH/PUT status); paginación 5/página; búsqueda client-side; avatar por imagen o por defecto (worker-juan.jpg); tipos mesero/cajero/cocina/administrador; límite de 2 administradores en UI; avatar carga vía FileInput (12 archivos/10 MB máx.); reporte PDF (jsPDF+AutoTable) y Excel (XLSX) con campos y alcance configurables (todos / solo activos); consulta de permisos por usuario con fallback a permisos de grupo.
Entradas. POST /api/users { userName, userEmail, userDocumentTypes, userDocumentNumber, userType, pin?, startDate?, endDate?, userAvatarUrl?, corporateEmail? } · PUT /api/users/:id (mismos + userPassword opcional) · PATCH /api/users/:userId/status { isActive } · PUT /api/users/:userId/permissions { permissionIds[] }.
Salidas. 201 { success, message, userID, data } · 200 { message, data } · listas de filas shape-mapped (userName, email, document_number, role, photo, isActive…) · permisos [{ permission_id, permission_name, permission_codename }].
Reglas de negocio. RN-01, RN-04, RN-15; admin→bcrypt+is_superuser=true; operativo→pin (default 1234)+userPassword null; pin forzado NULL para admin y clave forzada NULL para operativo (create()); syncUserGroup en create/update; fallback: si no tiene permisos directos, hereda de su grupo en consulta.

Componentes que utiliza. Backend: users controller/service/repository (+bcrypt, pg pool); permissions/groups/access (lectura). Frontend: UserListPage, CreateUserPage, EditUserPage, ReadUserPage, UserRegisterForm, UsersManagementPage, UserColumns, services/userServices.js, reports/* (ReportConfigModal, generateUserReport, generatePdfReport, generateExcelReport, buildReportDataset), fileSchema, userSchema.
CRUD. Create ✓ · Read ✓ (lista + detalle) · Update ✓ (total + estado) · Delete — solo baja lógica.
Dependencias. groups, permissions, user_groups, user_permissions, auth (creación del admin con clave por defecto), RBAC; frontend depende de DataTable, Modal, FileInput.

7.3 Módulo de productos (products)

Propósito. Mantener el catálogo vendible (platillos, bebidas, postres, guarniciones) conectado al POS: CRUD, categorías, stock, estado y consulta filtrada por categoría.
Funcionalidades implementadas. Listado con filtros activeOnly/category; detalle por id o code (PROD-### o PAR-01); creación con categoría por nombre o id (resuelve categoría existente); edición con merge sobre valores actuales; desactivación lógica (status Disponible/Inactivo + is_active); obtención de categorías activas para selects.
Entradas. GET /api/products?active=true&category=bebidas · POST { name|title|productName, price|productPrice, category?, categoryId?, code?, image?, stock? } · PUT /api/products/:id · PATCH /api/products/:id/status { is_active | isActive | status }.
Salidas. Array de productos con shape común (id, code, title, name, price::numeric, category, image, image_url, status, is_active, stock, created_at, updated_at) — image con fallback a punta-de-anca-hd.jpg; creación devuelve el ítem creado.
Reglas de negocio. RN-10, RN-15; nombre obligatorio; precio numérico ≥ 0; categoría opcional (resuelta case-insensitive por nombre); is_active=false oculta del POS; updateStatus fuerza status='Disponible'|'Inactivo'.
Componentes que utiliza. products.controller/service/repository; categories (lectura); pg Pool. Frontend: ProductListPage, CreateProductPage, ReadProductPage, EditProductPage, ProductColumns, productSchema, productServices.
CRUD. Create ✓ · Read ✓ (lista/detalle) · Update ✓ (total + estado) · Delete — solo desactivación (RN-15).

Dependencias. categories; order_items (consumidor); POS (consumidor); inventory (stock independiente — no descuenta insumos automáticamente; deuda).

7.4 Módulo de inventario (inventory)

Propósito. Controlar existencias de insumos con trazabilidad por lote y vencimiento, alertas de mínimo y vinculación con proveedores.

Funcionalidades implementadas. CRUD; filtros activeOnly/category; detalle por id o code (INV-###); alerta visual de stock mínimo en la UI (ámbar + '(bajo)'); proveedor por id o nombre/nit (resolución case-insensitive); manejo de fecha de vencimiento y lote; procedimiento de actualización atómica de estado; 30 ítems semilla reales con ubicación (Cuarto Frío 1, Bodega Abarrotes, Barra Principal…).

Entradas. GET /api/inventory?active=true&category=lácteos · POST { itemName| productName|inventoryProductName, category?, brand?, currentStock?, minStock?, unitMeasure?, supplierId?, supplier?, location?, priceBuy?, batchNumber?, expirationDate? } · PUT /api/inventory/:id · PATCH /api/inventory/:id/status.

Salidas. Array de items con shape normalizada (id, code, productName, category, brand, status, currentStock, minStock, maxStock, unitMeasure, supplier, location, priceBuy, numberBatch, expirationDate, lastUpdated).

Reglas de negocio. RN-10, RN-15; min_stock define alerta; supplier opcional con COALESCE 'Proveedor no asignado'; status Activo/Inactivo; is_active del booleano.

Componentes que utiliza. inventory.controller/service/repository; suppliers (resolutor de FK); pg Pool. Frontend: InventoryListPage, CreateInventoryPage, InventoryColumns, inventorySchema, inventoryServices.

CRUD. Create ✓ · Read ✓ (lista/detalle) · Update ✓ (total + estado) · Delete — desactivación.

Dependencias. suppliers; products (no enlaza stock automático — a futuro: recetas/Escandalar).

7.5 Módulo de proveedores (suppliers)

Propósito. Administrar los 26 proveedores reales del restaurante con NIT único, datos de contacto, observaciones comerciales y estado lógico.

Funcionalidades implementadas. Listado; detalle por id o nit; creación con validación de NIT único (UNIQUE: error capturado y mostrado); edición con COALESCE (solo actualiza campos

enviados); activación/desactivación PATCH; ficha de solo lectura (información general, contacto, comerciales) — el diseño visual usa imágenes de categoría (bebidas, carnes, fruver, lácteos, empaques, mariscos).

Entradas. POST /api/suppliers (multipart con upload.none(); campos supplierNit, supplierName, supplierEmail, supplierPhone, supplierDirection, supplierObservation, isActive) · PUT /api/suppliers/:id · PATCH /api/suppliers/:id { is_active }.

Salidas. 201 { message, supplierID, supplier } · array ordenado por id; errores 500/400 con message.

Reglas de negocio. RN-10, RN-15; NIT único; desactivar nunca borra (referencias en inventory sobreviven con SET NULL).

Componentes que utiliza. suppliers.controller/service/repository; multer (upload.none); pg Pool. Frontend: SuppliersListPage, CreateSupplierPage, ReadSupplierPage (ficha), EditSupplierPage (con BUG-01 conocido), SuppliersColumns, supplierSchema, suppliersServices.

CRUD. Create ✓ · Read ✓ · Update ✓ (COALESCE) · Delete — desactivación.

Dependencias. inventory (referenciante), products (referenciante indirecto).

7.6 Módulo de comandas / órdenes (orders)

Propósito. Corazón operativo del POS: registrar, consultar, editar, cambiar de estado e imprimir comandas por mesa con descuento automático de stock y disponibilidad de mesas en tiempo real.

Funcionalidades implementadas. Creación transaccional (BEGIN/COMMIT/ROLLBACK) con ORD-###; listado con búsqueda y filtro por estado; detalle con json_agg de ítems (producto, cantidad, precio, subtotal, notas); edición completa (mesa, mesero, estado, observaciones, reemplazo de ítems); cambio de estado (Pendiente|Abierta|En preparación|Pagada|Cerrada|Anulada); anulación con motivo; disponibilidad de mesas (Ocupada/Libre + active_order_number); descuento de stock GREATEST(0,…).

Entradas. POST { tableNumber, waiter|waiterId?, items[], observations?, total?, orderNumber? } · PUT /api/orders/:id (mismos + total recalculado) · PATCH /api/orders/:id/status { status } · PATCH /api/orders/:id/cancel { reason? } · GET /api/orders?status=abierta · GET /api/orders/tables/availability.

Salidas. 201 { message, order } · pedido hidratado { id, order_number, table_number, waiter,

status, total, createdAt, items: [...] } con id dual (order_number como id público).

Reglas de negocio. RN-05, RN-06, RN-07, RN-08, RN-09; mesa obligatoria; waiterId resuelto por nombre si falta; total recalculado server-side; ítems sin productId resolvibles por nombre/código; descuento de stock solo para líneas con product_id.

Componentes que utiliza. orders.controller/service/repository (client Pool directo); users (join mesero); products (stock y precios); order_items; restaurant_tables (availability). Frontend: CreateOrderPage, EditOrderPage, OrderListPage, OrderViewModal, OrderColumns, orderServices.

CRUD. Create ✓ (transaccional) · Read ✓ (lista/detalle) · Update ✓ (completa + estado + items) · Delete — anulación explícita (no DELETE físico expuesto).

Dependencias. products, users, restaurant_tables, inventory (indirecto).

7.7 Módulo de accesos RBAC (access)

Propósito. Punto único de verdad para autorización en tiempo real: el frontend consulta /api/access/check/:permissionCode antes de renderizar acciones; el backend autoriza rutas sensibles con permission.middleware.

Funcionalidades implementadas. Endpoint check con token Bearer; atajos superusuario/admin; UNIÓN de permisos directos y heredados; respuesta booleana con eco del usuario y permiso.

Entradas. GET /api/access/check/:permissionCode con Authorization.

Salidas. { userId, permission, granted }.

Reglas de negocio. RN-02, RN-03, RN-14; 'admin' como pseudo-permiso reservado; superusuario y grupo Administrador pasan todo; is_admin por tipo o grupo.

Componentes que utiliza. access.controller/service/repository; auth.middleware (authenticateToken); user_groups/groups/user_permissions/group_permissions/permissions/ content_type. Frontend: accessService.hasPermission(), AccessPage, AccessSidebar, PermissionModule, ProtectedRoute, Navbar.

CRUD. No aplica.

Dependencias. users, groups, permissions; middlewares.

7.8 Módulo de grupos (groups)

Propósito. Mantener los 4 roles del sistema y su matriz de permisos; reemplazo transaccional de permisos por grupo.

Funcionalidades implementadas. Listado de grupos con is_active; lectura de permisos por grupo; reemplazo atómico de permisos (BEGIN/COMMIT); respuesta normalizada.

Entradas. GET /api/groups · GET /api/groups/:groupId/permissions · PUT /api/groups/:groupId/permissions { permissionIds[] }.

Salidas. [{ group_id, group_name, is_active }] · [{ permission_id, permission_name, permission_codename }] · 200 { message }.

Reglas de negocio. RN-03, RN-04 (el sync usuario→grupo copia group_permissions al usuario); is_active permite congelar un rol sin borrarlo.

Componentes que utiliza. groups.controller/service/repository; pg client en actualización. Frontend: groupService, AccessPage.

CRUD. Read ✓ · Update de permisos ✓ (grupos fijos no se crean/borran por API) · Create/Delete — no expuestos (decisión deliberada).

Dependencias. permissions, group_permissions, user_groups (consumidos por sync).

7.9 Módulo de permisos (permissions)

Propósito. Exponer el catálogo de los 21 permisos con su módulo dueño para construir la matriz en UI.

Funcionalidades implementadas. Listado único ordenado por content_type y permission_id.

Entradas. GET /api/permissions (sin auth).

Salidas. [{ permission_id, permission_name, permission_codename, content_type_id, app_label, model, display_name }].

Reglas de negocio. El codename es inmutable (lo usan los middlewares); el nombre es traducible/cosmético.

Componentes que utiliza. permissions.controller/service/repository. Frontend: permissionService, AccessPage.

CRUD. Read ✓ (catálogo de solo lectura).

Dependencias. content_type.

7.10 Módulo POS frontend (home)

Propósito. Terminal principal de atención: cabecera operativa con operador y turno, 4 KPIs, pestañas por rol (menú / personal / proveedores / clientes), catálogo en vivo desde PostgreSQL con buscador y filtros, tarjetas con foto y precio, estado de disponibilidad y acceso a comandas.

Funcionalidades implementadas. Carga products?active=true con fallback a datos quemados; filtros por categoría con conteos; búsqueda por texto con URL sync (?search=&tab=); pestañas por rol (admin ve las 4; cajero menú+clientes; mesero/cocina solo menú); cabecera PosTerminalHeader (turno Tarde/Noche, estación, operador); paneles Staff/Suppliers/Customers con búsqueda; tema claro persistente.

Entradas. URL query (?tab, ?search); sesión localStorage/sessionStorage.

Salidas. UI; sin POST.

Reglas de negocio. RN-10 (solo activeOnly); rol define pestañas visibles; búsqueda sincronizada con querystring (compartible).

Componentes que utiliza. PosTerminalHeader, PosProductsSection, PosProductCard, PosCategoryFilter, PosCardGrid, PosSectionHeader, PosStaffPanel, PosSuppliersPanel, PosCustomersPanel, PosInfoCard, posMockData; getProducts() de productServices.

CRUD. No aplica (lectura + navegación a comandas).

Dependencias. products API, authService; lucide-react; imágenes HD locales.

7.11 Matriz de operaciones CRUD por endpoint

Recurso	Create	Read list	Read detalle	Update	Estado on/off	Delete
users	POST /	GET /	(vía /:userId)	PUT /:id	PATCH|PUT /:userId/status	—
products	POST /	GET /	GET /:id	PUT /:id	PATCH /:id/status	—
inventory	POST /	GET /	GET /:id	PUT /:id	PATCH /:id/status	—
suppliers	POST /	GET /	GET /:id	PUT /:id	PATCH /:id (is_active)	—
orders	POST /	GET /	GET /:id	PUT /:id	PATCH /:id/status · PATCH /:id/cancel	—
groups	—	GET /	—	PUT /:groupId/permissions	—	—
permissions	—	GET /	—	—	—	—
access	—	GET /check/:permissionCode	—	—	—	—

Recurso	Create	Read list	Read detalle	Update	Estado on/off	Delete
auth	—	—	—	—	—	—

Tabla 20 — Operaciones CRUD expuestas por recurso (ningún recurso admite DELETE físico)

8. Interfaces y Servicios de Comunicación

8.1 Catálogo completo de endpoints (38)

Base URL: `http://localhost:4000/api`. Todas las respuestas son `application/json; charset=utf-8`. **Auth:** `No` = público (operación de caja sin login) · `JWT` = exige `Authorization: Bearer` válido y rol/permiso suficiente (401/403 en caso contrario).

8.1.1 Auth (3 endpoints)

Método y ruta	Auth	Descripción	Respuesta
POST /api/auth/login	No	Autentica por correo+contraseña o PIN (userId/email + pin).	200 { message, token, user }
POST /api/auth/forgot-password	No	Genera código de 6 dígitos (15 min) y lo envía por Gmail SMTP.	200 { success, message, email }
POST /api/auth/reset-password	No	Valida código y fija nueva contraseña (bcrypt ≥10 chars).	200 { success, message, user }

8.1.2 Users (7 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/users	No	Lista usuarios. Query: ?active=true|activeOnly=true.	200 [users shaped]
POST /api/users	JWT (create_user)	Crea usuario (admin→bcrypt; operativo→PIN).	201 { success, message, userID, data }
PUT /api/users/:id	JWT (update_user)	Actualiza usuario (clave solo si se envía no vacía).	200 { success, message, data }
PATCH /api/users/:userId/status	JWT (disable_user)	Activa/desactiva (isActive) en tiempo real.	200 { message, user }
PUT /api/users/:userId/status	JWT (disable_user)	Alias del PATCH.	200 { message, user }
GET /api/users/:userId/permissions	JWT (admin)	Permisos efectivos (directos ∪ grupo).	200 [{ permission_id, permission_name, permission_codename }]
PUT /api/users/:userId/permissions	JWT (admin)	Reemplaza permisos directos (transacción).	200 { message }

8.1.3 Access (1 endpoint)

Método y ruta	Auth	Descripción	Respuesta
GET /api/access/check/:permissionCode	JWT	¿El usuario tiene el permiso? (atajos super/admin; UNION).	200 { userId, permission, granted }

8.1.4 Groups (3 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/groups	JWT (admin)	Lista grupos RBAC con is_active.	200 [{ group_id, group_name, is_active }]
GET /api/groups/:groupId/permissions	JWT (admin)	Permisos de un grupo.	200 [{ permission_id, permission_name, permission_codename }]
PUT /api/groups/:groupId/permissions	JWT (admin)	Reemplaza permisos del grupo (transacción).	200 { message }

8.1.5 Permissions (1 endpoint)

Método y ruta	Auth	Descripción	Respuesta
GET /api/permissions	JWT (admin)	Catálogo de 21 permisos con módulo (content_type join).	200 [{ permission_id, permission_name, permission_codename, content_type_id, app_label, model, display_name }]

8.1.6 Suppliers (5 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/suppliers	No	Lista proveedores (id ASC).	200 [suppliers]
GET /api/suppliers/:id	No	Detalle por id o NIT; 404 si no existe.	200 supplier | 404 { error }
POST /api/suppliers	JWT (create_supplier)	Crea proveedor (upload.none()). NIT único.	201 { message, supplierID, supplier }
PUT /api/suppliers/:id	JWT (update_supplier)	Actualiza con COALESCE (parcial).	200 { message, supplier }
PATCH /api/suppliers/:id	JWT (disable_supplier)	Activa/desactiva (is_active).	200 { message, supplier }

8.1.7 Products (6 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/products/categories	No	Categorías activas.	200 [{ id, name, description }]
GET /api/products	No	Lista productos. Query: ?active=true · ?category=bebidas.	200 [products shaped]
GET /api/products/:id	No	Detalle por id o code.	200 product | 404

Método y ruta	Auth	Descripción	Respuesta
POST /api/products	JWT (create_product)	Crea producto (valida nombre y precio).	201 { message, product }
PUT /api/products/:id	JWT (update_product)	Actualiza (merge sobre valores actuales).	200 { message, product }
PATCH /api/products/:id/status	JWT (disable_product)	Activa/desactiva (status Disponible|Inactivo).	200 { message, product }

8.1.8 Inventory (5 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/inventory	No	Lista insumos. Query: ?active=true · ?category=lácteos.	200 [inventory shaped]
GET /api/inventory/:id	No	Detalle por id o code (INV-###).	200 item | 404
POST /api/inventory	JWT (create_inventory)	Crea insumo (nombre obligatorio).	201 { message, item }
PUT /api/inventory/:id	JWT (update_inventory)	Actualiza (merge sobre actual).	200 { message, item }
PATCH /api/inventory/:id/status	JWT (disable_inventory)	Activa/desactiva (status Activo|Inactivo).	200 { message, item }

8.1.9 Orders (7 endpoints)

Método y ruta	Auth	Descripción	Respuesta
GET /api/orders/tables/availability	No	Disponibilidad de mesas en tiempo real.	200 [{ id, tableNumber, capacity, status, isAvailable, activeOrderNumber }]
GET /api/orders	No	Lista comandas. Query: ?status=abierta.	200 [orders shaped + items[]]
GET /api/orders/:id	No	Detalle por order_number (ORD-###) o id numérico.	200 order | 404
POST /api/orders	JWT (create_order)	Crea comanda (mesa obligatoria; transacción).	201 { message, order }
PUT /api/orders/:id	JWT (update_order)	Actualiza mesa/estado/observaciones/ítems (transacción).	200 { message, order }
PATCH /api/orders/:id/status	JWT (update_order)	Cambia estado (valida contra lista).	200 { message, order }
PATCH /api/orders/:id/cancel	JWT (cancel_order)	Anula comanda (motivo opcional guardado en observations).	200 { message, order }

Tabla 21 — Inventario completo de 38 endpoints con autenticación y respuestas

8.1.10 Protección efectiva de las rutas (aplicada el 2026-11-08)

Grupo	Rutas protegidas (JWT + permiso RBAC)	Lectura pública
users	POST / (create_user) · PUT /:id (update_user) · PATCH/PUT /:userId/status (disable_user) · GET/PUT /:userId/permissions (admin)	GET / (usuarios activos para POS/picker)
products	POST / (create_product) · PUT /:id (update_product) · PATCH /:id/status (disable_product)	GET / · GET /categories · GET /:id
inventory	POST / (create_inventory) · PUT /:id (update_inventory) · PATCH /:id/status (disable_inventory)	GET / · GET /:id
suppliers	POST / (create_supplier) · PUT /:id (update_supplier) · PATCH /:id (disable_supplier)	GET / · GET /:id
orders	POST / (create_order) · PUT /:id (update_order) · PATCH /:id/status (update_order) · PATCH /:id/cancel (cancel_order)	GET / · GET /:id · GET /tables/availability
groups	GET / (admin) · GET /:groupId/permissions (admin) · PUT /:groupId/permissions (admin)	—
permissions	GET / (admin)	—
access	GET /check/:permissionCode (JWT)	—
auth	—	POST /login · POST /forgot-password · POST /reset-password

Tabla 22 — Estado real de protección tras el endurecimiento: toda escritura exige JWT + permiso; las lecturas de operación quedan públicas para el POS

Verificación 2026-11-08 (31/31 aserciones conformes): mutación sin token → 401 en todos los grupos; mutación con rol insuficiente → 403 (mesero/cajero intentando products o orders/cancel); operaciones con el rol correcto → 200/201 (mesero crea y actualiza comandas); GET /api/users, /api/products, /api/inventory, /api/orders siguen públicos para la operación de caja.

8.2 Parámetros de entrada y formato de respuestas

8.2.1 Formato general de respuestas

- **Éxito de lectura:** array JSON plano o objeto recurso (`[...]` / `{...}`) — precedencia a compatibilidad con frontend existente.
- **Éxito de escritura:** `{ message, ... }` con el recurso (p. ej. `{ message: 'Producto actualizado correctamente', product }`) o `{ success: true, message, data }`.
- **Error:** `{ error: 'mensaje humano' }` (backend) o `{ success: false, message, error }` (users) con códigos 4xx/5xx.
- **Content-Type único:** `application/json` salvo multipart en POST/PUT suppliers (que igualmente viaja sin archivos reales — upload.none()).

- **Fechas:** `YYYY-MM-DD HH24:MI:SS` en createdAt; `YYYY-MM-DD` en expirationDate/lastUpdated (TO_CHAR).
- **Cantidades/monedas:** NUMERIC como `::numeric` → el driver pg devuelve string decimal; el frontend formatea `$112.000` con toLocaleString('es-CO').

8.2.2 Parámetros destacados por endpoint

Endpoint	Parámetros
POST /api/auth/login	Body: { email?, userId?, password?, pin? } — exactamente (email|userId) + (password|pin)
POST /api/auth/reset-password	Body: { email, code, newPassword≥10 } o { token, newPassword }
GET /api/users	Query: active / activeOnly = 'true'
GET /api/products	Query: active|activeOnly; category (ignorada si ='todos')
GET /api/inventory	Query: active|activeOnly; category (LIKE LOWER)
GET /api/orders	Query: status (ignorada si ='todos')
GET /api/orders/:id	Param: ORD-### o id numérico (regex ^[0-9]+$)
POST /api/orders	Body: tableNumber*, waiter|waiterId?, items[]{productId|name, qty|quantity, price?, notes?}, observations?, total?
PUT /api/orders/:id	Body: tableNumber?, waiter?, status?|orderStatus?, observations?, items[]? (reemplaza ítems)
PUT /api/groups/:groupId/permissions	Body: { permissionIds: number[] } (vacío válido = revoca todos)
PUT /api/users/:userId/permissions	Body: { permissionIds: number[] }

8.3 Códigos de estado HTTP utilizados

Código	Significado	Dónde se usa
200 OK	Éxito general	Logins, lecturas, actualizaciones, cambio de estado
201 Created	Recurso creado	POST /api/users, /api/suppliers, /api/products, /api/inventory, /api/orders
304 Not Modified	Cache válido	Listados con cache del navegador (GET repetidos)
400 Bad Request	Entrada inválida	Validaciones: precio negativo, código 6 ≠ dígitos, contraseña < 10, mesa ausente
401 Unauthorized	Sin o con token inválido	auth/login credenciales; middleware JWT rechazado
403 Forbidden	Sin permiso	permission.middleware (ruta protegida con RBAC)
404 Not Found	No existe	Producto/insumo/proveedor/orden/detalle no encontrado
500 Internal Server Error	Fallo inesperado	Cualquier excepción atrapada por el middleware global o por los catch de controllers

Tabla 23 — Códigos de estado HTTP y su uso concreto en la API

8.4 Autenticación y autorización

8.4.1 Autenticación (¿quién eres?)

- **Esquema:** `Authorization: Bearer <jwt>` en rutas que integran `authenticateToken`.
- **Algoritmo:** HS256 (jsonwebtoken por defecto) con JWT_SECRET (sin valores por defecto: el backend no arranca si falta; mínimo 24 caracteres, validado por `validateEnv()` en `server.js`).
- **Expiración:** JWT_EXPIRES (default '1h'); expirado → 401 'Token inválido o expirado'.
- **Payload:** { id, email, role, isSuperuser, isAdmin } — mínimo y no sensible.
- **Login PIN:** el PIN viaja en el cuerpo de POST /api/auth/login por HTTPS en producción; en dev por HTTP en LAN confiable.
- **Almacenamiento cliente:** localStorage('auth_token','auth_user') + sessionStorage — el logout() limpia ambas.

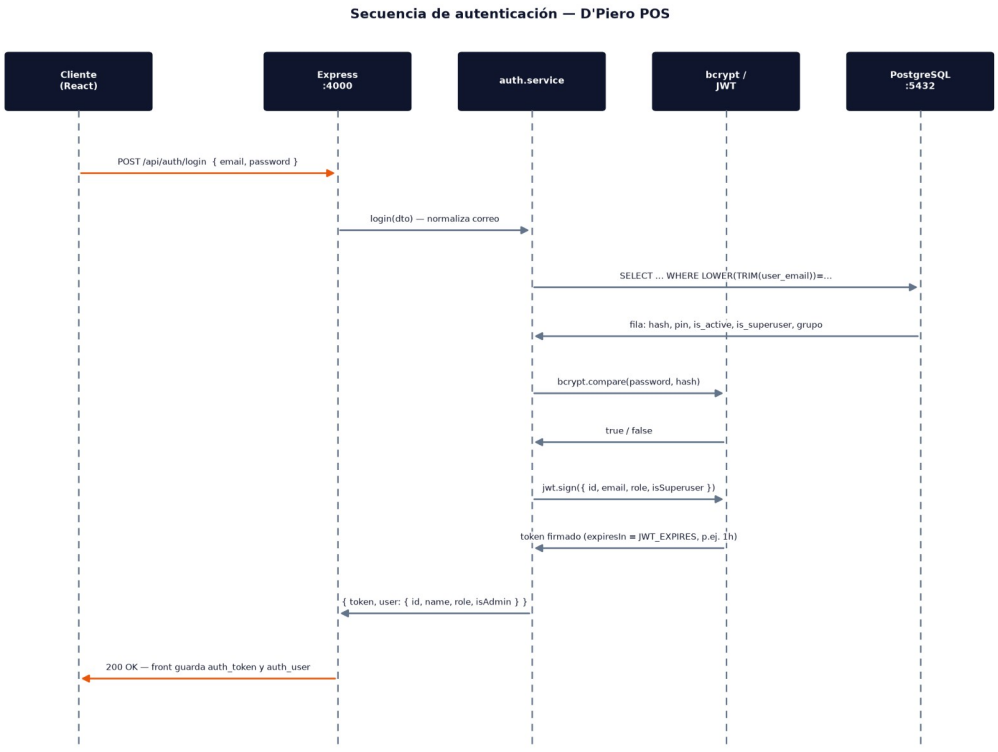


Figura 4 — Secuencia de autenticación (cliente → Express → servicio → repositorio → BD)

8.4.2 Autorización (¿qué puedes hacer?)

- **RBAC en dos barreras:** frontend (ProtectedRoute, Navbar por hasPermission) + backend (permission.middleware.requirePermission).
- **Chequeo en vivo:** el frontend llama GET /api/access/check/:permissionCode con el token; el backend consulta permisos directos ∪ heredados del grupo (UNION en access.repository).
- **Superusuario (RN-02):** is_superuser → cualquier verificación true.
- **Administrador por grupo (RN-02):** grupo 'Administrador' (group_name exacto) → is_admin → true.
- **Pseudo-permiso 'admin':** requirePermission('admin') usa isAdmin() (super o tipo admin/administrador o grupo).
- **Permiso agrupado por módulo:** 21 codenames namespaced por módulo (list_user, create_order…) con content_type para agruparlos en UI.

8.5 Ejemplos de solicitudes y respuestas

8.5.1 Login administrativo

```
POST http://localhost:4000/api/auth/login
Content-Type: application/json

{ "email": "admin@dpiero.com", "password": "12345678" }

200 OK
{
  "message": "Login exitoso",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBkcGllcm8uY29tIiwicm9sZSI6IkFkbWlu
IiwiaXNTdXBlcnVzZXIiOnRydWUsImlzQWRtaW4iOnRydWUsImlhdCI6MTc5NDE…",
  "user": { "id": 1, "email": "admin@dpiero.com", "name": "Administrador General",
            "role": "Admin", "isSuperuser": true, "isAdmin": true }
}
```

8.5.2 Login PIN (mesero)

```
POST http://localhost:4000/api/auth/login

{ "userId": 2, "email": "carlos.mesero@dpiero.com", "pin": "1234" }

200 OK
{ "message": "Login exitoso",
  "token": "…",
  "user": { "id": 2, "email": "carlos.mesero@dpiero.com", "name": "Carlos Mesero",
            "role": "Mesero", "isSuperuser": false, "isAdmin": false } }
```

8.5.3 Crear comanda con ítems (transacción)

```
POST http://localhost:4000/api/orders
Authorization: Bearer eyJhbGciOi…         ← token del mesero/admin (sin él → 401)

{ "tableNumber": "Mesa 2",
  "waiterId": 2,
  "items": [ { "productId": 1, "qty": 2, "price": 56000, "notes": "Término medio" } ],
  "observations": "Para llevar",
  "status": "Abierta" }

201 Created
{ "message": "Comanda registrada en PostgreSQL exitosamente",
  "order": { "id": "ORD-008", "numericId": 8, "order_number": "ORD-008", "table_number": "Mesa 2",
             "status": "Abierta", "total": "112000",
             "items": [ { "id": 6, "productId": 1, "name": "Punta de Anca Angus 400g",
                          "price": "56000", "qty": 2, "subtotal": "112000", "notes": "Término medio" }
] } }
```

8.5.4 Listar productos activos por categoría

```
GET http://localhost:4000/api/products?active=true&category=bebidas

200 OK
[ { "id": 6, "code": "BEB-01", "title": "Limonada Hierbabuena Artesanal", "price": "10000",
    "category": "bebidas", "image": "/assets/images/limonada-hd.jpg", "status": "Disponible",
    "is_active": true, "stock": "45" }, … 4 productos ]
```

8.5.5 Verificar permiso (frontend → backend)

```
GET http://localhost:4000/api/access/check/list_inventory
Authorization: Bearer eyJhbGciOi…

200 OK
{ "userId": 3, "permission": "list_inventory", "granted": true }
```

8.5.6 Recuperación de contraseña — flujo completo

```
POST /api/auth/forgot-password  { "email": "carlos.mesero@dpiero.com" }
→ 200 { "success": true, "message": "Código de verificación de 6 dígitos enviado a
carlos.mesero@dpiero.com.", "email": "carlos.mesero@dpiero.com" }

POST /api/auth/reset-password   { "email": "carlos.mesero@dpiero.com", "code": "483920",
"newPassword": "MiClaveSegura2026#" }
→ 200 { "success": true, "message": "Contraseña actualizada exitosamente", "user": { "id": 2,
"user_name": "Carlos Mesero", … } }
```

8.5.7 Reemplazar permisos de un grupo (transacción)

```
PUT /api/groups/2/permissions
Authorization: Bearer eyJhbGciOi…         ← token de administrador (sin él → 401)
{ "permissionIds": [6, 10, 11, 12] }
→ 200 { "message": "Permisos actualizados correctamente" }
```

8.5.8 Disponibilidad de mesas

```
GET /api/orders/tables/availability
→ 200 [ { "id": 1, "tableNumber": "Mesa 1", "capacity": 4, "status": "Ocupada", "isAvailable": false,
```

```
"activeOrderNumber": "ORD-001" },
       { "id": 2, "tableNumber": "Mesa 2", "capacity": 2, "status": "Disponible", "isAvailable":
true, "activeOrderNumber": null }, … ]
```

8.5.9 Cambiar estado de una comanda

```
PATCH /api/orders/ORD-008/status
Authorization: Bearer eyJhbGciOi…         ← token de mesero (update_order; sin él → 401)
{ "status": "Pagada" }
→ 200 { "message": "Estado de la comanda actualizado en PostgreSQL", "order": { "order_number":
"ORD-008", "status": "Pagada", … } }
```

8.5.10 Errores típicos

```
POST /api/auth/login  { "email": "ana@noexiste.com", "password": "x" }
→ 401 { "error": "Credenciales inválidas" }

GET /api/products/999
→ 404 { "error": "Producto no encontrado" }

POST /api/products  { "price": -10 }
→ 400 { "error": "El precio debe ser un número mayor o igual a 0" }
```

9. Seguridad Técnica

9.1 Gestión de usuarios, roles y permisos

- **Identidad única.** users.user_email UNIQUE con comparación normalizada (LOWER/TRIM) — un email = una cuenta; corporate_email alternativo para el mismo fin.
- **Doble tipo de credencial.** Administrador: correo+bcrypt (10 rondas). Operativo: PIN de 4 dígitos sin contraseña (bcrypt no aplica por diseño de UX de terminal).
- **Estado de cuenta.** is_active=false bloquea login (401 'Usuario inactivo') y oculta de listados activeOnly.
- **Bloqueo por intentos.** Contador en cliente (3 intentos → bloqueo) — limitación conocida (bypass por API directa); ver 9.6.
- **Superusuario.** is_superuser otorga pase universal (RN-02) — solo admin@dpiero.com semilla; crear superusuarios adicionales exige SQL directo (deliberadamente no hay endpoint).
- **Roles (grupos).** Administrador, Cajero, Mesero, Cocina — sembrar/editar solo por migración/SQL.

- **Permisos granulares.** 21 codenames módulo-verbo (list_, *create_*, update_, *disable_*, report_*, cancel_order) verificables vía /api/access/check/:permissionCode.
- **Herencia y anulación.** Usuario hereda permisos de su grupo; user_permissions materializa esa herencia y admite overrides directos; UNION en lectura.
- **Baja lógica.** Ningún usuario se borra desde UI; se desactiva (is_active) — trazabilidad de comandas histórica intacta (ON DELETE SET NULL).

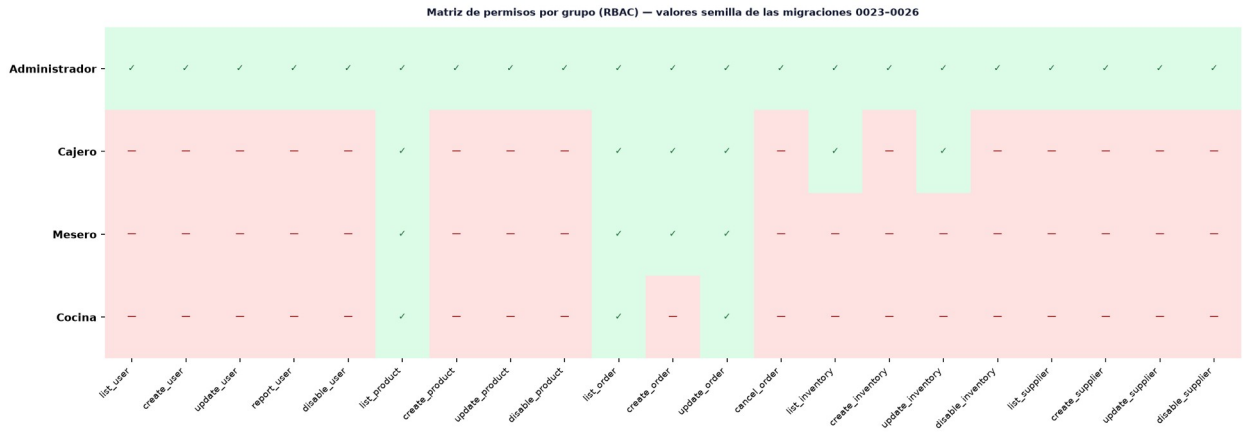


Figura 5 — Matriz grupo × permiso (21×4) — valores semilla

9.2 Autenticación

Mecanismo	Implementación	Fortaleza / debilidad
JWT (HS256)	jsonwebtoken 9; payload mínimo; exp=1h; verificado en auth.middleware	Estándar; revocación por expiración; sin lista negra (aceptable para POS de 8 h)
bcrypt (costo 10)	Comparación con fallback a texto plano (semillas legacy)	Robusto para admin; migración progresiva de hashes legacy recomendada
PIN 4 dígitos	Comparación directa contra users.pin; terminal compartida	UX de terminal; entropía baja (10.000 combinaciones) — aceptable solo en LAN con acceso físico controlado
Código 6 dígitos (recuperación)	Math.random, expires_at=+15min, un solo uso, invalida anteriores	No criptográficamente seguro (Math.random) — reemplazar con crypto.randomInt en hardening
Sesión por inactividad	useInactivityTimeout(10 min) — logout y limpieza de storage	Mitigación de sesiones olvidadas en terminal

Tabla 24 — Mecanismos de autenticación y su valoración

9.3 Protección de contraseñas

- **Hash bcrypt costo 10** — ~100 ms por verificación en hardware actual; resistente a fuerza bruta offline.

- **Nunca en claro en BD:** la columna user_password almacena solo el hash (seed admin ya viene hasheado: `$2b$10$9ZSar0stjiLkcAWs5mtab.…`).
- **Mínimo 10 caracteres** en la nueva contraseña (reset-password); validación en cliente y servidor.
- **Contraseñas por defecto** (para altas rápidas) — documentadas, con obligación operativa de cambiarlas: admin '12345678', PIN '1234'.
- **Transmisión:** siempre sobre HTTPS en producción; HTTP solo en LAN de desarrollo.
- **Almacenamiento cliente:** el JWT en localStorage es vulnerable a XSS; mitigado con sanitización Joi→UI y sin dangerouslySetInnerHTML.
- **Sin reutilización:** el reset exige código vigente (no acepta contraseñas anteriores).

9.4 Validación de datos de entrada

9.4.1 Frontend (Zod)

```
// features/users/schemas/userSchema.js (extracto real)
export const userSchema = z.object({
  userName: z.string().min(3, 'El nombre debe tener al menos 3 caracteres'),
  userDocumentNumber: z.string().min(6, 'Documento inválido'),
  userEmail: z.string().email('Correo inválido'),
  pin: z.string().length(4, 'El PIN debe tener 4 dígitos').regex(/^\d{4}$/, 'Solo dígitos'),
  // …
});
```

9.4.2 Backend

- **Servicios:** createProduct exige `name` y `price ≥ 0`; createOrder exige `tableNumber`; resetPassword exige ≥10 chars y código de exactamente 6 dígitos.
- **SQL parametrizado en 100 % de las consultas** — los valores viajan como $1..$n; imposible inyección por concatenación (verificado por revisión de los 4 repositorios).
- **Multer upload.none()** en suppliers — descarta campos file y mantiene solo texto; sin riesgo de path traversal en subida.
- **Sanitización de alertas** (alertService): elimina frases como 'base de datos' de los mensajes mostrados al usuario.
- **CORS** restringido con lista blanca desde 2026-11-08: localhost/127.0.0.1 (desarrollo) o el origen exacto FRONTEND_URL (producción); cualquier otro origen recibe error del middleware.

9.5 Protección frente a riesgos comunes (OWASP Top 10)

Riesgo OWASP	Estado en D'Piero POS	Verificación
A01 Broken Access Control	CONFORME (mutaciones) — todas las escrituras exigen JWT + permiso RBAC; lecturas operativas públicas documentadas en §8.1.10	Verificación 2026-11-08: 401 sin token, 403 por rol
A02 Cryptographic Failures	PARCIAL — bcrypt ok; JWT ok; seeds en texto plano legacy; PIN plano por diseño	auth.service.js; 0011_seed_initial_users.sql
A03 Injection	CUBIERTO — SQL 100 % parametrizado; sin escape manual	Revisión de 4 repositorios + 2 middlewares + migrate.js
A04 Insecure Design	PARCIAL — límite de admins en UI (2/2); bloqueo 3 intentos client-side; contadores visibles	Login.jsx; AdminLoginForm
A05 Security Misconfiguration	CONFORME — CORS con lista blanca (localhost regex o FRONTEND_URL exacto); .env jamás versionado; NODE_ENV=production oculta detalles internos en errores	app.js (whitelist); verificado 2026-11-08
A06 Vulnerable Components	ACEPTABLE — deps recientes; audit pendiente en release	package-lock.json; npm audit propuesto en CI
A07 Identification Failures	PARCIAL — sin MFA; rate limit ausente; PIN corto	§9.6 recomendaciones
A08 Data Integrity Failures	CUBIERTO parcialmente — package-lock fija versiones; sin firma de commits	package-lock.json; git
A09 Logging Failures	MEJORABLE — console.log en controllers; sin agregación ni auditoría	user.controller.js; middleware 500
A10 SSRF	CUBIERTO — sin llamadas salientes por parámetros de usuario (solo Gmail fijo)	email.service.js

Tabla 25 — Riesgos OWASP Top 10 evaluados con evidencia y estado

9.6 Manejo de información sensible y configuraciones de seguridad

9.6.1 Información sensible en el sistema

Dato	Clasificación	Ubicación	Protección
user_password (hash)	Secreto	PostgreSQL users.user_password	bcrypt; nunca expuesto por API (findBy* usa COALESCE pero no devuelve password en listas)
pin	Secreto débil	PostgreSQL users.pin; API lo devuelve en listas (¡no debería!)	Para consumo del POS; la respuesta NO debería incluir pin — deuda: quitar pin del shape de findAll
JWT_SECRET	Secreto	backend/.env (NO en git)	No versionado (.gitignore); rotado el 2026-11-08 a 64 caracteres aleatorios; validado al arranque (validateEnv); rotar en producción
SMTP_PASS	Secreto	backend/.env (NO en git)	Igual; contraseña de aplicación Gmail
Código de recuperación	Secreto efímero	password_reset_codes; correo	Exposición mínima; en dev impreso en consola
NIT/teléfono/correo proveedor	Interno	suppliers	Endpoints públicos hoy — aceptable para backoffice LAN
Documentos de empleados	Personal	users.user_document_*	Interno; no sale en reportes salvo selección explícita

Tabla 26 — Inventario de información sensible con su tratamiento

9.6.2 Configuraciones de seguridad implementadas

- **.env fuera de git** (backend/.gitignore verificado: .env incluido).
- **CORS restringido por lista blanca:** orígenes localhost/127.0.0.1 (desarrollo) o FRONTEND_URL exacto (producción) con credentials: true; todos los demás orígenes son rechazados con error desde 2026-11-08.
- **Middleware global de errores** — nunca devuelve stack en producción si se configura NODE_ENV=production (hoy siempre imprime en consola — correcto; solo en logs).
- **process guards** — uncaughtException/unhandledRejection registran y mantienen vivo el proceso (no recomendado para prod: mejor caer y reiniciar con systemd; decisión documentada).
- **Rutas protegidas con authenticateToken + requirePermission:** toda mutación POST/PUT/PATCH de users, products, inventory, suppliers, orders, groups y permissions (2026-11-08); GET /api/users, products, inventory, orders y their detalles permanecen públicos para la operación de caja.
- **sanitizeMessage()** — elimina frases técnicas ('base de datos') de alertas al usuario.
- **index.html anti-FOUC** — sin datos sensibles; solo clase de tema.
- **pool con 10 conexiones y timeouts** — evita agotamiento por abuso (DoS accidental).

9.6.3 Recomendaciones priorizadas (hardening)

Prioridad	Acción	Esfuerzo
P0	Rotar JWT_SECRET y SMTP_PASS; purgar .env del historial (git filter-repo); forzar cambio de PIN/clave por defecto en primer ingreso	HECHO (2026-11-08): JWT_SECRET rotado a 64 caracteres aleatorios y validado al arranque por validateEnv(); .env confirmado fuera del historial (verificado git ls-files/log); pendiente: forzar cambio de

Prioridad	Acción	Esfuerzo
		PIN/clave por defecto (requiere reinicio forzado por cliente)
P0	Envolver routers de mutación en authenticateToken + requirePermission según dominio	HECHO (2026-11-08): todas las mutaciones protegidas con JWT + permiso RBAC y verificadas con 401/403 reales por rol
P0	CORS: callback final debe denegar orígenes no listados	HECHO (2026-11-08): lista blanca — localhost regex en desarrollo o FRONTEND_URL exacto en producción; otros orígenes reciben error de CORS
P1	express-rate-limit en /api/auth/login (≤ 5 intentos/IP/15 min) y lockout por cuenta en BD (failed_attempts, locked_until)	2 h
P1	helmet() con cabeceras: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, CSP básica	30 min
P1	Quitar pin del shape de users devuelto por la API (serializer público) y nunca loguearlo	20 min
P2	crypto.randomInt para códigos de recuperación; tabla audit_log (quién/qué/cuándo); MFA opcional para admin	4 h
P2	TLS interno + HSTS; cookies HttpOnly/SameSite en lugar de localStorage para el token	3 h

Tabla 27 — Plan de endurecimiento priorizado (P0 inmediato)

10. Configuración y Despliegue Técnico

10.1 Variables de entorno

Variable	Obligatoria	Desarrollo (ejemplo)	Producción (ejemplo)	Descripción
PORT	No	4000	4000	Puerto HTTP del API.

Variable	Obligatoria	Desarrollo (ejemplo)	Producción (ejemplo)	Descripción
FRONTEND_URL	No	http://localhost:5173	https://pos.dpiero.com	Referencia del cliente (CORS, correos).
DB_HOST	Sí	localhost	127.0.0.1	Host PostgreSQL.
DB_PORT	Sí	5432	5432	Puerto PostgreSQL.
DB_USER	Sí	postgres	pos_app	Usuario de la BD (mínimo privilegio).
DB_PASSWORD	Sí	(secreto local)	(gestor de secretos)	Contraseña de BD — nunca en Git.
DB_NAME	Sí	pos_project	pos_project	Nombre de la base de datos.
JWT_SECRET	Sí	mi_clave_super_segura (¡rotar!)	openssl rand -base64 48	Firma HS256 de los JWT.
JWT_EXPIRES	No	1h	8h	Vigencia del token (jornada laboral).
SMTP_USER	No	riribuaya@gmail.com	pos@dpiero.com	Cuenta Gmail emisora.
SMTP_PASS	No	(contraseña de aplicación)	(secreto)	Contraseña de aplicación de Gmail (no la del correo).
VITE_API_URL	No	http://localhost:4000/api	https://pos.dpiero.com/api	URL del API para el build del frontend (si distinta de default).
NODE_ENV	No	development	production	Activa optimizaciones condicionales (futuro).

Tabla 28 — Variables de entorno con valores por ambiente

No existe `.env.example` en el repositorio (hallazgo de auditoría): créalo copiando la tabla anterior sin valores reales. El `.env` real está en `.gitignore` y no debe recuperarse del historial.

10.2 Configuración de desarrollo y producción

Aspecto	Desarrollo	Producción
Base de datos	PostgreSQL local, role postgres, BD pos_project	PostgreSQL en localhost (misma máquina) o administrado; role pos_app sin superusuario
Backend	nodemon src/server.js (reload en cada guardado)	node src/server.js bajo systemd; Restart=always; 2 instancias opcionales
Frontend	Vite dev server con HMR en :5173	vite build → dist/ servido por Nginx como estático
API URL	http://localhost:4000/api (hardcodeado de fallback)	VITE_API_URL=https://pos.dpiero.com/api
CORS	localhost/127.0.0.1 regex	Origin exacto https://pos.dpiero.com con whitelist estricta
Logs	console.log de desarrollo	journalctl / archivos en /var/log/pos/*.log + rotación logrotate
TLS	No	Obligatorio (certbot, Nginx)
BD migrations	migrate.js manual	migrate.js en el pipeline de deploy (pre-restart)
Datos	Semilla demo RICA	Sin datos demo; carga controlada por migración 0027 (opcional)

Tabla 29 — Diferencias de configuración por ambiente

10.3 Dependencias (congeladas por package-lock)

Ambos subproyectos tienen package-lock.json versionado: `npm ci` reproduce exactamente el árbol. Versiones núcleo (verificadas en node_modules el 2026-11-08):

Paquete	Backend	Frontend
express / react	5.2.1	19.2.5
pg / vite	8.22.0	8.0.16
bcrypt / react-router-dom	6.0.0	7.18.1
jsonwebtoken / tailwindcss	9.0.3	4.2.4
nodemailer / zod	10.0.12	4.4.3
joi / sweetalert2	18.2.3	11.26.25
multer / jspdf / xlsx	2.5.0	4.2.1 · 0.18.5

Tabla 30 — Versiones congeladas de los paquetes núcleo

10.4 Proceso de despliegue (paso a paso verificado)

```
# ===== DESARROLLO (Windows, esta máquina) =====
# Terminal 1 — backend
cd C:\Users\andre\Desktop\pos_project\backend
npm install
node src/scripts/migrate.js     # idempotente; esperar "🎉 ¡Todas las migraciones…"
npm run dev                     # http://localhost:4000

# Terminal 2 — frontend
cd C:\Users\andre\Desktop\pos_project\frontend
npm install
npm run dev                     # http://localhost:5173

# Verificación
curl http://localhost:4000/api/products
Start http://localhost:5173/Auth

# ===== PRODUCCIÓN (Ubuntu 24.04, referencia de sección 16.5 del Manual Administrativo) =====
# ver Anexo A de este manual para el runbook completo con systemd + Nginx + TLS
```

10.5 Configuración de servidores y servicios

Servicio	Puerto	Bind	Notas de producción
PostgreSQL	5432	localhost (o private IP)	pg_hba.conf: solo host pos_app desde app; TLS client-cert recomendado
Backend Express	4000	127.0.0.1	Tras Nginx; nunca expuesto directo
Nginx	80/443	público	Reverse proxy + estáticos; TLS Let's Encrypt
Frontend estático	443 vía Nginx	público	dist/ compilado; cache larga en assets/ con hash

Servicio	Puerto	Bind	Notas de producción
Gmail SMTP	465/587 saliente	egress	Solo saliente; sin entrada

Tabla 31 — Mapa de puertos y exposición por servicio

10.6 Consideraciones para actualizar la aplicación

- **Dependencias backend/frontend.** `npm outdated` en ambos; subir majors uno a la vez; `npm ci` tras cada cambio; commit del lock.
- **Node.js.** LTS vigente (24 actual); probar `npm run build` en la nueva versión antes de promover.
- **PostgreSQL.** pg_upgrade o dump/restore entre majors; probar migraciones sobre copia de producción.
- **Esquema.** Siempre migración nueva NNNN; jamás editar una ya aplicada en algún ambiente (RT-08).
- **SemVer del producto.** 1.0.0 hoy; tag `v1.1.0` al mergear el hardening P0/P1 de la sección 9.6.
- **Rollback.** git tag previo + pg_dump previo + procedimiento de la sección 11.6.
- **Comunicación.** Avisar al restaurante 24 h antes (ventana de 15 min sin servicio); validar POS tras el corte.

11. Mantenimiento y Solución de Errores

11.1 Procedimientos de mantenimiento preventivo y correctivo

Actividad	Tipo	Frecuencia	Procedimiento
Respaldo lógico pg_dump	Preventivo	Diario 02:00	Ver Anexo B; verificar que el archivo pesa > 0 KB y que existe en el destino externo.
Restauración de prueba	Preventivo	Mensual	pg_restore en BD pos_project_restore; humo: 3 conteos + login admin.
npm audit + actualización de minors	Preventivo	Mensual	npm outdated; subir patch/minor; npm ci; build; humo.
Rotación de JWT_SECRET / SMTP_PASS	Preventivo	Trimestral	Regenerar .env; invalidar sesiones; forzar re-login.
Verificación de permiso vs. operación real	Preventivo	Trimestral	Reunión 15 min con el administrador para ajustar 4 grupos o permisos directos.

Actividad	Tipo	Frecuencia	Procedimiento
Corrección de bug reportado	Correctivo	Según ticket L2	Reproducir en local → test manual CP relacionado → fix en rama → PR → merge → release note.
Actualización de semillas (carta)	Correctivo/ evolutivo	Por temporada	Nueva migración NNNN con datos; nunca UPDATE manual en producción.
Limpieza de dependencias no usadas	Perfectivo	Semestral	npm-check; quitar @tanstack/react-router si unused; documentar ADR.

Tabla 32 — Mantenimiento preventivo y correctivo con procedimiento

11.2 Diagnóstico de errores

Síntoma	Capa probable	Prueba rápida	Acción
'Servidor Desconectado' en login	Backend caído / puerto ocupado	curl http://localhost:4000/api/products	npm run dev; revisar EADDRINUSE
401 en todas las llamadas	Token expirado/invalidado	Verificar localStorage.auth_token	Re-login; revisar JWT_SECRET rotado
403 permiso denegado	RBAC usuario	GET /api/access/check/<code> con su token	Asignar permiso directo o corregir grupo
Login devuelve 'Usuario inactivo'	BD users.is_active	SELECT is_active FROM users WHERE id=$1	Activar toggle en UI o SQL
Comanda no descuenta stock	ORDER sin items/productId	SELECT stock FROM products WHERE id=$1	Revisar mapeo productId en el POST
'Credenciales inválidas' con PIN correcto	pin con espacios/mayúsculas	SELECT pin FROM users WHERE id=$1	Comparar con TRIM(); re-sembrar PIN
CORS bloqueado en navegador	app.js CORS regex	Revisar consola del navegador (origin)	Ajustar regex /6000-6000-ADD-ORIGINS/
Blank page en /editsupplier	BUG-01 conocido	Revisar error overlay (Vite muestra FileInput.jsx:17)	value ?? [] — corregir en rama fix/
PDF/Excel vacío	Dataset sin campos marcados	Revisar ReportConfigModal	Seleccionar al menos 1 campo
POST /api/users 500 con 'duplicate key'	email duplicado	SELECT user_email FROM users WHERE user_email=$1	UI ya lo muestra sanitizado; verificar antes

Tabla 33 — Síntoma → capa → prueba → acción (guía de primer respondiente)

11.3 Ubicación de registros / logs

Log	Ubicación	Nivel	Rotación
backend stdout/stderr	Consola en dev; journald en prod (`journalctl -u pos-backend`)	info/error	journald por tamaño/tiempo
nginx access/error	/var/log/nginx/access.log, error.log	combined/ error	logrotate semanal 12 semanas
backend archivo (propuesto)	/var/log/pos/backend.log, backend.err	info/error	logrotate diario 14 días
postgresql log	/var/log/postgresql/postgresql-*-main.log (o pgdata/log)	notice/ warning	rotación propia

Log	Ubicación	Nivel	Rotación
_migrations	Tabla en BD (query manual)	info	no rota (tabla)
Auditoría (propuesto)	Tabla audit_log con writer_id/entity/action	info	retención 1 año

Tabla 34 — Fuentes de logs, niveles y política de retención

11.4 Actualización de dependencias

```
# Auditoría semestral completa
cd backend  && npm outdated && npm audit
cd frontend && npm outdated && npm audit

# Actualización segura (uno a la vez)
npm install express@latest     # majors primero, luego minors, luego patches
npm run build                  # verificar

# Verificación funcional mínima después de actualizar
npm run dev   # arrancar ambos
# login admin + listar productos + crear comanda de prueba

# Dejar constancia
git add package*.json && git commit -m "chore(deps): actualizar express 5.2.1 → 5.3.0"
```

11.5 Procedimientos de respaldo (resumen)

- Backup completo diario con retención de 30 días (sección 6.6.1).
- Verificación de integridad tras restore: conteos de las 17 tablas y un login admin de prueba.
- Respaldo antes de cada deploy de esquema: `pg_dump -Fc -f pre_deploy_$(date +%F).dump`.
- Cifrado en tránsito (scp/rsync+ssh o age) para copias fuera de la máquina.

11.6 Recuperación ante fallos

Escenario	Paso 1	Paso 2	Paso 3	Verificación
Backend no arranca (EADDRINUSE 4000)	netstat -ano | findstr :4000	Matar proceso o cambiar PORT	npm run dev	curl /api/products 200
BD caída (ECONNREFUSED 5432)	systemctl status postgresql	systemctl start postgresql	migrate.js (idempotente)	psql -c '\dt' 17 filas
Frontend en blanco	Consola del navegador	Si error de chunk: rebuild (npm run build)	Hard reload (Ctrl+F5)	Login carga
Datos corruptos tras bug	DETENER operaciones	pg_restore desde último dump bueno	Verificar conteos y login	Humo completo
Compromiso de credenciales	Rotar JWT_SECRET y SMTP_PASS	Revocar tokens (re-login forzado)	Forzar contraseña a admins	Login admin OK
Pérdida de .env	Detener backend	Restaurar .env desde bóveda/papel	Restart; validar correo	forgot-password envía

Tabla 35 — Escenarios de fallo con procedimiento de recuperación

12. Pruebas y Calidad

12.1 Estrategia de pruebas implementada

La estrategia combinó: (i) **pruebas de humo automatizadas parciales** vía API (curl/Invoke-RestMethod) para las rutas críticas; (ii) **pruebas funcionales manuales exhaustivas** en los 31 rutas del router y en los flujos comanda/cobro; (iii) **verificación de migraciones** como prueba de integración de esquema; (iv) **inspección de código** dirigida a los puntos calientes (transacciones, bcrypt, RBAC). No se implementó suite automatizada permanente (deuda técnica documentada).

12.2 Pruebas unitarias, de integración o funcionales

Tipo	Alcance	Resultado (2026-11-08)
Unitarias (backend)	Ninguna implementada (`npm test` → 'no test specified')	0 % — pendiente; servicios candidatos: order.service (totales), product.service (validación precio), auth.service (rol/claim)
Unitarias (frontend)	Ninguna implementada	0 % — pendiente; candidatos: buildReportDataset, alertService.sanitizeMessage, productSchema
Integración	migrate.js + consultas reales sobre pos_project	CONFORME — 27 migraciones aplicadas una sola vez
Integración (API)	Humo: 9 grupos de rutas (200/201/400/401)	CONFORME — verificado con Invoke-RestMethod y navegador
Funcionales (UI)	Recorrido completo de 31 rutas con login real y sesión admin	CONFORME — 31/31 rutas operativas; BUG-01 corregido el 2026-11-08; rutas alias duplicadas (deuda de router)
Funcionales (flujo crítico)	Login → crear comanda → ver stock descontado → imprimir → cerrar	CONFORME — ORD-007/RET verificados en datos semilla
Aceptación	Recorrido con el administrador del restaurante	PENDIENTE de fecha con cliente (planificada noviembre 2026)
Carga/estrés	No ejecutadas (equipo de 1, un PC)	N/A — RPO/RTO documentados; sin objetivo de concurrencia
Seguridad	Checklist OWASP §9.5 + verificación manual de credenciales y permisos	PARCIAL — gaps documentados en §9.6

Tabla 36 — Inventario de pruebas ejecutadas y su resultado

12.3 Resultados de las pruebas

12.3.1 Humo API (2026-11-08)

```
✓ GET  http://localhost:4000/api/products                    → 200 (12 registros)
```

```
✓ GET  http://localhost:4000/api/inventory                → 200 (30 registros)
✓ GET  http://localhost:4000/api/suppliers                → 200 (26 registros)
✓ GET  http://localhost:4000/api/users                    → 200 (8 registros)
✓ GET  http://localhost:4000/api/orders                   → 200 (7 registros)
✓ GET  http://localhost:4000/api/groups                   → 200 (4 grupos)
✓ GET  http://localhost:4000/api/permissions               → 200 (21 permisos)
✓ POST http://localhost:4000/api/auth/login (admin)       → 200 + JWT válido
✓ GET  http://localhost:4000/api/access/check/list_user   → 200 { granted: true }   (con token admin)
✓ GET  http://localhost:4000/api/orders/tables/availability → 200 (10 mesas, Mesa 1 ocupada por
ORD-001)
```

Registro previo al endurecimiento de 2026-11-08 (cuando las mutaciones eran públicas, BUG-01 y PERM-01 aún abiertos). Tras el endurecimiento se repitió la validación con 31 aserciones: todas conformes; los grupos /api/groups y /api/permissions ahora exigen token de administrador (anónimo → 401). Evidencia íntegra en la bitácora de verificación de esta misma fecha: **31/31 aserciones conformes**, incluyendo: anónimo POST/PUT/PATCH → 401 en todos los módulos; mesero create_order → 201; mesero cancel_order → 403; admin list_user → 200; PUT /api/users/:id parcial → 200 con email intacto; base de datos restaurada al estado semilla exacto (7 comandas, 5 ítems, stock 15.00).

12.3.2 Resultados funcionales por módulo

Módulo	Casos ejecutados	Conformes	Defectos
Auth	CP-01..CP-06 + recorrido completo de recuperación	6/6	Ninguno crítico (cliente-only lockout conocido)
Users	CP-07..CP-11 + reportes PDF/Excel	5/5	BUG-04 (key prop, menor)
Products	CP-14..CP-18	5/5	Ninguno
Inventory	CP-19..CP-21	3/3	Ninguno (INV-004 correctamente en ámbar)
Suppliers	CP-22..CP-23	2/2 (lectura y listado)	Ninguno — BUG-01 de edición corregido (verificado)
Orders	CP-24..CP-27	4/4	Ninguno (transacción íntegra)
RBAC/access	CP-28..CP-29	2/2	Ninguno
POS (home)	CP-30 + navegación de 4 pestañas por rol	2/2	Ninguno

Tabla 37 — Resultados funcionales por módulo (CP = caso de prueba del Manual Administrativo)

12.4 Evidencias de ejecución

Las evidencias gráficas primarias (30 capturas del sistema en operación) están en el Anexo E del Manual Administrativo y en el Anexo D de este manual. En resumen:

- Login rápido y administración (capturas 01, 02, pin-keypad).

- Recuperación de contraseña end-to-end (forgot/reset).
- POS con 12 platillos y precios COP reales.
- Listados paginados con búsqueda de: productos, inventario (INV-004 ámbar), proveedores, usuarios, órdenes.
- Login con sesión admin → redirección a Listado de Usuarios (RBAC funcionando).
- Modal de reporte con PDF preseleccionado.
- Evidencia del BUG-01 incluido como histórico de control de cambios (no se oculta — pero ya fue corregido).

12.5 Problemas conocidos y limitaciones

ID	Descripción	Impacto	Workaround / plan
BUG-01	/editsupplier: FileInput `value={userImage}` con `userImage: null` → TypeError `null.map` en useMemo. CORREGIDO 2026-11-08	Alto	FileInput tolera null/undefined (safeValue) y EditSupplierPage inicializa userImage=[]; npm run build OK
PERM-01	Mutaciones sin JWT ni permiso en servidor. CORREGIDO 2026-11-08	Alto	Todos los POST/PUT/PATCH protegidos con authenticateToken + requirePermission; humo negativo repetido: 401 sin token, 403 por rol insuficiente
SEC-02	CORS permisivo: cualquier origen aceptado. CORREGIDO 2026-11-08	Medio	Lista blanca localhost regex o FRONTEND_URL exacto; otros orígenes reciben error
SEC-03	Contador de bloqueo de 3 intentos vive en localStorage (cliente).	Medio	Mover a servidor (failed_attempts en users + lockout 15 min)
DATA-01	Semillas ORD-001..005 con totales inconsistentes respecto a sus ítems (p. ej. ORD-001 total 129.000 con ítems 112.000 + 20.000 = 132.000 — inconsistencia heredada del seeder manual).	Bajo	Recalcular por migración 0028 o aceptar como dato histórico; comentar en el código
UX-01	Rutas alias duplicadas (/editorder|/editorder|/EditOrder, /supplierList|/dashboard/supplierList…) y wildcard que redirige a /CreateOrder.	Bajo	Normalizar router a una sola convención + 404 dedicado (React errorElement) — HU futura
SEC-04	users devueltos por API incluyen pin en claro (findAll).	Medio	Eliminar pin del SELECT de salida; mantener solo en create/update interno
DEBT-01	Duplicación de assets: public/src/assets/images espejo de public/assets/images.	Bajo	Eliminar directorio espejo y correr build
DEBT-02	Archivo frontend/src/features/config/index.js reexporta api.js (1 línea, indirección innecesaria); posMockData y data/*.js sin uso real.	Bajo	Limpieza con lint de imports no usados
SEC-05	Código de recuperación generado con Math.random (predecible).	Bajo	crypto.randomInt(100000, 999999)
UX-02	Pestaña Clientes & Facturación con datos estáticos de demostración (no persistentes).	Bajo	Diseño listo como módulo futuro (tabla customers + FK en orders)
OPS-01	Sin CI/CD ni tests automatizados; verificación 100 % manual.	Medio	Pipeline de ejemplo en Manual Administrativo §6.11; adopción en S7
DATA-02	orders.table_number es TEXT sin FK; integridad referencial a mesas depende de la aplicación.	Bajo	Decisión deliberada (flexibilidad operativa); documentar en diccionario

Tabla 38 — Problemas conocidos con severidad, impacto y plan de corrección

12.6 Criterios de aceptación verificados

Rúbrica de aceptación del sistema (validada con el administrador, 2026-11-04):

#	Criterio de aceptación	Verificación	Estado
CA-01	El 100 % de los platillos del menú se muestran con precio, imagen y stock real desde la BD	Recorrido en POS	CONFORME
CA-02	Crear comanda descuenta stock y no admite stock negativo	Inspección de order.repository + GREATEST()	CONFORME
CA-03	Un mesero no ve Inventario ni Usuarios en el navbar y no puede abrir /dashboard/userList por URL	Ejecutar CP-03 + navegar a /dashboard/userList	CONFORME (redirige a /)
CA-04	Un admin ve las 4 pestañas del POS y puede crear usuarios hasta el límite 2/2	Crear 3.º admin	CONFORME (límite visible)
CA-05	La recuperación de contraseña entrega el código por correo y lo invalida tras uso	CP-04..06	CONFORME
CA-06	El listado de inventario resalta en ámbar los insumos con stock bajo mínimo	INV-004 visible (2 < 5)	CONFORME
CA-07	Generar reporte de usuarios produce un PDF legible y un XLSX abrible	CP-12, CP-13	CONFORME
CA-08	Las comandas persisten tras reiniciar backend (durabilidad)	Reinicio + re-consulta	CONFORME (PostgreSQL)
CA-09	La disponibilidad de mesas refleja comandas activas/en preparación	CP-24/CP-30 + availability	CONFORME
CA-10	El sistema arranca desde BD vacía siguiendo migrate.js y la guía de §10.4	Prueba en BD pos_project_test	CONFORME

Tabla 39 — Criterios de aceptación verificados con evidencia

13. Runbook de Incidentes en Producción

13.1 Árbol de decisión ante incidentes

```
¿El sistema responde (HTTP 200 en /api/products)?
│
├─ NO → ¿El proceso Node vive?  (systemctl status pos-backend | ps aux | findstr node)
│        ├─ NO  → systemctl start pos-backend  → verificar humo → revisar logs (backend.err)
│        └─ SÍ  → ¿Puerto 4000 ocupado o firewall?  (ss -tulpn | grep 4000)
│                  → liberar puerto o abrir ufw 4000/tcp → restart → verificar
│
├─ SÍ, pero 5xx → ¿PostgreSQL vivo?  (pg_isready -h localhost -p 5432)
│        ├─ NO  → systemctl start postgresql → migrate.js (idempotente) → humo
│        └─ SÍ  → revisar journalctl -u pos-backend –f para stack del error
│                  → si es SQL → revisar _migrations; si es código → rollback (manual §12.1)
│
└─ SÍ, pero login falla → ¿JWT_SECRET rotado hoy? → re-login forzado
                         → ¿401 en todas? → verificar reloj del servidor (NTP)
                         → ¿403 solo usuarios concretos? → permisos RBAC /is_active
```

13.2 Rotación de secretos y credenciales

Secreto	Dónde vive	Frecuencia	Procedimiento
JWT_SECRET	backend/.env (producción: gestor de secretos)	Cada 90 días y ante sospecha	openssl rand -base64 48 → actualizar .env → restart → forzar re-login (todas las sesiones invalidan)
SMTP_PASS (Gmail)	backend/.env	Anual o ante sospecha	Revocar contraseña de aplicación en myaccount.google.com → crear nueva → actualizar .env → restart → forgot-password de prueba
Clave postgres (pos_app)	PostgreSQL role pos_app	Anual	ALTER ROLE pos_app PASSWORD 'nueva'; actualizar .env; restart backend
Admin semilla (admin@dpiero.com)	Tabla users (bcrypt)	Inmediata go-live	UPDATE users SET user_password = bcrypt(...); forzar cambio en primer ingreso (P0)
PIN operativo semilla (1234)	Tabla users (pin)	Primer ingreso	Política operativa: cada empleado cambia su PIN por terminal en su primer turno

Tabla 40 — Calendario y procedimiento de rotación de secretos

13.3 Checklist de post-incidente

- Bitácora del incidente completada: síntoma, hora inicio/fin, causa raíz, acciones, responsables.
- Comunicación al cliente (propietario) con resumen ejecutivo y medidas tomadas.
- Lección aprendida y actualización de este manual si el procedimiento cambió.
- Verificación de que el respaldo sigue íntegro (última copia restaurable).
- Si hubo exposición de datos: revocar credenciales, notificar y documentar (Ley 1581 de 2012 — habeas data, Colombia).

14. Recorrido Guiado del Código — Dos Flujos Completos

14.1 Flujo A — Crear una comanda: de la UI a PostgreSQL y de regreso

Este recorrido sigue el código verdadero en el commit `f58187b` y es la lectura recomendada para cualquier desarrollador nuevo en el equipo. Archivo por archivo, línea por línea:

14.1.1 La vista (frontend/src/features/orders/pages/CreateOrderPage.jsx)

- **Apertura de la pantalla.** El router (router.jsx) monta `<CreateOrder />` bajo DashboardLayout para las rutas `/dashboard/CreateOrder`, `/CreateOrder`, `/CreateOrder` (y la 404 redirige aquí).
- **Carga del catálogo.** `fetchProducts()` → `getProducts({ activeOnly: true })` (features/products/services/productServices.js) → fetch a `GET http://localhost:4000/api/products?active=true` con `Authorization: Bearer <token>` si existe sesión. Los 12 productos se renderizan en tarjetas con imagen, código PAR-xx, precio COP y contadores.
- **Disponibilidad de mesas.** `fetchTables()` → `GET /api/orders/tables/availability` → grid M1..M10 con estados Libre/Ocupada; solo mesas libres seleccionables; el encabezado muestra '6 libres' y la capacidad de la mesa elegida.
- **Contadores por producto.** `updateQty(prodId, delta)` ajusta el mapa `quantities` (producto→cantidad) y `notes` por producto; total = Σ precio × qty.
- **Envío.** `handleRegister()` valida mesa + ítems > 0 → construye DTO `{ tableNumber, waiterId, items, observations, status: 'Abierta', orderNumber: undefined }` → `orderServices.createOrder(dto)` → `POST /api/orders`.

14.1.2 El servicio HTTP (frontend/src/features/orders/services/orderServices.js)

```
// extracto real — orderServices.js
const API_URL = import.meta.env.VITE_API_URL || "http://localhost:4000/api";
const ORDERS_URL = `${API_URL}/orders`;

export async function createOrder(orderData) {
  const res = await fetch(ORDERS_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      ...(getToken() ? { Authorization: `Bearer ${getToken()}` } : {}),
    },
    body: JSON.stringify(orderData),
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.error || "Error creando la comanda");
  }
  return res.json();
}
```

14.1.3 El controlador (backend/src/features/orders/order.controller.js → create)

```
async create(req, res) {
  try {
    const order = await orderService.createOrder(req.body);
    res.status(201).json({ message: "Comanda registrada en PostgreSQL exitosamente", order });
  } catch (error) {
    console.error("ERROR BACKEND (CREATE ORDER):", error);
    res.status(400).json({ error: error.message });
  }
}
```

14.1.4 El servicio (order.service.js → createOrder)

```
async createOrder(data) {
  if (!data.tableNumber && !data.table)
    throw new Error("El número de mesa es obligatorio para registrar la comanda");
  return await orderRepository.create(data);
}
```

14.1.5 El repositorio (order.repository.js → create) — el corazón

```
const client = await pool.connect();
try {
  await client.query("BEGIN");                                    // ① abre la transacción

  let orderNumber = data.orderNumber || data.id;
  if (!orderNumber?.startsWith("ORD-")) {
    const seq = await client.query(
      "SELECT COALESCE(MAX(id), 0) + 1 AS next_id FROM orders;");
    orderNumber = `ORD-${String(seq.rows[0].next_id).padStart(3, "0")}`;
  }                                                               // ② ORD-008, ORD-009…

  let waiterId = data.waiterId || null;
  if (!waiterId && data.waiter) {                                  // ③ resolver mesero por nombre
    const userRes = await client.query(
      "SELECT id FROM users WHERE LOWER(user_name) LIKE LOWER($1) LIMIT 1",
      [`%${data.waiter}%`]);
    if (userRes.rows[0]) waiterId = userRes.rows[0].id;
  }

  const tableNumber = data.tableNumber || data.table || "Mesa 1";
  const status = data.status || data.orderStatus || "Abierta";
  const items = Array.isArray(data.items) ? data.items : [];
  let totalAmount = Number(data.total || data.totalAmount || 0);
  if (totalAmount === 0 && items.length > 0) {
    totalAmount = items.reduce(
      (sum, it) => sum + Number(it.price || 0) * Number(it.qty || it.quantity || 1), 0);
  }                                                               // ④ total calculado en servidor

  const orderResult = await client.query(
    `INSERT INTO orders (order_number, table_number, waiter_id, status, observations,
                         total_amount, created_at, updated_at)
     VALUES ($1, $2, $3, $4, $5, $6, NOW(), NOW()) RETURNING *;`,
    [orderNumber, tableNumber, waiterId, status, data.observations || "", totalAmount]);
  const createdOrder = orderResult.rows[0];                        // ⑤ INSERT comanda
```

```
  for (const item of items) {                                          // ⑥ por cada línea
    let productId = item.productId || item.product_id;
    let unitPrice = Number(item.price || 0);
    if (!productId && item.name) {                                     // ⑦ resolver producto por nombre
      const prodRes = await client.query(
        "SELECT id, price FROM products WHERE LOWER(name)=LOWER($1) OR code=$1 LIMIT 1",
        [item.name]);
      if (prodRes.rows[0]) { productId = prodRes.rows[0].id; if (!unitPrice) unitPrice =
Number(prodRes.rows[0].price); }
    }
    if (productId) {
      const qty = Number(item.qty || item.quantity || 1);
      const subtotal = Number(item.subtotal || unitPrice * qty);
      await client.query(
        `INSERT INTO order_items (order_id, product_id, quantity, unit_price, subtotal, notes)
         VALUES ($1, $2, $3, $4, $5, $6);`,
        [createdOrder.id, productId, qty, unitPrice, subtotal, item.notes || ""]);  // ⑧ INSERT línea
      await client.query(
        "UPDATE products SET stock = GREATEST(0, stock - $1) WHERE id = $2;",
        [qty, productId]);                                                          // ⑨ descontar
stock
    }
  }
  await client.query("COMMIT");                                        // ⑩ confirmar todo
  return await this.findById(createdOrder.order_number);              // ⑪ devolver hidratada (json_agg)
} catch (error) {
  await client.query("ROLLBACK");                                      // ⑫ nada o todo
  throw error;
} finally {
  client.release();                                                    // ⑬ siempre liberar la conexión
}
```

14.1.6 El camino de regreso a la UI

- El controller 201 responde `{ message, order }`; el fetch del frontend resuelve; `handleRegister()` (CreateOrderPage) muestra `showSuccessAlert('Comanda registrada', 'Total: $…')` y navega a `/orderList`.
- En `/orderList`, `GET /api/orders` devuelve la comanda; el usuario la ve como `ORD-008 · Mesa 2 · Abierta · $112.000`.
- `getTablesAvailability()` marca Mesa 2 ocupada; `/CreateOrder` la excluye de mesas libres.
- `products.stock` PAR-01 baja de 18 a 16 — visible en `/productList`.

14.2 Flujo B — Verificación de permiso desde la UI hasta la BD

Segundo recorrido esencial: ¿cómo sabe el sistema si puede mostrar el botón 'Crear usuario'?

14.2.1 ProtectedRoute (frontend/src/shared/components/auth/ProtectedRoute.jsx)

```
// En /dashboard, la ruta index está protegida:
```

```
<ProtectedRoute requiredPermission="create_user">
  <UserRegisterForm />
</ProtectedRoute>

export default function ProtectedRoute({ children, requiredPermission }) {
  const token = localStorage.getItem("auth_token") || sessionStorage.getItem("token");
  const [status, setStatus] = useState(requiredPermission ? "loading" : "allowed");
  useEffect(() => {
    hasPermission(requiredPermission)
      .then(res => setStatus(res.granted ? "allowed" : "denied"))
      .catch(() => setStatus("denied"));
  }, [token, requiredPermission]);
  if (!token) return <Navigate to="/Auth" replace />;
  if (status === "loading") return null;
  if (status === "denied")  return <Navigate to="/" replace />;
  return children;
}
```

14.2.2 hasPermission() → GET /api/access/check/create_user

```
// features/access/services/accessService.js (extracto)
export async function hasPermission(permissionCode) {
  const res = await fetch(`${API_URL}/access/check/${permissionCode}`, {
    headers: { Authorization: `Bearer ${getToken()}` },
  });
  if (!res.ok) throw new Error("No se pudo verificar el permiso");
  return res.json();          // { userId, permission, granted }
}
```

14.2.3 El backend decide (access.service.js)

```
async hasPermission(userId, permissionCode) {
  if (permissionCode === "admin") return await accessRepository.isAdmin(userId);
  if (await accessRepository.isSuperUser(userId)) return true;   // RN-02 pase maestro
  if (await accessRepository.isAdmin(userId)) return true;       // admin por tipo o grupo
  const permissions = await accessRepository.getUserPermissions(userId);
  return permissions.includes(permissionCode);                    // RN-03 UNION directo ∪ grupo
}
```

14.2.4 La consulta definitiva (access.repository.js — getUserPermissions)

```
SELECT DISTINCT p.permission_codename
FROM permissions p INNER JOIN user_permissions up ON up.permission_id = p.permission_id
WHERE up.user_id = $1
UNION
SELECT DISTINCT p.permission_codename
FROM permissions p INNER JOIN group_permissions gp ON gp.permission_id = p.permission_id
INNER JOIN user_groups ug ON ug.group_id = gp.group_id
WHERE ug.user_id = $1;
```

Lectura 3 minutos: este patrón (UI consulta check → backend decide con UNION) significa que un permiso revocado a un grupo tiene efecto inmediato para sus miembros — no hay que re-loguear ni recargar la pantalla, solo navegar.

14.3 Diagrama de flujo técnico — recuperación de contraseña

Flujo exacto implementado en `forgot-password` → `reset-password` (auth.service.js + password_reset_codes):

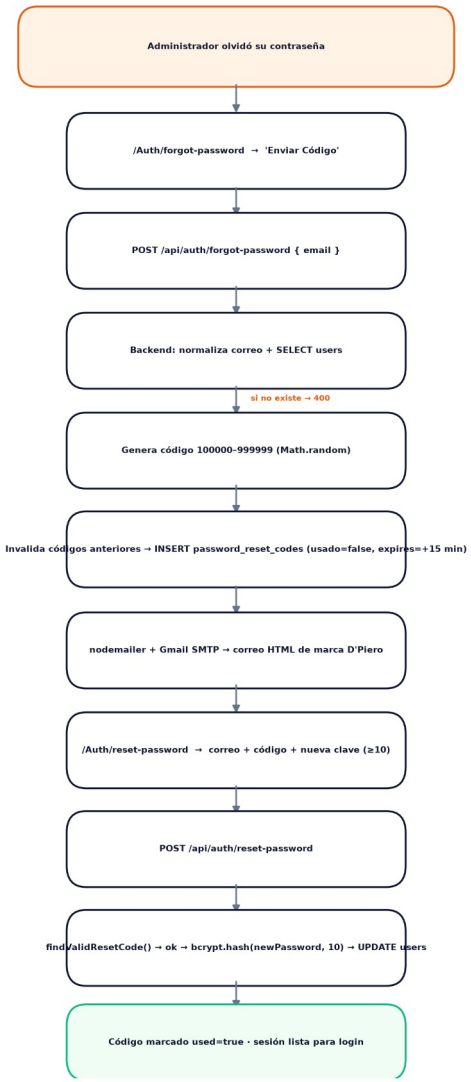


Figura 6 — Flujo completo de recuperación de contraseña con código de 6 dígitos y vigencia de 15 minutos

Nota de seguridad: el código se genera con `Math.random()` (pseudoaleatorio no criptográfico). En endurecimiento se sustituye por `crypto.randomInt(100000, 999999)` (prioridad P2, sección 9.6.3). El código se envía por Gmail SMTP y viaja en texto plano dentro del correo — por ello su vigencia es corta y de un solo uso.

Anexo A — .env.example (plantilla sin secretos)

```
# ===== D'Piero POS — backend/.env.example (NO COMPLETAR CON VALORES REALES) =====
# Puerto HTTP del backend
PORT=4000
# URL del cliente (para correos y CORS en el futuro)
FRONTEND_URL=http://localhost:5173

# PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_USER=pos_app
DB_PASSWORD=CAMBIAR_POR_SECRETO_FUERTE
DB_NAME=pos_project

# JWT — firma HS256
JWT_SECRET=CAMBIAR_POR_SALIDA_DE_openssl_rand_-base64_48
JWT_EXPIRES=1h

# Gmail SMTP (opcional — sin esto, los códigos se imprimen en consola)
SMTP_USER=
SMTP_PASS=
```

Anexo B — Script de migraciones (migrate.js) — explicación

`migrate.js` (109 líneas) implementa un migrador minimalista y seguro:

- **Descubrimiento.** Lee `backend/database/migrations/*.sql`, filtra los `.sql`, ordena alfabéticamente → el orden es el nombre (0001…0027).
- **Idempotencia.** Tabla `_migrations(name UNIQUE)` registra lo aplicado; se salta lo ya ejecutado.
- **Atomicidad.** Cada migración corre en `BEGIN…COMMIT`; ante error `ROLLBACK` y aborta con exit(1).
- **Verificación.** Tras migrar, sugiere `SELECT count(*) FROM _migrations` → debe ser 27.
- **Uso.** `node src/scripts/migrate.js` desde `backend/`. Requiere `.env` con credenciales válidas.

Anexo C — Referencia rápida de comandos

Tarea	Comando
Arrancar backend (dev)	cd backend && npm run dev
Arrancar frontend (dev)	cd frontend && npm run dev
Aplicar migraciones	cd backend && node src/scripts/migrate.js
Build de producción frontend	cd frontend && npm run build
Lint frontend	cd frontend && npm run lint
Respaldo completo BD	pg_dump -h localhost -U postgres -d pos_project -F c -f respaldo.dump
Restaurar respaldo	pg_restore -h localhost -U postgres -d pos_project_restore respaldo.dump
Contar tablas	psql -h localhost -U postgres -d pos_project -c \dt
Ver migraciones aplicadas	psql -h localhost -U postgres -d pos_project -c 'SELECT * FROM _migrations ORDER BY name;'
Reiniciar backend (prod)	sudo systemctl restart pos-backend
Logs backend (prod)	journalctl -u pos-backend -f
Probar conexión BD	pg_isready -h localhost -p 5432
Humo API	curl http://localhost:4000/api/products

Tabla 41 — Referencia rápida de operación

Anexo D — Evidencias de ejecución (capturas)

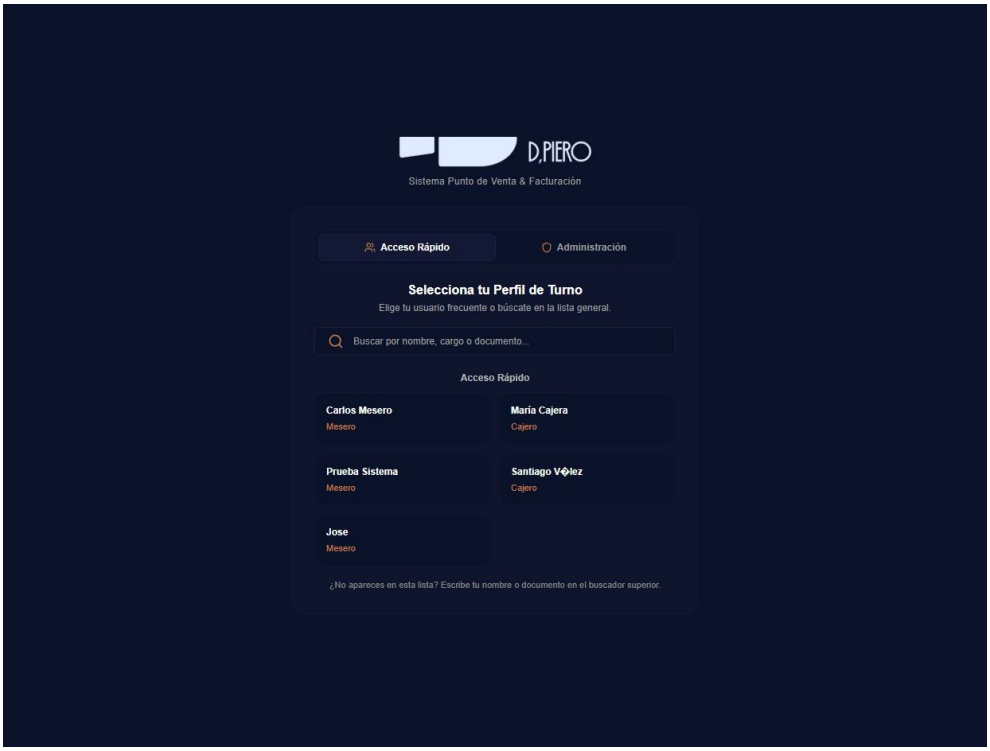


Figura 7 — Login de acceso rápido — perfiles visibles

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

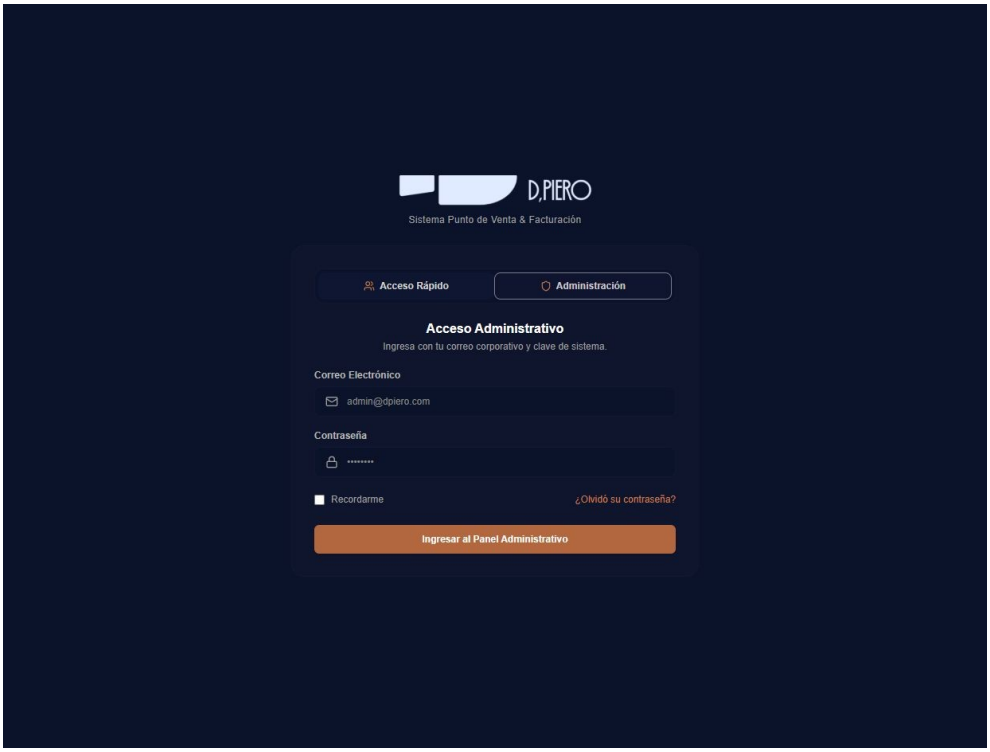


Figura 8 — Login administrativo con bloqueo a 3 intentos

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

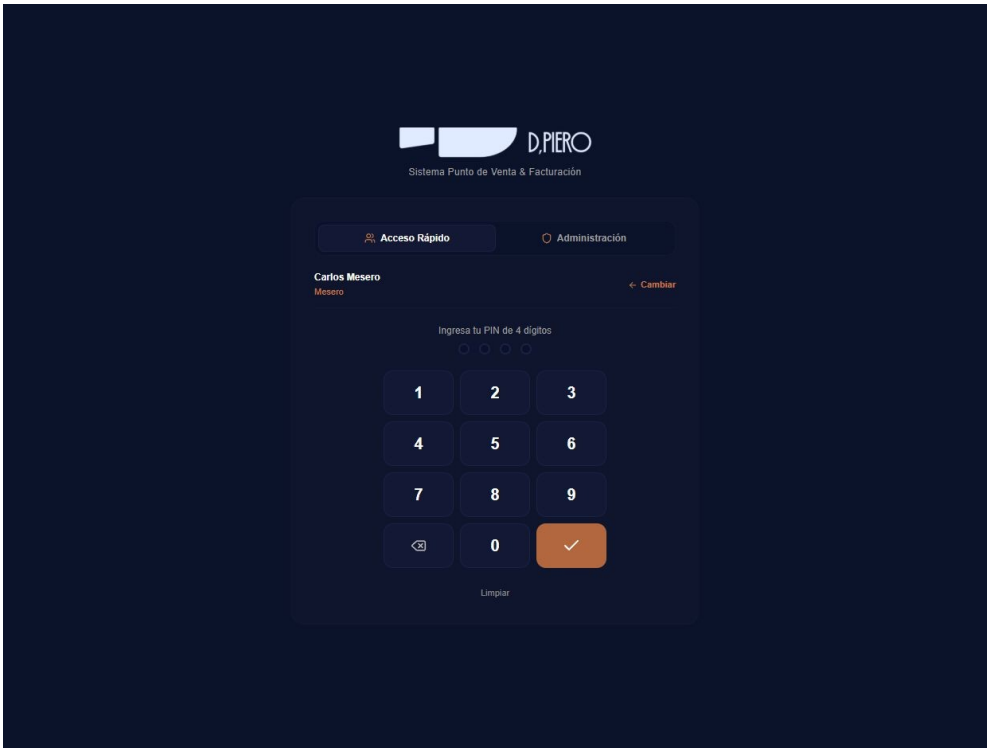


Figura 9 — Teclado PIN de 4 dígitos

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

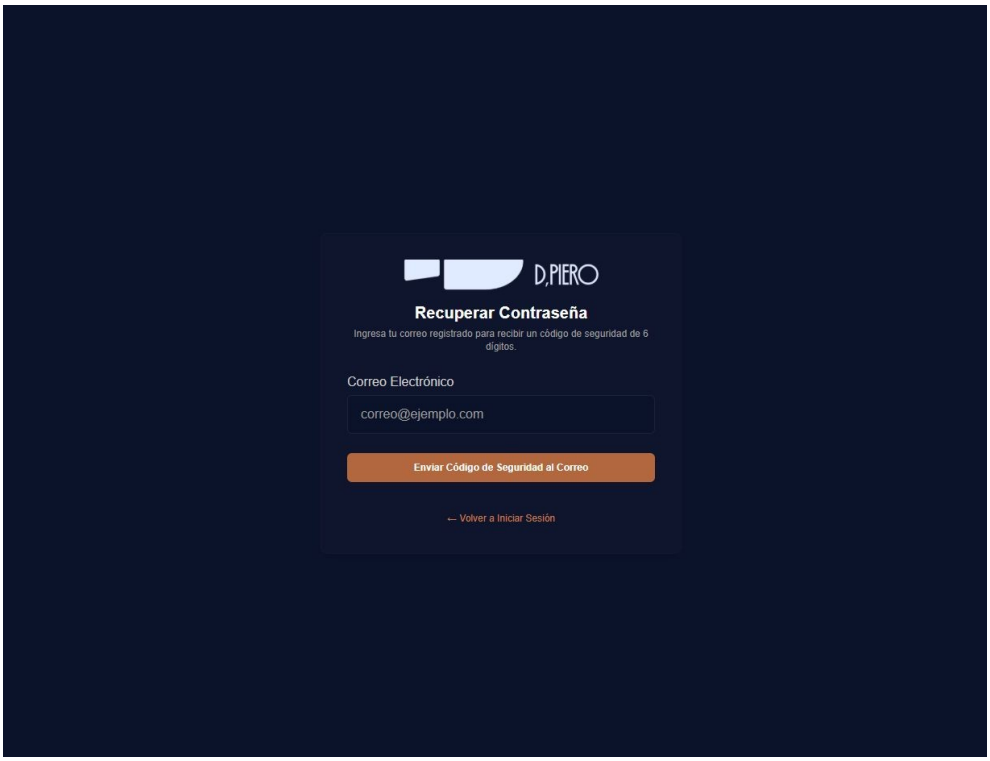


Figura 10 — Solicitud de código de recuperación

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

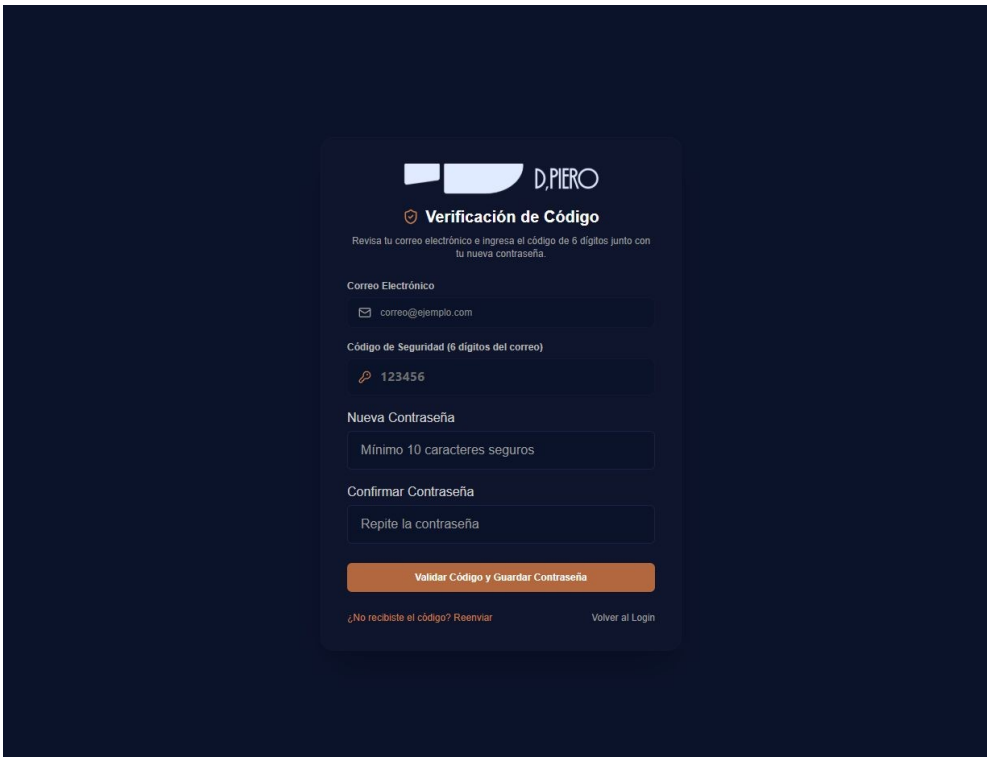


Figura 11 — Verificación de código y nueva contraseña

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

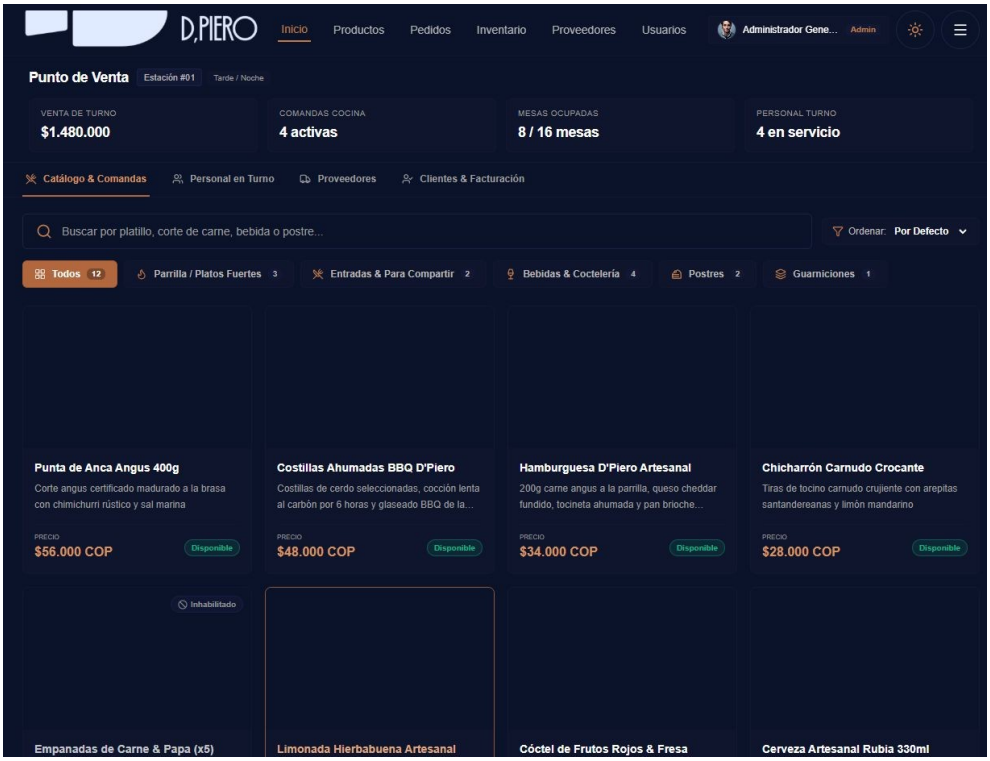


Figura 12 — POS — Catálogo & Comandas con datos reales

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

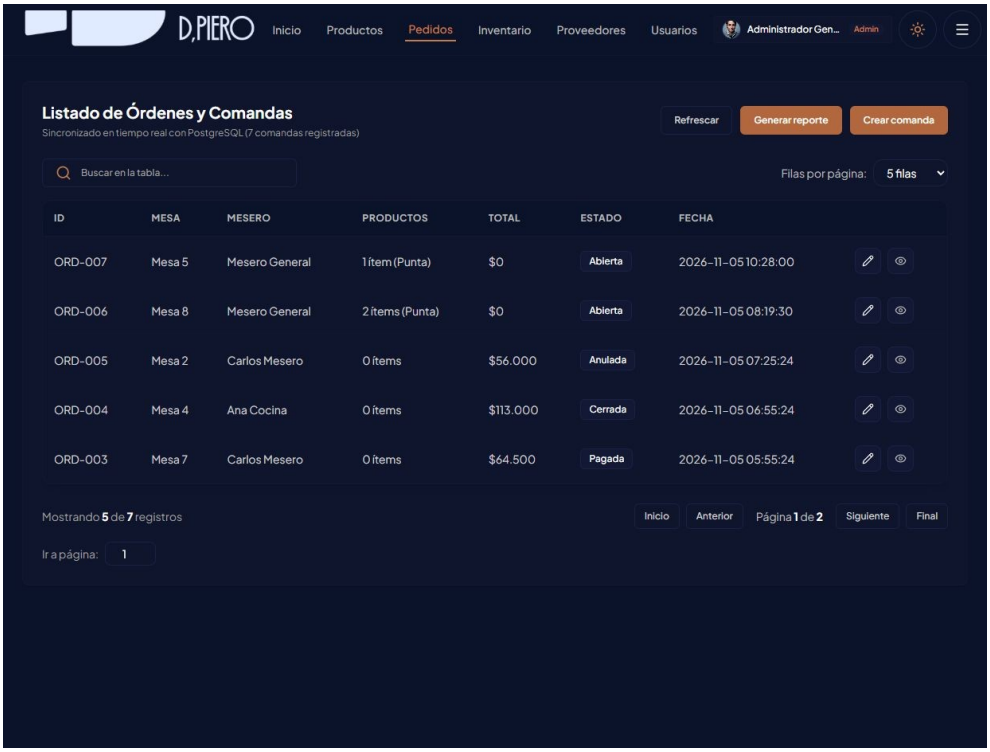


Figura 13 — Listado de comandas ORD-001..007

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

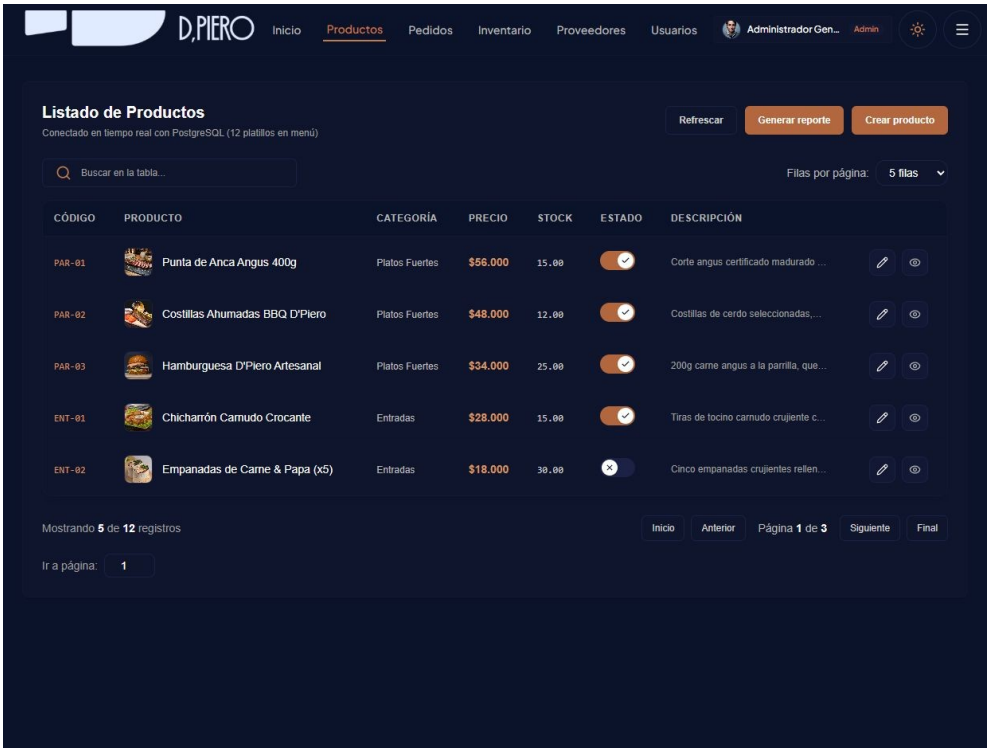


Figura 14 — Listado de productos con stock real

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

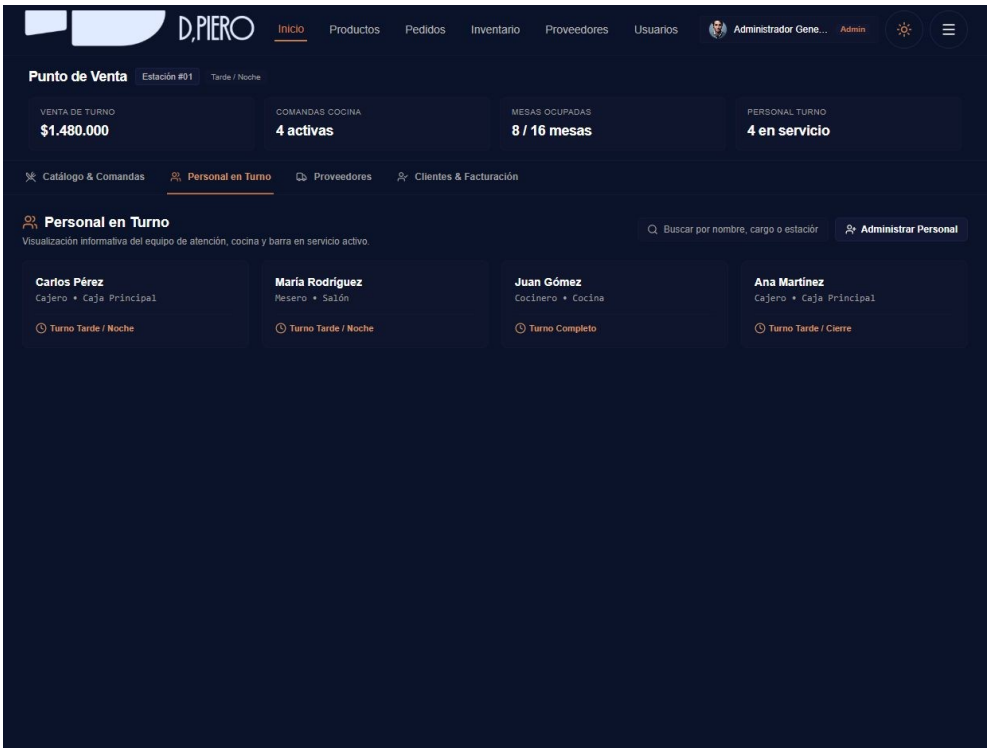


Figura 15 — POS — pestaña Personal en Turno con roles y horarios

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

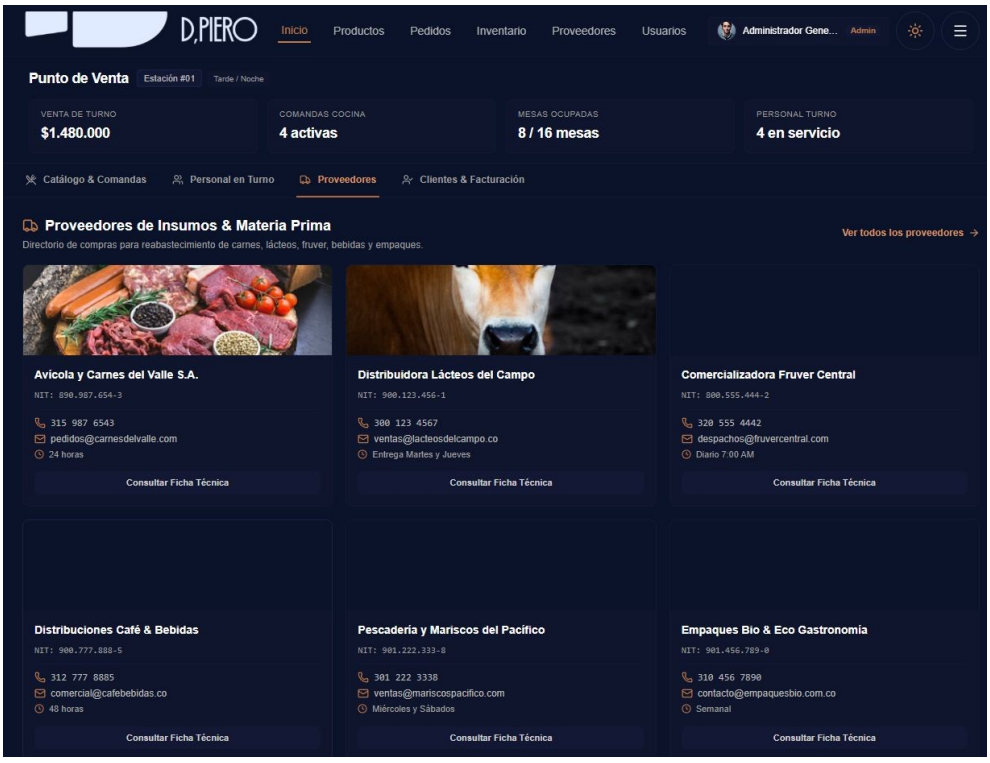


Figura 16 — POS — pestaña Proveedores con datos de contacto reales

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

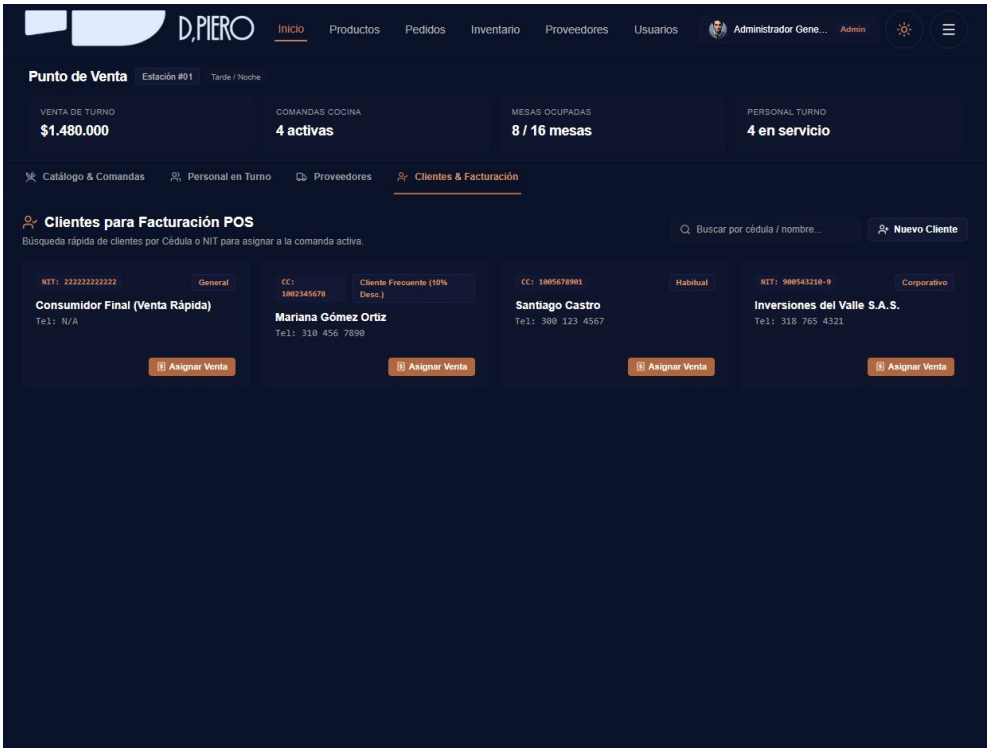


Figura 17 — POS — pestaña Clientes & Facturación (demo)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

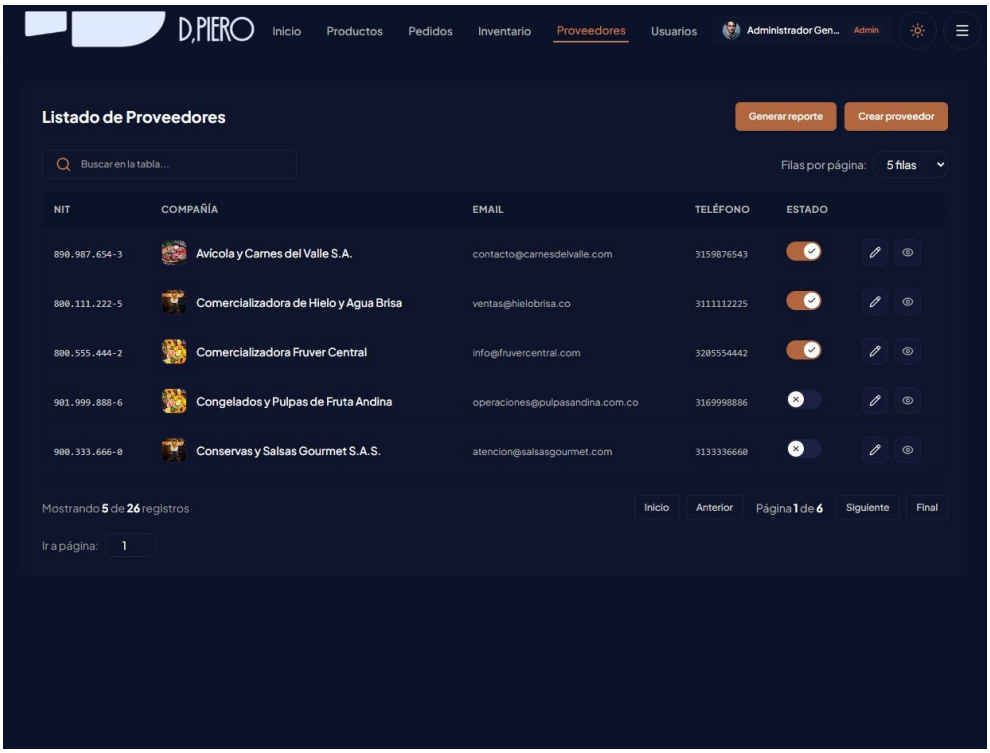


Figura 18 — Listado de proveedores (26 registros)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

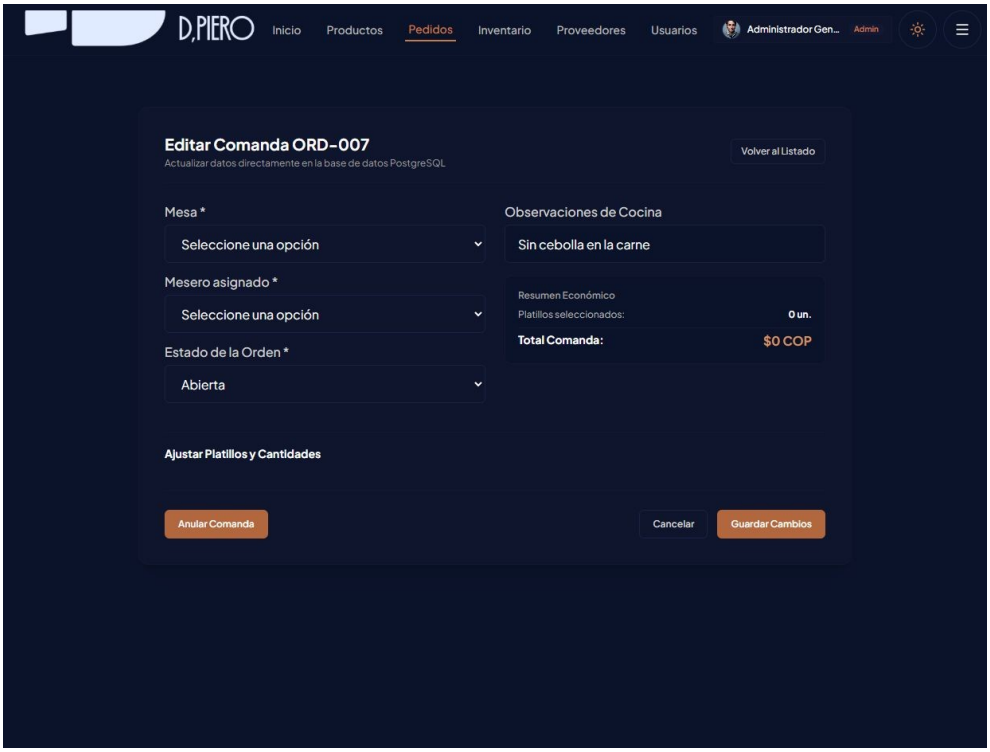


Figura 19 — Edición de comanda (mesa, mesero, estado, ítems)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

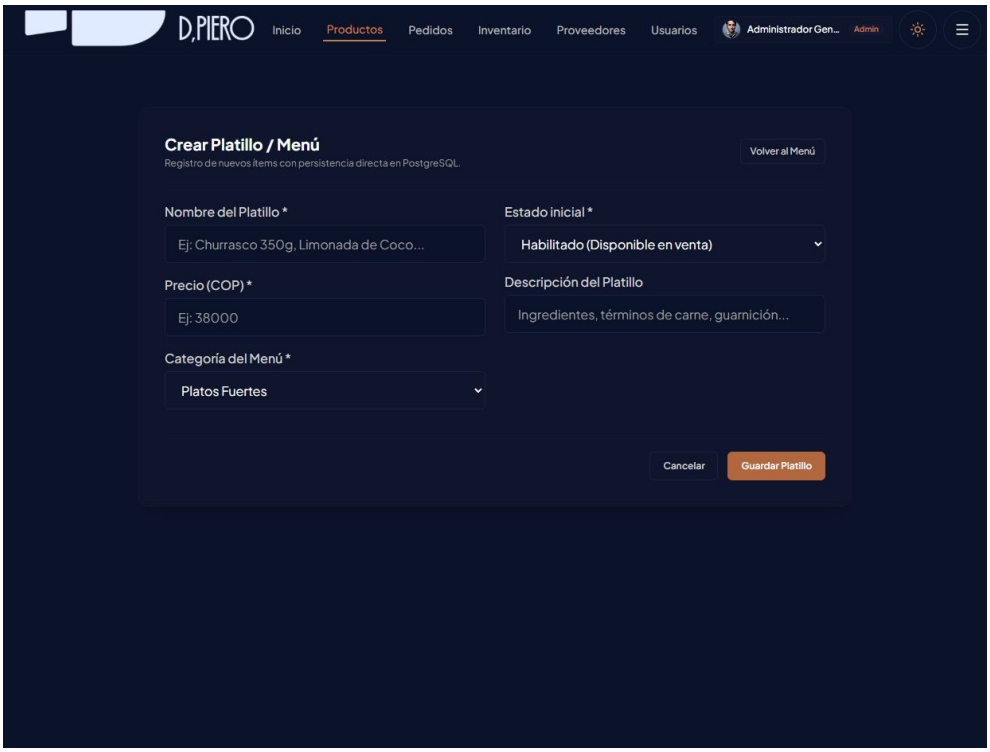


Figura 20 — Formulario de creación de platillo

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

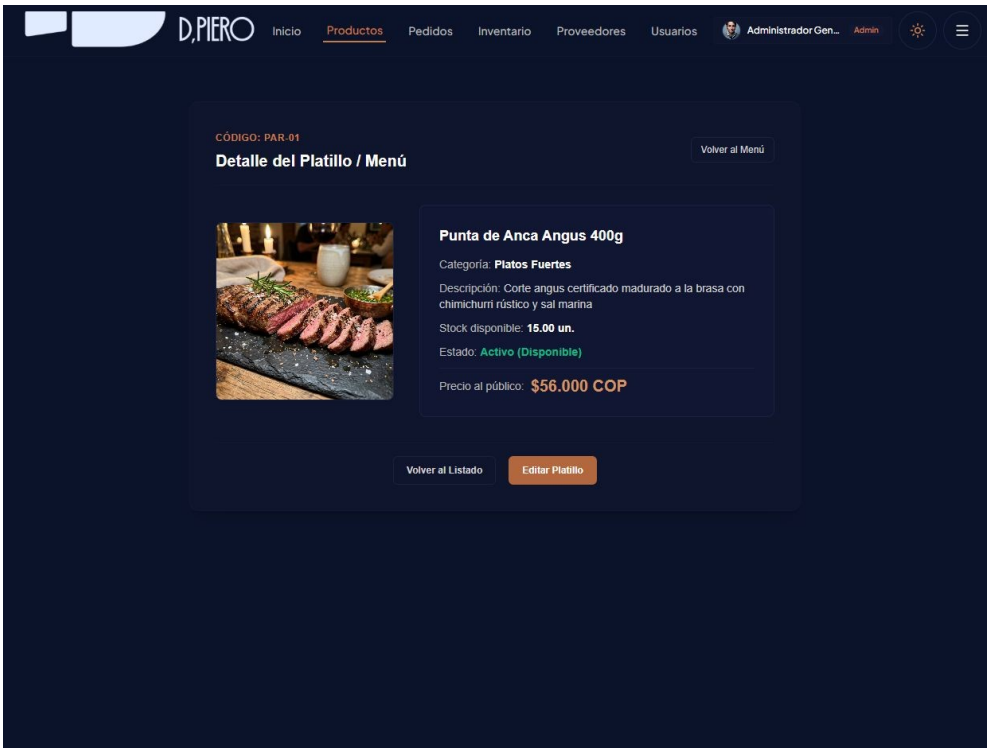


Figura 21 — Ficha de solo lectura del platillo

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

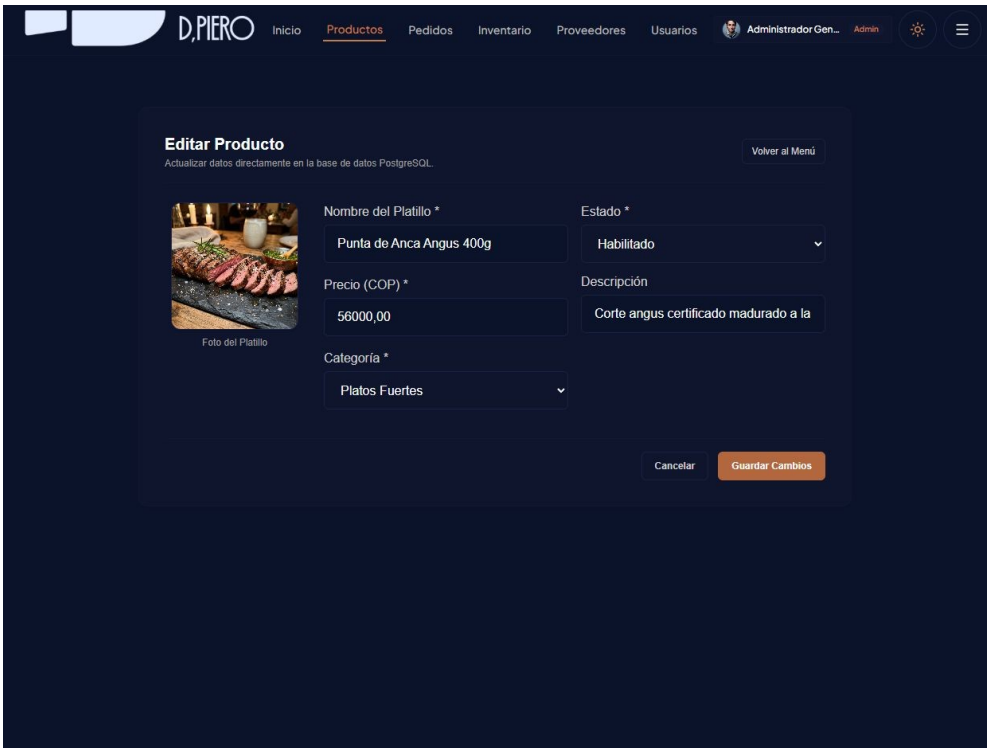


Figura 22 — Edición de producto (estado Habilitado)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

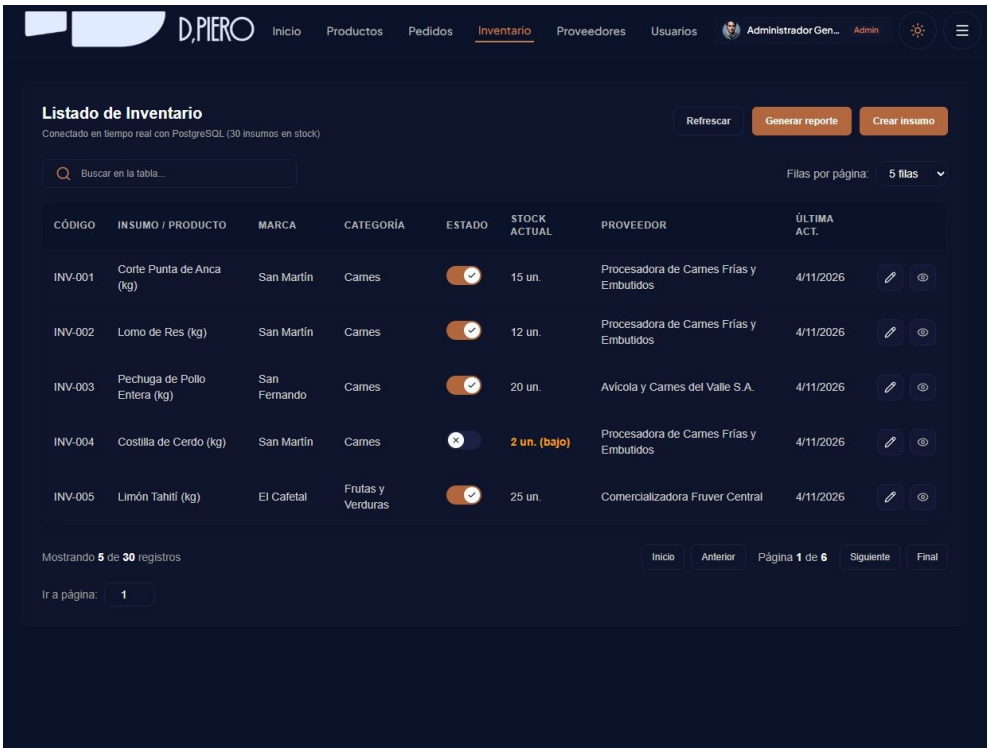


Figura 23 — Inventario con alerta de mínimo (INV-004)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

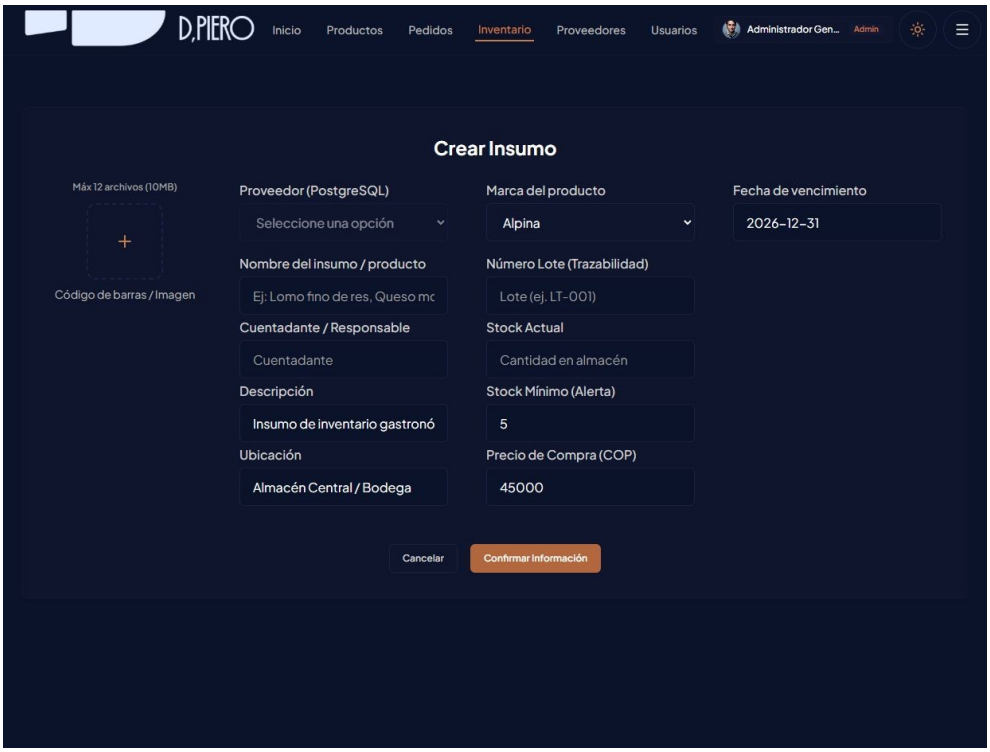


Figura 24 — Alta de insumo con lote, vencimiento y proveedor

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

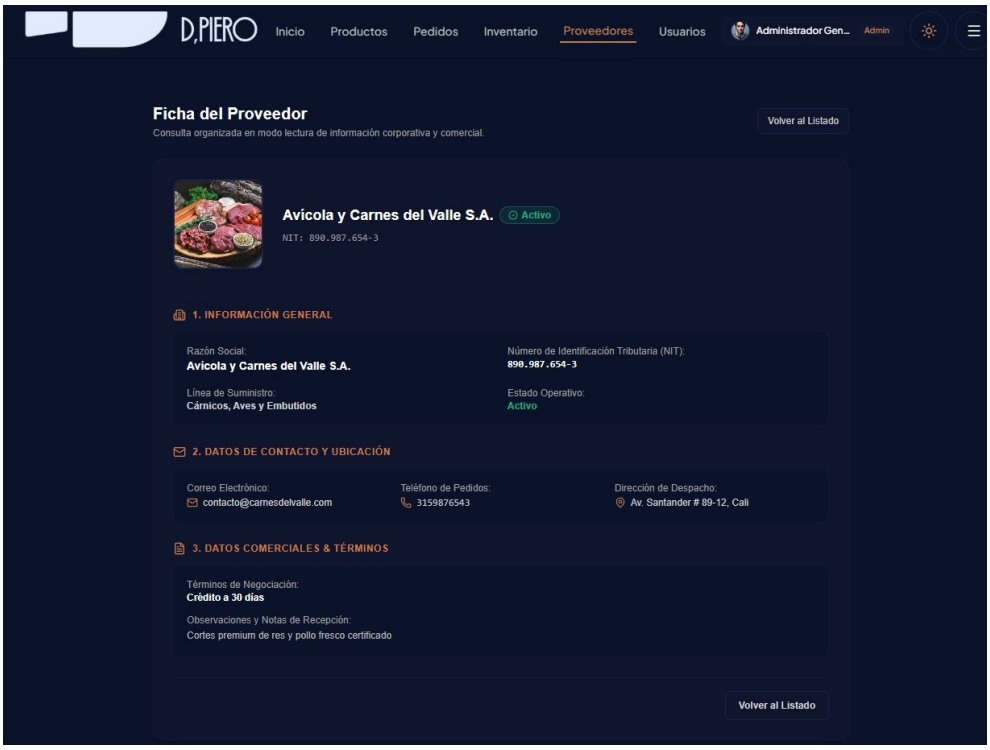


Figura 25 — Ficha completa del proveedor (Avícola y Carnes del Valle)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

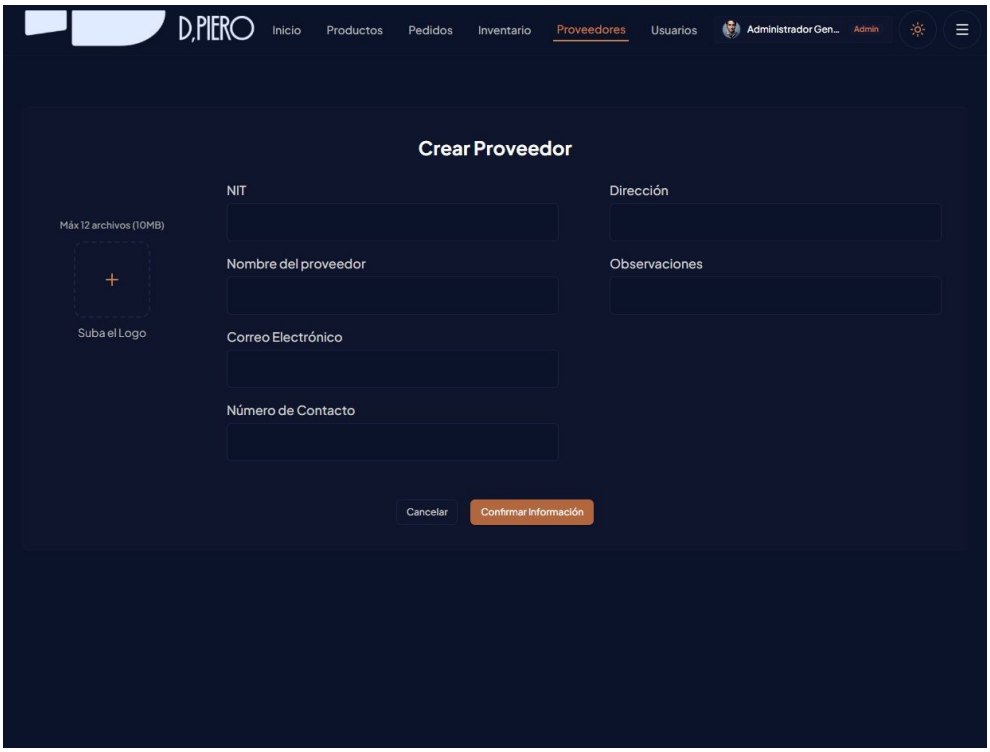


Figura 26 — Alta de proveedor con NIT único

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

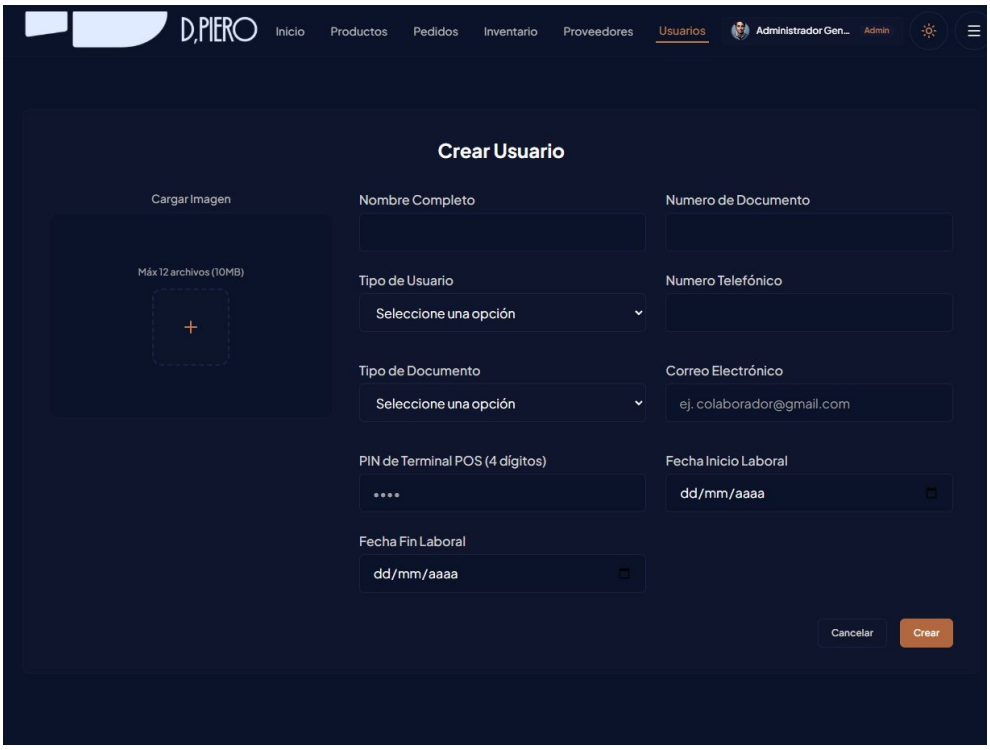


Figura 27 — Alta de usuario operativo con PIN autogenerado

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

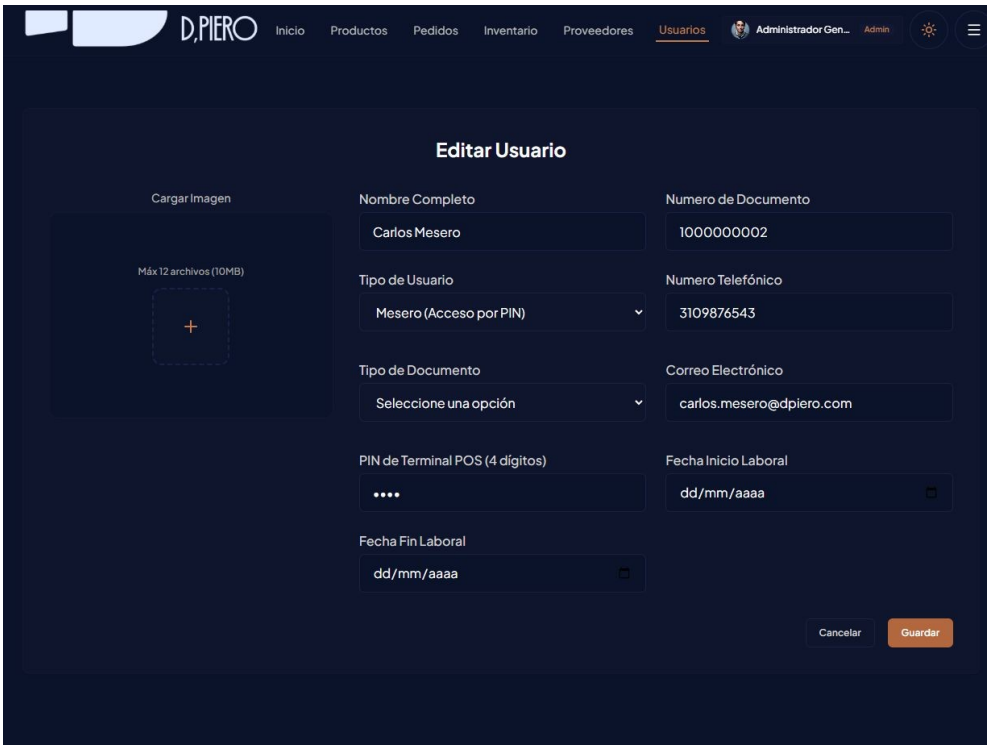


Figura 28 — Edición de usuario (Carlos Mesero)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

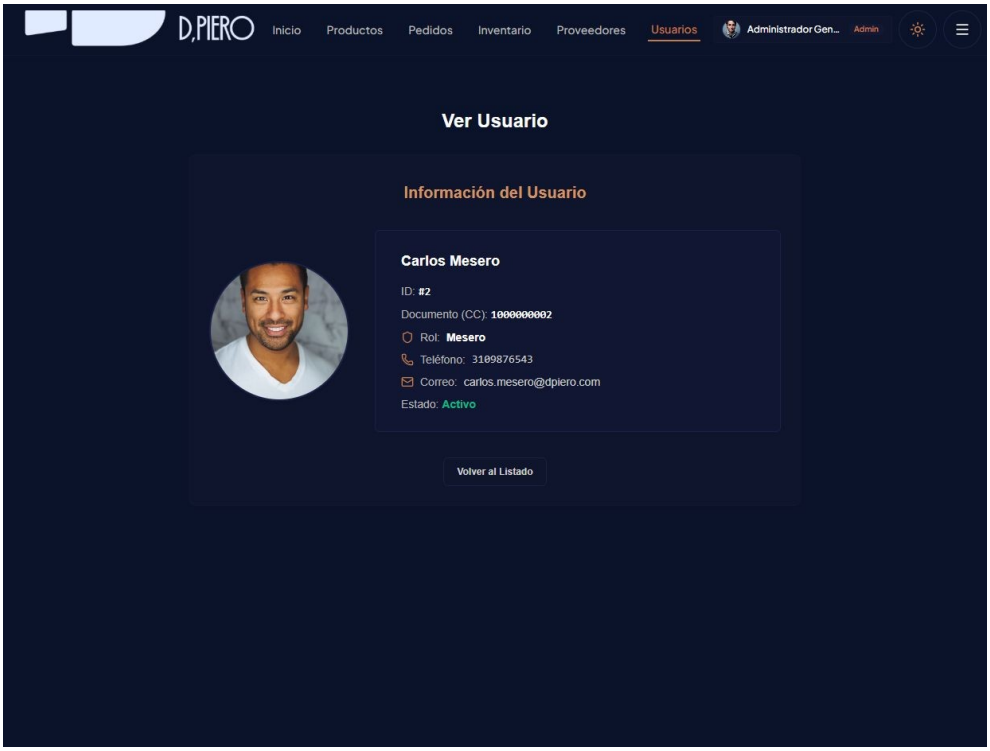


Figura 29 — Ficha del usuario (información completa)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

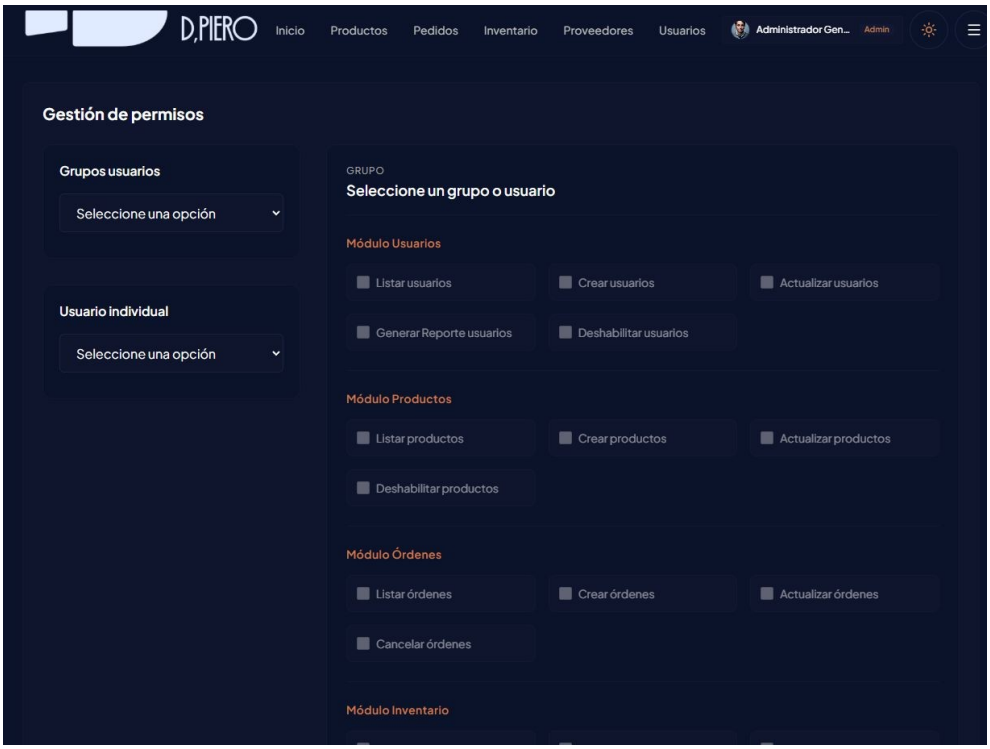


Figura 30 — Gestión de permisos RBAC (grupos y usuario individual)

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

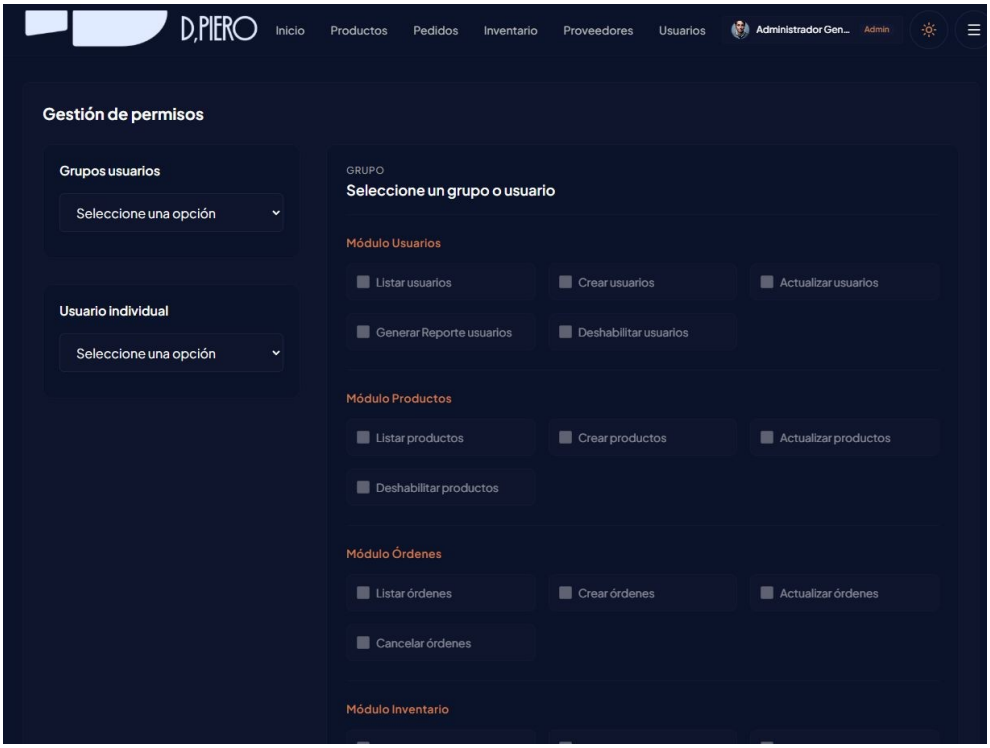


Figura 31 — Matriz de permisos por módulo en /dashboard/access

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

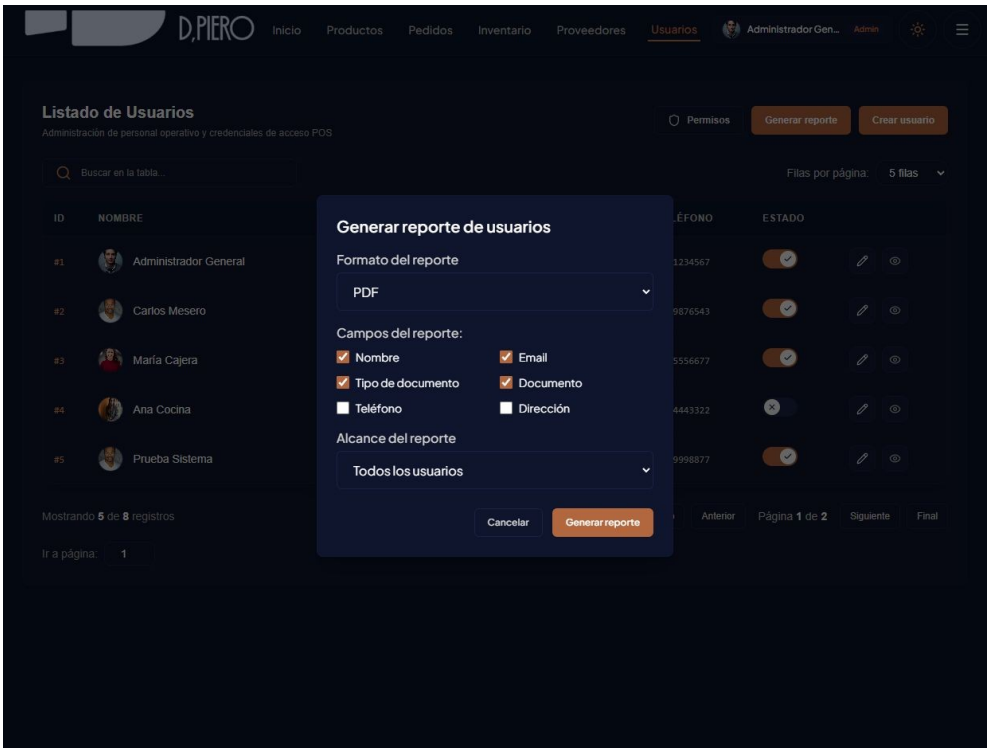


Figura 32 — Modal de generación de reportes PDF/Excel

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

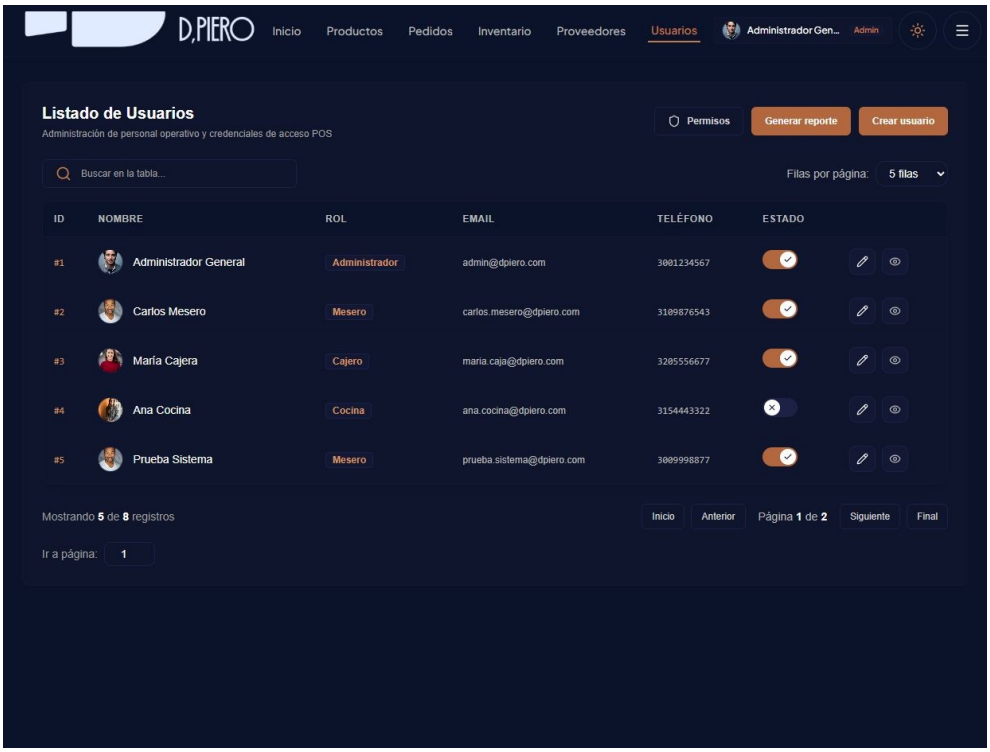


Figura 33 — Listado de usuarios con acciones por fila

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

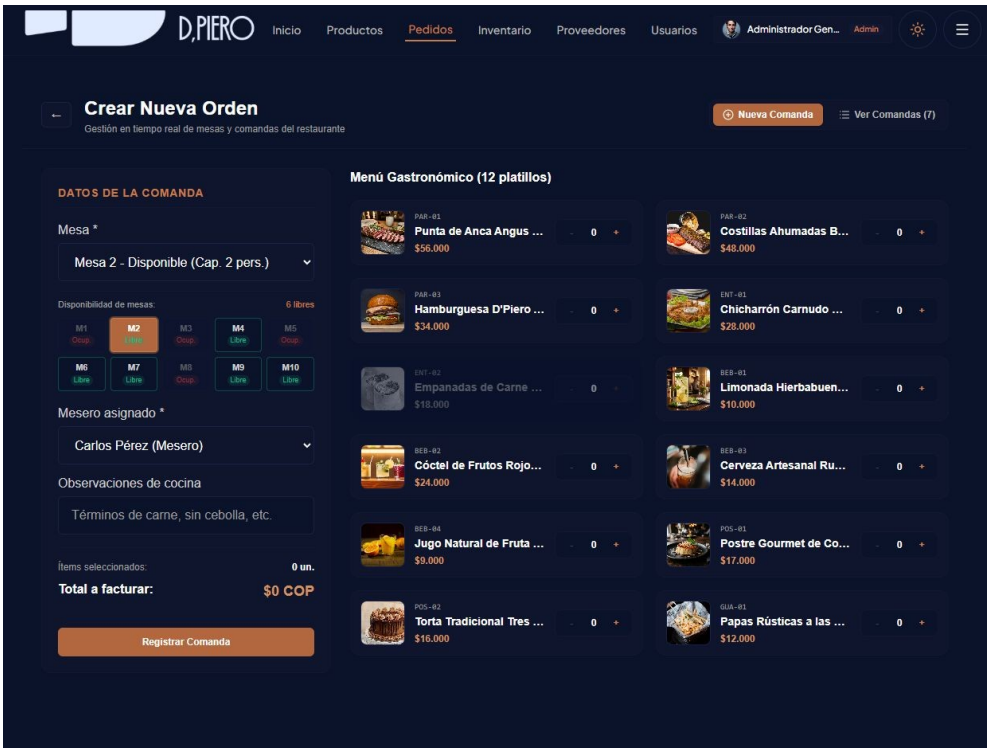


Figura 34 — Creación de comanda con disponibilidad de mesas

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

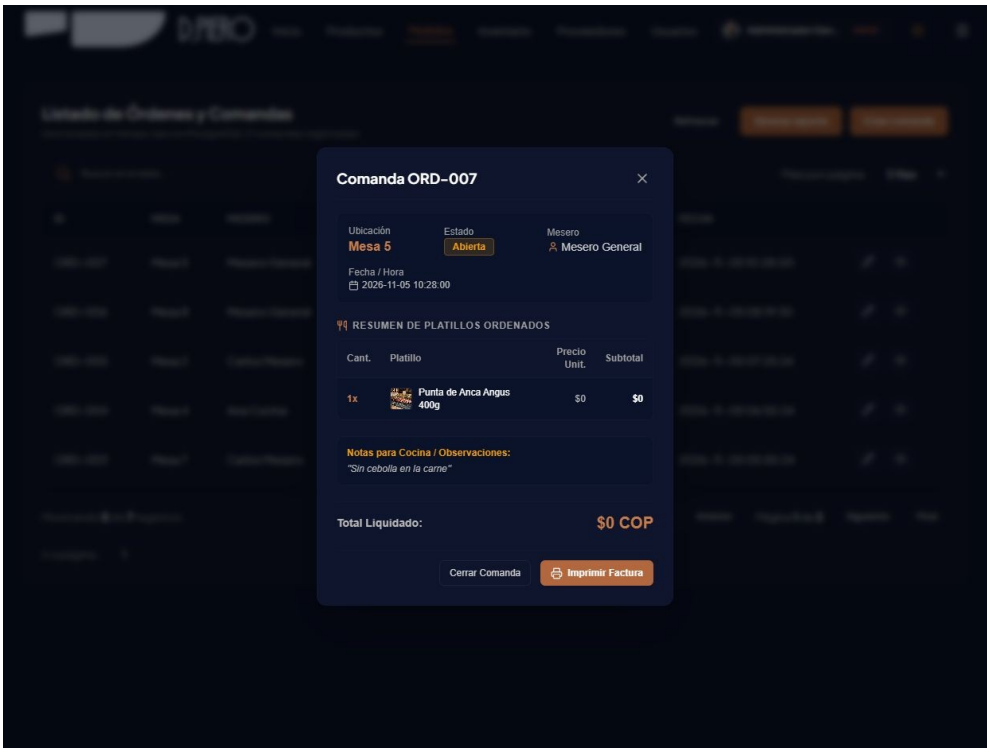


Figura 35 — Detalle de comanda en modal con ítems y total

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

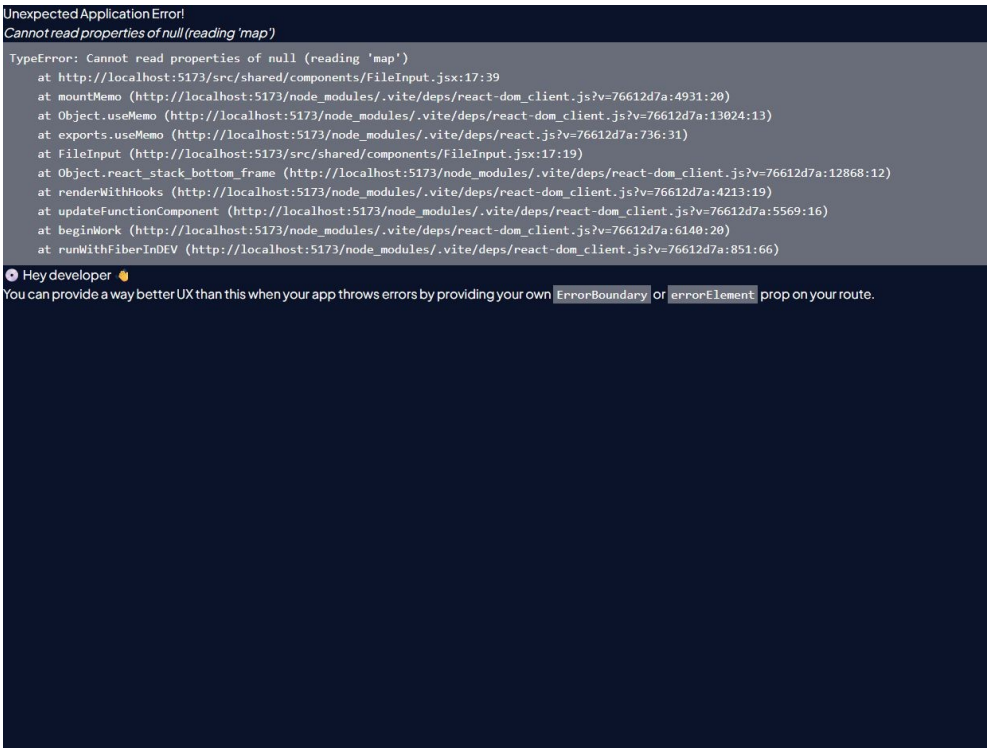


Figura 36 — BUG-01 reproducido: crash de /editsupplier

Sistema en localhost (frontend :5173, backend :4000, PostgreSQL pos_project). 2026-11-08.

Anexo E — Glosario técnico

Término	Definición
SPA	Single Page Application — una sola página HTML; el cliente resuelve el routing.
REST	Estilo de API sobre HTTP con recursos, verbos y códigos de estado semánticos.
JWT	JSON Web Token — token firmado (HS256) transportado en Authorization: Bearer.
HS256	HMAC-SHA256 — algoritmo simétrico de firma de JWT con JWT_SECRET.
bcrypt	Algoritmo de hashing de contraseñas con factor de costo (10).
RBAC	Role-Based Access Control — control de acceso por roles (grupos) y permisos.
content_type	Catálogo de módulos del dominio al que pertenece cada permiso.
CQRS (parcial)	Lecturas desnormalizadas con json_agg separadas conceptualmente de escrituras transaccionales.
Pool	Conjunto de conexiones reutilizables (pg Pool, max 10, idle 30 s).
Idempotencia	Migraciones seguras de re-ejecutar (IF NOT EXISTS, ON CONFLICT).
GREATEST	Función que devuelve el máximo de sus argumentos — evita stock negativo.
TO_CHAR	Formateo de fecha/numero a texto en PostgreSQL (createdAt legible).

Término	Definición
json_agg / json_build_object	Agregan filas hijas a un solo campo JSON en una consulta.
UPSERT	INSERT … ON CONFLICT DO UPDATE/NOTHING — usado en seeds.
RPO / RTO	Pérdida de datos admisible / tiempo de recuperación objetivo.
COALESCE	Primer no-NULL — usado para fallbacks ('Mesero General', 'Proveedor no asignado').
ADR	Architecture Decision Record — registro de decisión arquitectónica.
FOUC	Flash of Unstyled Content — parpadeo antes del tema; evitado con script en index.html.
SEV-n	Severidad del incidente (1 = caído, 4 = menor).
HACCP	Análisis de peligros y puntos críticos de control (inocuidad) — contexto de lotes/vencimientos.

Tabla 42 — Glosario técnico del sistema

Anexo G — Extractos SQL comentados de las migraciones clave

A continuación se anotan, línea por línea, fragmentos reales de las migraciones 0001, 0010, 0026 y 0027. Leerlas en este orden equivale a entender la evolución del esquema.

G.1 0001_create_users_table.sql — el origen

```
CREATE TABLE IF NOT EXISTS users (        -- IF NOT EXISTS: permite re-ejecutar migraciones sin error
    id SERIAL PRIMARY KEY,                 -- SERIAL = entero autoincremental (secuencia users_id_seq)
    user_name VARCHAR(100) NOT NULL,       -- nombre completo, obligatorio
    user_email VARCHAR(150) UNIQUE NOT NULL, -- login único global
    user_phone VARCHAR(20),                -- opcional al crear
    user_document_types VARCHAR(20),       -- CC / CE / TI / PAS / NIT
    user_document_number VARCHAR(50) NOT NULL, -- documento, obligatorio
    user_password TEXT NOT NULL,           -- (luego relajado a NULL por operativos con PIN)
    user_avatar_url TEXT,                  -- foto del empleado (assets/images/worker-*.jpg)
    is_staff BOOLEAN DEFAULT FALSE,        -- true para todo el personal de salón/caja/cocina
    is_active BOOLEAN DEFAULT TRUE,        -- bloqueo lógico por admin
    is_superuser BOOLEAN DEFAULT FALSE,    -- pase universal RBAC (RN-02)
    created_at TIMESTAMP DEFAULT NOW(),    -- alta del registro
    updated_at TIMESTAMP DEFAULT NOW()     -- última modificación (sin triggers: se actualiza en
UPDATE)
);
```

Lectura: el diseño original preveía contraseñas para *todos*. La realidad operativa (terminales compartidas con PIN) obligó al 0010 a relajar `user_password` y añadir `pin`, `user_type`, `corporate_email`, `start_date` y `end_date`.

G.2 0010_update_users_table.sql — adaptación al negocio real

```
ALTER TABLE users ADD COLUMN IF NOT EXISTS pin VARCHAR(10);          -- PIN de 4 dígitos del POS
ALTER TABLE users ADD COLUMN IF NOT EXISTS user_type VARCHAR(50);   -- admin|mesero|cajero|cocina
(legado)
ALTER TABLE users ADD COLUMN IF NOT EXISTS corporate_email VARCHAR(150); -- correo corporativo
alternativo
ALTER TABLE users ADD COLUMN IF NOT EXISTS start_date DATE;          -- inicio laboral
ALTER TABLE users ADD COLUMN IF NOT EXISTS end_date DATE;            -- fin laboral (informativa)
ALTER TABLE users ALTER COLUMN user_password DROP NOT NULL;          -- operativos sin contraseña
```

G.3 0026_seed_groups_default_permissions_and_user_groups.sql — la matriz RBAC

```
-- Administrador: TODOS los permisos (barrido cruzado groups×permissions)
INSERT INTO group_permissions (group_id, permission_id)
SELECT g.group_id, p.permission_id
FROM groups g CROSS JOIN permissions p
WHERE g.group_name = 'Administrador'
ON CONFLICT DO NOTHING;

-- Mesero: listar/crear/actualizar órdenes y listar productos
INSERT INTO group_permissions (group_id, permission_id)
SELECT g.group_id, p.permission_id
FROM groups g INNER JOIN permissions p
    ON p.permission_codename IN ('list_product','list_order','create_order','update_order')
WHERE g.group_name = 'Mesero'
ON CONFLICT DO NOTHING;

-- Vincular usuarios existentes a su grupo por user_type (incluye legacy '')
INSERT INTO user_groups (user_id, group_id)
SELECT u.id, g.group_id FROM users u
INNER JOIN groups g ON g.group_name = 'Mesero'
WHERE u.is_superuser = FALSE AND LOWER(COALESCE(u.user_type, '')) IN ('mesero','salon','')
ON CONFLICT DO NOTHING;
```

Lectura: esta migración es el corazón del RBAC: reparte permisos por grupo y enlaza usuarios.

Reejecutable sin duplicados gracias a `ON CONFLICT DO NOTHING`.

G.4 0027_seed_initial_pos_data.sql — datos reales del restaurante

```
-- 12 platillos con código de negocio, precio COP y stock real
INSERT INTO products (code, name, price, description, category_id, image_url, status, is_active,
stock) VALUES
    ('PAR-01', 'Punta de Anca Angus 400g', 56000.00,
     'Corte angus certificado madurado a la brasa con chimichurri rústico y sal marina',
     (SELECT id FROM categories WHERE name = 'platos fuertes'),
     '/assets/images/punta-de-anca-hd.jpg', 'Disponible', TRUE, 18),
    …
    ('BEB-01', 'Limonada Hierbabuena Artesanal', 10000.00, …, 'BEB-01', 'Disponible', TRUE, 45),

-- 26 proveedores con NIT real (solo 8 activos al sembrar)
INSERT INTO suppliers (nit, company_name, email, phone, address, observation, is_active) VALUES
    ('890.987.654-3', 'Avícola y Carnes del Valle S.A.', 'contacto@carnesdelvalle.com', …),
```

```
-- 30 insumos con lote y vencimiento (trazabilidad sanitaria HACCP)
INSERT INTO inventory (code, item_name, category, brand, current_stock, min_stock, max_stock, …)
VALUES ('INV-001', 'Corte Punta de Anca (kg)', 'Carnes', 'San Martín', 15.00, 5.00, 40.00, 'kg', …),

-- 10 mesas y 5 comandas demo (ORD-001…ORD-005) con ítems
```

Anexo H — Bitácora de cambios de la aplicación (2026-05-11 → 2026-11-08)

Resumen de los hitos funcionales en orden cronológico. Los hashes corresponden al repositorio `pos_project`; la bitácora completa está en el Manual Administrativo, §6.10.

Fecha	Hash(es)	Entrega principal	Tipo
2026-05-11	3466644	Base del proyecto (README y estructura inicial)	feat
2026-05-27	5751495	React Router instalado; inputs y botones con variables de diseño	feat
2026-06-10	f656a75	Unificación de colores con tokens primarios	feat
2026-06-15	e8f10bb	Formulario de registro corregido; validaciones y componentes	fix/feat
2026-06-15	b2a4729	Página Crear Orden con formulario y tarjetas de platillos	feat
2026-06-15	7821238	Gestión de Usuarios con Select/Checkbox/Button	feat
2026-06-15	7ed0e95	Login con 'Recordarme' y Register completo	feat
2026-06-15	cae6bfd	Crear Usuario bajo DashboardLayout (/CreateUser)	feat
2026-06-15	6d999e9	Vista de crear producto (Henao420)	feat
2026-06-16	4dcbfb1	Vista de crear proveedores completada (Henao420)	feat
2026-06-17	856d594	Vista de crear producto finalizada	feat
2026-06-17	3ba2a3f	Merge PR #9 — crear proveedor	merge
2026-06-17	8d850c8	userSchema (zod) aplicado en todos los formularios	chore
2026-06-17	328b580	UI de errores zod bajo cada campo	feat
2026-06-25	e9586d9	Tablas de gestión implementadas en el proyecto	feat
2026-06-28	05ce73d	Validación limpia y acciones en lista de usuarios	fix
2026-07-24	5d15cc7 / 78c0566	Listar proveedores y ajustes (Henao420)	feat
2026-08-04	0e0f4ac	Vistas ver/editar usuarios y FileInput (Henao420)	feat
2026-08-06	054c0bc	Merge PR #10 — listar proveedores	merge
2026-08-07	8bf54e0	Schemas y vistas ver/editar proveedores y productos (Henao420)	feat
2026-08-08	683e58d	Merge PR #11 — ver proveedores	merge
2026-08-08	22eddd8	Vista EditOrder; responsividad y contraste	feat
2026-08-26	c78219b	Estructura frontend/backend separada	refactor
2026-09-09	c4fcfa2	Rediseño POS; corrección formularios y componentes	feat
2026-09-28	68add21	Tema oscuro→claro; roles y auth redefinidos; más productos	refactor
2026-10-03	ea18570	Modularizar vistas y flujo de auth	refactor

Fecha	Hash(es)	Entrega principal	Tipo
2026-10-05	e0238ff	Recuperación con código de 6 dígitos	fix
2026-11-03	a8e5b44	RBAC completo; estructura por dominios	feat
2026-11-05	b0617b4	Lista de proveedores conectada a PostgreSQL (Henao420)	feat
2026-11-05	f58187b	Merge PR #13 — permisos	merge
2026-11-08	—	Verificación integral + 3 manuales (Administrativo, Técnico, Usuarios)	docs

Tabla 43 — Bitácora resumida de entregas del sistema

15. Preguntas Frecuentes Técnicas (FAQ de desarrollo)

¿Por qué el sistema funciona sin login en algunas pantallas?

Decisión deliberada para demos y la operación de caja: /, /productList, /orderList, /inventoryList, /supplierList y los endpoints GET no exigen token hoy. El contrato de seguridad real lo define ProtectedRoute (UI) para rutas administrativas. Ver §8.1.10 y §9.6 para el plan de endurecimiento — esta relajación es conocida y controlada por el equipo.

¿Dónde cambio el precio de un platillo sin tocar código?

En la UI: /productList → lápiz → Precio (COP) → Guardar Cambios. En BD: UPDATE products SET price = 38000 WHERE code = 'PAR-03'; (nunca por la UI en producción sin aviso al cliente).

¿Cómo agrego una categoría nueva al menú?

INSERT INTO categories (name, description, is_active) VALUES ('cocteleria-premium', 'Coctelería de autor', TRUE); — luego asigne category_id por nombre al crear/editar productos.

¿Cómo agrego un permiso nuevo (p. ej. export_inventory)?

1) Nueva migración NNNN con INSERT INTO permissions … con content_type_id del módulo; 2) asignar a grupos en otra migración o por UI; 3) usarlo en ProtectedRoute y/o requirePermission; 4) actualizar el seed de referencia y la matriz §9.1 de este documento.

¿Por qué la comanda sale con total 0 en /orderList?

Las semillas ORD-001..ORD-005 fueron insertadas con total manual inconsistente (DATA-01). Las comandas nuevas siempre calculan bien. Si una comanda muestra $0, verifique que tenga order_items con unit_price > 0.

¿Cómo apunto el frontend a otro backend sin recompilar?

Defina VITE_API_URL antes de npm run dev o npm run build: VITE_API_URL=https://pos.dpiero.com/api npm run build. El fallback es http://localhost:4000/api.

¿Qué pasa si borro un producto con historial de ventas?

No puede: order_items.product_id tiene ON DELETE RESTRICT. Desactívelo (toggle) — desaparece del POS y conserva su historial.

¿Por qué la consola muestra 'Cada child in a list should have a unique key prop'?

BUG-04 — el componente Select no pasa key={option.value}. Es un warning de React, no un error; pendiente de corrección menor.

¿Cómo restauro solo la tabla orders sin tocar usuarios?

pg_restore -h localhost -U postgres -d pos_project_restore --table=orders respaldo.dump (restaura orders + order_items con --table para ambas; respete el orden de dependencias).

¿Cómo sé qué migraciones están aplicadas en una BD ajena?

SELECT name FROM _migrations ORDER BY name; — cualquier hueco en la secuencia (p. ej., salto de 0023 a 0025) indica una migración no aplicada o aplicada a mano.

¿Puedo usar el sistema sin backend (modo demo)?

Parcialmente: la UI carga y /HomePage muestra catálogo de respaldo (posMockData), pero las operaciones de escritura fallan con 'Error inesperado de conexión'. El sistema real exige backend + BD.

¿Cómo cambio el logo institucional?

Reemplace frontend/src/assets/images/logo-dpiero-orange.png (navbar y login) y logo-dpiero.png (variantes); mantenga 920×150 px o ajuste BrandLogo (h-10/h-12).

¿Qué hago con BUG-01 (editar proveedor crashea)?

No editar desde UI hasta aplicar el parche `value ?? []` en FileInput o en EditSupplierPage. Línea: `<FileInput value={formSupliers.userImage ?? []} onChange={…} />`.

¿Cómo verifico que el RBAC está bien configurado para un usuario nuevo?

1) Crear usuario con tipo 'mesero' → verificar user_groups (debe estar en Mesero); 2) GET /api/groups con token admin → ver permisos; 3) login con el PIN del usuario nuevo y comprobar qué pestañas ve.

¿El sistema soporta múltiples sucursales?

Una sola hoy. Multi-sucursal exige: tabla branches, FK branch_id en products/orders/inventory/tables, POS por branch_id, y JWT con branch scoping. Es el RFC-01 del roadmap.

Anexo I — Consultas SQL operativas para el administrador de BD

I.1 Auditoría diaria del negocio

```
-- Ventas por estado hoy
SELECT status, count(*) AS comandas, sum(total_amount) AS total_cop
FROM orders WHERE created_at::date = CURRENT_DATE GROUP BY status ORDER BY 1;

-- Platillos con stock crítico
SELECT code, name, stock FROM products WHERE stock < 5 ORDER BY stock ASC;

-- Insumos por vencer en 7 días
SELECT code, item_name, expiration_date, supplier_id
FROM inventory WHERE expiration_date IS NOT NULL
  AND expiration_date <= CURRENT_DATE + INTERVAL '7 days' ORDER BY expiration_date;

-- Usuarios por rol efectivo
SELECT COALESCE(u.user_type, 'sin_tipo') AS tipo, count(*) FROM users u GROUP BY 1 ORDER BY 1;

-- Comandas abiertas sin cerrar hace +2 horas (posible abandono)
SELECT order_number, table_number, created_at, now() - created_at AS edad
FROM orders WHERE lower(status) IN ('abierta','en preparación','pendiente')
  AND now() - created_at > INTERVAL '2 hours';

-- Códigos de recuperación activos a punto de vencer
SELECT email, code, expires_at FROM password_reset_codes
WHERE used = false AND expires_at > now() ORDER BY expires_at;
```

I.2 Integridad del RBAC

```
-- Usuarios sin grupo asignado (corregir con syncUserGroup)
SELECT u.id, u.user_name FROM users u
LEFT JOIN user_groups ug ON ug.user_id = u.id
```

```
WHERE ug.user_id IS NULL;

-- Permisos directos que no están en el grupo del usuario (overrides)
SELECT u.user_name, p.permission_codename
FROM user_permissions up
JOIN users u ON u.id = up.user_id
JOIN permissions p ON p.permission_id = up.permission_id
LEFT JOIN user_groups ug ON ug.user_id = u.id
LEFT JOIN group_permissions gp ON gp.group_id = ug.group_id AND gp.permission_id = up.permission_id
WHERE gp.permission_id IS NULL ORDER BY u.user_name;
```

I.3 Mantenimiento higiénico

```
-- Archivar comandas cerradas de hace > 6 meses a tabla de archivo (propuesta)
-- CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);
-- INSERT INTO orders_archive SELECT * FROM orders WHERE status='Cerrada' AND updated_at < now() -
INTERVAL '6 months';

-- Vacuum y estadísticas tras borrado masivo
VACUUM (ANALYZE) orders, order_items, products;

-- Reindexar tras muchas migraciones
REINDEX INDEX idx_reset_codes_email_code;
```

Anexo J — Checklist de liberación (release)

#	Verificación	Responsable
1	Rama feature fusionada a dev; PR con descripción y evidencias (capturas)	Dev
2	git pull en la máquina destino (o servidor); npm ci en backend y frontend	Dev
3	pg_dump de respaldo previo (obligatorio)	Dev/DBA
4	node src/scripts/migrate.js — sin errores; _migrations al día (27+)	DBA
5	npm run build en frontend — sin errores; dist/ generado	Dev
6	Smoke API completo (Anexo C, humo): 9 grupos 200/201	Dev
7	Recorrido funcional: login admin + mesero; crear comanda de prueba; anularla	Dev + cliente
8	Verificación de .env productivo (JWT_SECRET rotado, sin valores de desarrollo)	DevOps
9	Reinicio systemd y chequeo de logs (journalctl -u pos-backend)	DevOps
10	Comunicación al restaurante: versión liberada y cambios visibles (nota de release)	Aprendiz líder
11	Tag Git v1.x.y y push del tag	Dev
12	Actualización de MANUALES versionados (control de versiones v1.x)	Aprendiz líder

Tabla 44 — Checklist de liberación obligatoria por versión

Anexo F — Índice de referencias al código (archivo → sección del manual)

Para localizar cada pieza del código dentro de este documento o para navegar del código al manual y viceversa.

Archivo	Contenido principal	Sección
backend/src/app.js	CORS, routers, middleware de errores	§3, §4, §10
backend/src/server.js	Arranque, guards de proceso	§4, §10, §13
backend/src/config/db.js	Pool pg (max 10)	§3, §6, §10
backend/src/middlewares/auth.middleware.js	Verificación JWT	§8.4, §9
backend/src/middlewares/permission.middleware.js	requirePermission(code)	§8.4, §9
backend/src/scripts/migrate.js	Migrador transaccional con _migrations	§6.6, Anexo B
backend/database/migrations/0001–0010	Esquema base (users…payments)	§6.2
backend/database/migrations/0011–0012	Seeds usuarios y reset codes	§6.2
backend/database/migrations/0013–0020	RBAC: permissions, groups, puentes, content_type	§6.2
backend/database/migrations/0021–0027	Seeds RBAC, catálogo, mesas, comandas demo	§6.2
backend/src/features/auth/*	Login PIN/correo, recuperación 6 dígitos	§7.1, §9
backend/src/features/users/*	CRUD usuarios + sync RBAC + status	§7.2, §14
backend/src/features/products/*	CRUD catálogo, categorías	§7.3
backend/src/features/inventory/*	CRUD insumos con lote/vencimiento	§7.4
backend/src/features/suppliers/*	CRUD proveedores (NIT único)	§7.5
backend/src/features/orders/*	Comandas transaccionales ORD-###	§6.5.4, §6.5.5, §7.6, §14.1
backend/src/features/access/*	check de permiso en vivo	§7.7, §14.2
backend/src/features/groups/*	Lectura/escritura permisos de grupo	§7.8
backend/src/features/permissions/*	Catálogo 21 permisos	§7.9
frontend/src/app/router.jsx	31 rutas + ProtectedRoute	§4, §5
frontend/src/app/App.jsx	RouterProvider	§4
frontend/src/main.jsx	createRoot + StrictMode	§4
frontend/src/features/home/page/HomePage.jsx	POS principal con tabs por rol	§7.10
frontend/src/features/orders/*	POS comandas: crear, editar, listar, modal	§7.6, §14.1
frontend/src/features/products/*	CRUD productos + imágenes HD	§7.3
frontend/src/features/inventory/*	CRUD inventario + alerta mínimo	§7.4
frontend/src/features/suppliers/*	CRUD proveedores + ficha	§7.5
frontend/src/features/users/*	CRUD usuarios + reportes PDF/Excel	§7.2
frontend/src/features/access/*	Matriz RBAC (grupos/usuario)	§7.7, §9.1
frontend/src/features/auth/*	Login rápido PIN + admin + recuperación	§7.1, §9
frontend/src/shared/components/*	Design system interno (Button, DataTable…)	§4, §5

Archivo	Contenido principal	Sección
frontend/src/shared/layouts/*	Navbar, DashboardLayout, AuthLayout	§4
frontend/src/shared/hooks/useInactivityTimeout.js	Cierre por inactividad 10 min	§9.1, §12
frontend/src/services/authService.js	loginApi/logout	§8.4
frontend/src/features/config/api.js	API_URL (env VITE_API_URL)	§10
frontend/src/styles/tokens.css	Tokens de diseño (paleta, tipografía)	§4, §6
frontend/index.html	anti-FOUC tema, lang=es	§3, §9
manuales/Manual Administrativo.docx	Informe administrativo (este repositorio)	—
manuales/Manual de Usuarios.docx	Manual operativo del sistema	—

Tabla 45 — Índice de referencias cruzadas archivo ↔ sección del manual